

Block 3: Graph Algorithms and Searching Techniques

Short explanations, worked examples and self-check questions for every unit, with every idea also explained in Malayalam.

ഓരോ ആശയവും ലളിതമായ ഇംഗ്ലീഷിലും മലയാളത്തിലും, ഉദാഹരണങ്ങളോടെ.

UNIT 7

Advanced Trees

Search, insert, delete എന്നിവ വേഗത്തിൽ നടത്താൻ സഹായിക്കുന്ന tree-കളാണ് ഈ unit-ൽ: Binary Search Tree, balance നിലനിർത്തുന്ന AVL tree, disk-ൽ ഉപയോഗിക്കുന്ന B-tree, സ്വയം ക്രമീകരിക്കുന്ന Splay tree, നിറം ഉപയോഗിക്കുന്ന Red-Black tree, level ഉപയോഗിക്കുന്ന AA-tree, Treap എന്നിവ.

Binary search trees

Traversing a BST

Insertion into a BST

Deletion from a BST

AVL trees and the balance factor

Insertion into an AVL tree

Deletion from an AVL tree

AVL tree rotations

Applications of AVL trees

B-trees

Operations on B-trees

Applications of B-trees

Splay trees and splaying steps

Top-down splaying and the splaying algorithm

Amortized algorithm analysis

Red-black trees: properties and rotations

Insertion into a red-black tree

Deletion from a red-black tree

AA-trees

Treaps

Binary search trees

Binary Search Tree (BST): a binary tree that is either empty or in which every node has a key, all keys in its left subtree are smaller, all keys in its right subtree are larger, and both subtrees are again BSTs.

Binary Search Tree (BST): ഒന്നുകിൽ empty ആയ, അല്ലെങ്കിൽ ഓരോ node-നും ഒരു key ഉള്ളതും, left subtree-ലെ എല്ലാ key-കളും ചെറുതും right subtree-ലെ എല്ലാ key-കളും വലുതും, രണ്ട് subtree-കളും വീണ്ടും BST ആയതുമായ binary tree.

IGNOU defines a BST as a binary tree that is either **empty** or a node containing a key value, a left child and a right child. A non-empty BST satisfies three conditions: (a) keys on the left of a node are smaller than the node's key, (b) keys on the right are greater, and (c) the left and right subtrees are again BSTs. Because of this ordering, a search compares with the root and throws away one whole side at every step.

മലയാളത്തിൽ

ഒരു library-യിൽ പുസ്തകങ്ങൾ number അനുസരിച്ച് വെച്ചിരിക്കുന്നു എന്ന് കരുതുക: നോക്കുന്ന number ചെറുതാണെങ്കിൽ ഇടത്തേക്ക്, വലുതാണെങ്കിൽ വലത്തേക്ക്. BST-യും ഇതുതന്നെ. ഓരോ node-ലും ഇടതുവശത്ത് ചെറിയ key-കൾ, വലതുവശത്ത് വലിയ key-കൾ. അതുകൊണ്ട് search ചെയ്യുമ്പോൾ ഓരോ step-ലും പകുതിയോളം tree ഒഴിവാക്കാം.

left subtree keys < node key < right subtree keys (at every node)

Search / insert / delete time = $O(h)$, h = height of the tree

Best case (balanced): $h \approx \log_2 n$; worst case (keys inserted in sorted order): $h = n - 1$

Operations listed in the unit: create an empty tree, traverse, count internal (non-leaf) nodes, count external (leaf) nodes, count all nodes, find the height, insert, search, find the smallest and largest element, and delete. The smallest key is reached by going left until there is no left child; the largest by going right.

EXAMPLE

Is the tree 10 (left 7, right 15), with 12 as the right child of 7, a BST?

No. 12 is in the left subtree of 10, but $12 > 10$. Checking only parent-child pairs ($7 < 12$ is fine) is not enough; every key in the left subtree must be smaller than 10.

Exam tip: The unit's conditions (a) and (b) mention only the left and right child. Write the full rule in the exam: all keys in the left subtree are smaller and all keys in the right subtree are larger. The unit assumes distinct keys.

Traversing a BST

Traversal: visiting every node of a tree exactly once in a fixed order; for binary trees the orders are preorder (NLR), inorder (LNR) and postorder (LRN).

Traversal: ഒരു നിശ്ചിത ക്രമത്തിൽ tree-യിലെ ഓരോ node-ഉം കൃത്യം ഒരു തവണ visit ചെയ്യൽ; binary tree-യിൽ ഈ ക്രമങ്ങൾ preorder (NLR), inorder (LNR), postorder (LRN) ആണ്.

Preorder: visit the node, traverse the left subtree in preorder, then the right subtree in preorder. **Inorder:** left subtree, root, right subtree. **Postorder:** left subtree, right subtree, root. The inorder traversal of a BST always lists the keys in ascending order, which is a quick way to check your answer.

മലയാളത്തിൽ

Node-നെ എപ്പോൾ visit ചെയ്യുന്നു എന്നതാണ് വ്യത്യാസം: ആദ്യം (pre), നടുക്ക് (in), അവസാനം (post). ഒരു BST-യുടെ inorder എപ്പോഴും sorted order-ൽ വരും; അതുകൊണ്ട് exam-ൽ inorder എഴുതിക്കഴിഞ്ഞാൽ ക്രമം ശരിയാണോ എന്ന് ഉടനെ പരിശോധിക്കാം.

Preorder = Node, Left, Right (NLR)

Inorder = Left, Node, Right (LNR) → sorted keys for a BST

Postorder = Left, Right, Node (LRN)

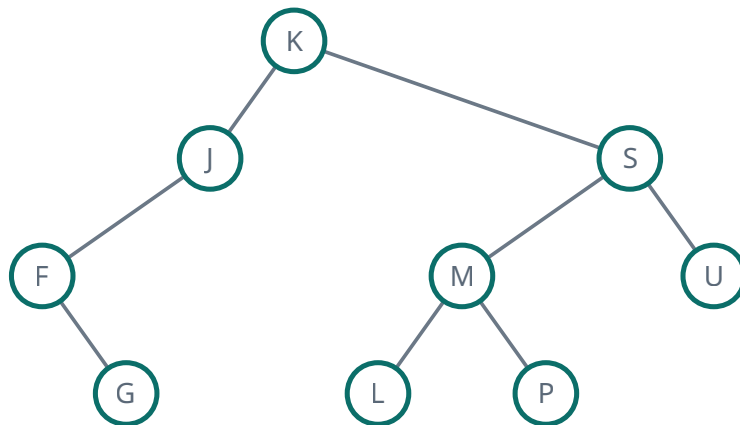


Figure 7.1: a BST with root K.

EXAMPLE (FIGURE 7.1)

K has children J and S. J has left child F, and F has right child G. S has left child M (children L, P) and right child U. Find the three traversals.

Preorder: K, then the whole left subtree J F G, then the right subtree S M L P U → **K J F G S M L P U**.

Inorder: left subtree of K gives F G J, then K, then L M P S U → **F G J K L M P S U** (alphabetical, as expected).

Postorder: G F J, then L P M U S, then K → **G F J L P M U S K**.

These agree with the unit's answers, and with the program output in the next topic.

Exam tip: Check Your Progress 1, Q1 ("different ways of traversing a BST") expects the three names plus the three-step rule for each. The first node of preorder is always the root; the last node of postorder is always the root.

Insertion into a BST

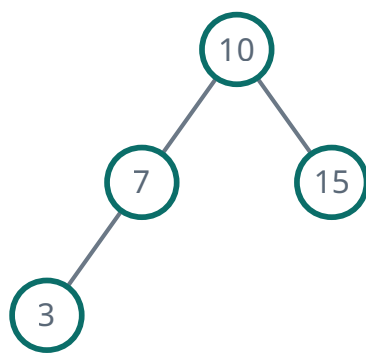
BST insertion: searching the tree from the root for the new key and attaching it as a new leaf at the empty position where the unsuccessful search ends.

BST insertion: പുതിയ key-ക്കായി root മുതൽ search ചെയ്ത്, search പരാജയപ്പെട്ട് നിൽക്കുന്ന empty സ്ഥാനത്ത് അതിനെ പുതിയ leaf ആയി ചേർക്കൽ.

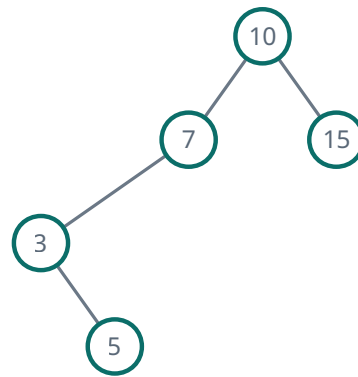
A BST is built by repeated insertion. Two tasks are done: (1) search the tree to find where the node belongs; (2) insert the node there. Insertion must keep the order: smaller keys go left, larger keys go right. A new node is always a leaf.

മലയാളത്തിൽ

Bus-ൽ സീറ്റ് നോക്കുന്നതുപോലെ: ഓരോ node-യും "ചെറുതോ വലുതോ?" എന്ന് ചോറിച്ച് ഇടത്തോട്ടോ വലത്തോട്ടോ നീങ്ങുക. ഒഴിഞ്ഞ സ്ഥലം (NULL) കിട്ടുമ്പോൾ അവിടെ പുതിയ node ഇരുത്തുക. പുതിയ node എപ്പോഴും leaf ആയിരിക്കും.



Before: Figure 7.2



After inserting 5 (Figure 7.3)

EXAMPLE (FIGURES 7.2 AND 7.3)

Insert 5 into the BST 10 (left 7, right 15), where 3 is the left child of 7.

$5 < 10 \rightarrow$ go left to 7. $5 < 7 \rightarrow$ go left to 3. $5 > 3 \rightarrow$ the right child of 3 is empty, so 5 becomes the **right child of 3**.

EXAMPLE: BUILD A BST

Insert K, J, S, F, G, M, U, L, P in this order.

K is the root. $J < K \rightarrow$ left of K. $S > K \rightarrow$ right. $F \rightarrow$ left of J. $G: < K, < J, > F \rightarrow$ right of F. $M: > K, < S \rightarrow$ left of S. $U \rightarrow$ right of S. $L: > K, < S, < M \rightarrow$ left of M. $P \rightarrow$ right of M. The result is exactly Figure 7.1.

The C program below builds this tree and performs the operations the unit lists: traversals, counting nodes, leaves and internal nodes, height, search, smallest and largest, and deletion (next topic).

```

#include <stdio.h>
#include <stdlib.h>

struct node {
    char key;
    struct node *left, *right;
};

struct node *insert(struct node *t, char k)
{
    if (t == NULL) {
        t = malloc(sizeof *t);
        t->key = k;
        t->left = t->right = NULL;
    } else if (k < t->key)
        t->left = insert(t->left, k);
    else if (k > t->key)
        t->right = insert(t->right, k);
    return t;          /* duplicates are ignored */
}

struct node *search(struct node *t, char k)
{
    while (t != NULL && t->key != k)
        t = (k < t->key) ? t->left : t->right;
    return t;
}

struct node *findmin(struct node *t)
{
    while (t != NULL && t->left != NULL)
        t = t->left;
    return t;
}

struct node *delete(struct node *t, char k)
{
    struct node *tmp;
    if (t == NULL)
        return NULL;          /* key not present */
    if (k < t->key)
        t->left = delete(t->left, k);
    else if (k > t->key)
        t->right = delete(t->right, k);
    else if (t->left != NULL && t->right != NULL) {
        tmp = findmin(t->right);    /* inorder successor */
        t->key = tmp->key;
        t->right = delete(t->right, tmp->key);
    } else {                    /* 0 or 1 child */
        tmp = t;
        t = (t->left != NULL) ? t->left : t->right;
    }
    free(tmp);
    return t;
}

```

```

        free(tmp);
    }
    return t;
}

void preorder(struct node *t)
{
    if (t) { printf("%c ", t->key); preorder(t->left); preorder(t->right); }
}

void inorder(struct node *t)
{
    if (t) { inorder(t->left); printf("%c ", t->key); inorder(t->right); }
}

void postorder(struct node *t)
{
    if (t) { postorder(t->left); postorder(t->right); printf("%c ", t->key); }
}

struct node *findmax(struct node *t)
{
    while (t != NULL && t->right != NULL)
        t = t->right;
    return t;
}

int count(struct node *t)
{
    return t ? 1 + count(t->left) + count(t->right) : 0;
}

int leaves(struct node *t)
{
    if (t == NULL) return 0;
    if (t->left == NULL && t->right == NULL) return 1;
    return leaves(t->left) + leaves(t->right);
}

int height(struct node *t)          /* height of empty tree = -1 */
{
    int hl, hr;
    if (t == NULL) return -1;
    hl = height(t->left);
    hr = height(t->right);
    return 1 + (hl > hr ? hl : hr);
}

int main(void)
{
    struct node *root = NULL;
    const char *keys = "KJSFGMULP";
    int i;
    for (i = 0; keys[i] != '\0'; i++)
        root = insert(root, keys[i]);
}

```

```

printf("Preorder : "); preorder(root); printf("\n");
printf("Inorder : "); inorder(root); printf("\n");
printf("Postorder: "); postorder(root); printf("\n");
printf("Nodes = %d, leaves = %d, internal = %d, height = %d\n",
       count(root), leaves(root), count(root) - leaves(root), height(root));
printf("Smallest = %c, largest = %c, search P: %s\n",
       findmin(root)->key, findmax(root)->key,
       search(root, 'P') ? "found" : "not found");

root = delete(root, 'K');          /* root has two children */
printf("After deleting K, inorder: "); inorder(root); printf("\n");
printf("New root = %c\n", root->key);
return 0;
}

```

```

Preorder : K J F G S M L P U
Inorder  : F G J K L M P S U
Postorder: G F J L P M U S K
Nodes = 9, leaves = 4, internal = 5, height = 3
Smallest = F, largest = U, search P: found
After deleting K, inorder: F G J L M P S U
New root = L

```

Exam tip: The unit's Figure 7.3 appears to draw 5 as the right child of 7; that is wrong, because $5 < 7$ means 5 can never be on the right of 7. Its correct place is the right child of 3. In a BST drawing, always place the new key by comparing from the root down; the new key is always a leaf.

Deletion from a BST

Inorder successor: the node with the smallest key greater than a given node's key; for a node with a right subtree it is the leftmost node of that right subtree.

Inorder successor: ഒരു node-ന്റെ key-യെക്കാൾ വലിയ key-കളിൽ ഏറ്റവും ചെറിയ key ഉള്ള node; right subtree ഉണ്ടെങ്കിൽ അത് ആ right subtree-യിലെ ഏറ്റവും ഇടത്തെ node ആണ്.

Deleting a node p has three cases. (1) p has **no children**: just remove it. (2) p has **one subtree**: its only child moves up to take its place. (3) p has **two subtrees**: its inorder successor s takes its place. The successor has no left child, so the right child of s (if any) moves up into s 's old place.

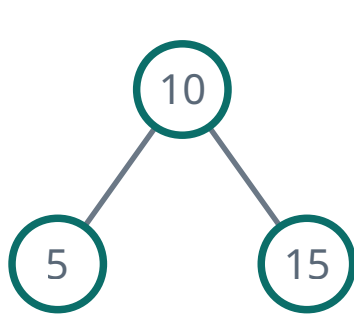
മലയാളത്തിൽ

ഒരു കുടുംബ വീട്ടിലെ കാരണവർ മാറുമ്പോൾ ആർ ആ സ്ഥാനം എടുക്കും എന്ന് ആലോചിക്കുന്നതുപോലെ. കുട്ടികളില്ലെങ്കിൽ നേരെ മാറ്റാം. ഒരു കുട്ടി മാത്രമാണെങ്കിൽ ആ കുട്ടി മുകളിലേക്ക് വരും. രണ്ട് കുട്ടികളും ഉണ്ടെങ്കിൽ order തെറ്റാതിരിക്കാൻ inorder successor (right subtree-ലെ ഏറ്റവും ചെറിയ key) ആ സ്ഥാനത്ത് വരും.

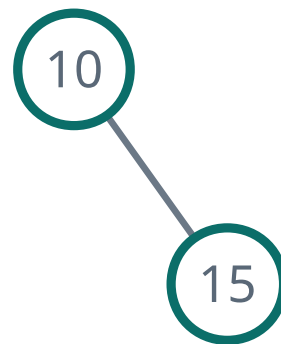
Case 1 (leaf): remove p

Case 2 (one child): parent's pointer to p $\leftarrow$ p's only child

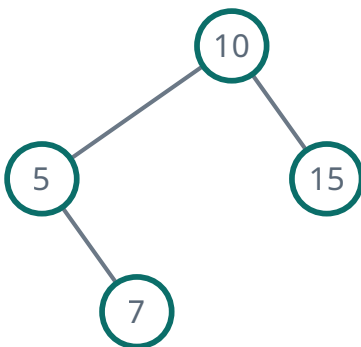
Case 3 (two children): copy successor s = leftmost node of p's right subtree into p, then delete s (s has at most a right child)



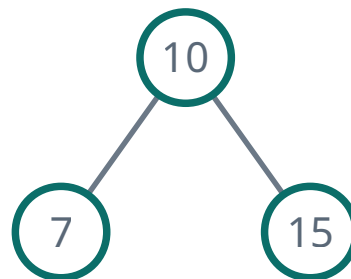
Case 1: delete leaf 5



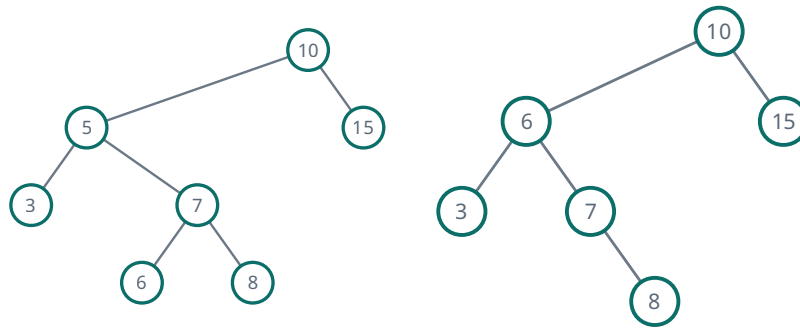
after



Case 2: delete 5 (one child)



after: 7 moves up



Case 3: delete 5 (two children) after: successor 6 takes its place

EXAMPLE (DELETE 5 IN EACH CASE)

Case 1: in 10 (5, 15), 5 is a leaf; set 10's left pointer to NULL.

Case 2 (the unit's example): in 10 (5, 15) where 5 has only the right child 7, link 7 directly to 10. Result 10 (7, 15).

Case 3 (the unit leaves this without a figure): 5 has children 3 and 7, and 7 has children 6 and 8. The successor of 5 is the leftmost node of 7's subtree, which is 6. Copy 6 into 5's place and delete the old 6 (a leaf). Result: 6 (3, 7 (-, 8)). Inorder 3 6 7 8 10 15 is still sorted.

EXAMPLE (PROGRAM ABOVE)

Delete the root K of Figure 7.1.

K has two children. Its successor is the leftmost node of S's subtree: $S \rightarrow M \rightarrow L$. L is copied into the root and the leaf L is removed. The output shows new root L and inorder F G J L M P S U.

Exam tip: You may use the inorder predecessor (rightmost node of the left subtree) instead of the successor; both keep the BST order. Say which one you use.

AVL trees and the balance factor

AVL tree: a binary search tree in which, for every node, the heights of the left and right subtrees differ by at most one.

Balance factor: for a node, $\text{height}(\text{right subtree}) - \text{height}(\text{left subtree})$ in the unit's convention; in an AVL tree it must be $-1, 0$ or $+1$.

AVL tree: ഓരോ node-യും left subtree-ന്റെയും right subtree-ന്റെയും height തമ്മിലുള്ള വ്യത്യാസം പരമാവധി ഒന്ന് മാത്രമുള്ള binary search tree.

Balance factor: unit-ന്റെ രീതിയിൽ ഒരു node-ന്റെ $\text{height}(\text{right subtree}) - \text{height}(\text{left subtree})$; AVL tree-യിൽ ഇത് $-1, 0, +1$ ഇവയിൽ ഒന്നായിരിക്കണം.

AVL stands for G. M. Adelson-Velskii and E. M. Landis, who invented it. Each node stores its key and its current balance: **left-high** (-1 , left subtree taller by one), **balanced** (0) or **right-high** ($+1$). Because the tree stays balanced, its height is $O(\log n)$, so search takes $O(\log n)$ time even in the worst case. When an insertion or deletion makes some balance factor -2 or $+2$, the tree is repaired by rotations.

മലയാളത്തിൽ

ഒരു തുലാസ് (ത്രാസ്) ഓർക്കൂ: രണ്ട് തട്ടിലും ഏകദേശം തുല്യ ഭാരം വേണം. AVL tree-യിൽ ഓരോ node-യും ഇടത്തെയും വലത്തെയും subtree-കളുടെ height വ്യത്യാസം ഒന്നിൽ കൂടരുത്. കൂടിയാൽ (± 2) rotation ചെയ്ത് വീണ്ടും balance ആക്കും. അതുകൊണ്ട് tree ഒരിക്കലും നീണ്ട വരി പോലെ ആകില്ല, search എപ്പോഴും $O(\log n)$.

$\text{bf}(\text{node}) = \text{h}(\text{right}) - \text{h}(\text{left})$ (unit's convention)

-1 = left-high, 0 = balanced, $+1$ = right-high; -2 or $+2$ = unbalanced

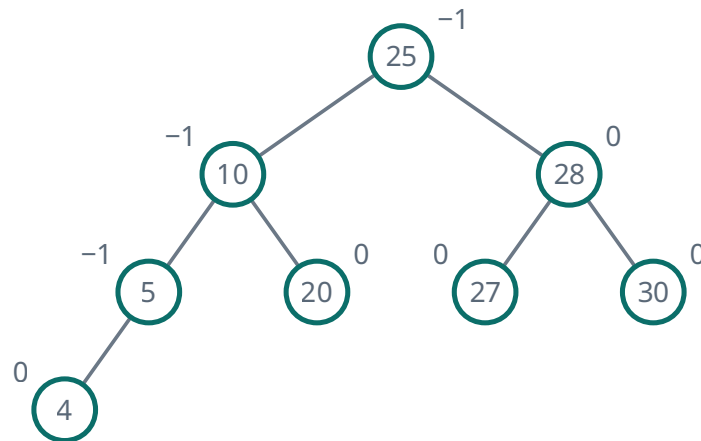
height of an empty tree = -1 , of a single node = 0

Height of an AVL tree with n nodes $< 1.44 \log_2(n + 2)$, i.e. $O(\log n)$

EXAMPLE

Find the balance factors in the tree $25 (10 (5 (4, -), 20), 28 (27, 30))$.

Leaves $4, 20, 27, 30$ have bf 0 . Node 5 : $\text{h}(\text{right}) = -1, \text{h}(\text{left}) = 0 \rightarrow \text{bf} = -1$. Node 10 : $\text{h}(\text{left}) = 1$ (the $5-4$ chain), $\text{h}(\text{right}) = 0 \rightarrow \text{bf} = -1$. Node 28 : $0 - 0 = 0$. Root 25 : $\text{h}(\text{left}) = 2, \text{h}(\text{right}) = 1 \rightarrow \text{bf} = -1$. All are in $\{-1, 0, +1\}$, so it is an AVL tree.



The tree 25 (10 (5 (4), 20), 28 (27, 30)) with balance factors

EXAMPLE

Is the tree 10 (-, 20 (-, 30)) an AVL tree?

No. At 10, $h(\text{left}) = -1$ (empty) and $h(\text{right}) = 1$, so $bf(10) = +2$.

Exam tip: Many textbooks define $bf = h(\text{left}) - h(\text{right})$, which flips the sign (left-high = +1). Either is accepted if you state your convention and use it consistently. Also note: every AVL tree is a BST, but not every BST is an AVL tree.

Insertion into an AVL tree

AVL insertion: inserting the key as in an ordinary BST, then retracing the path back towards the root, updating balance factors and applying a rotation at the first node whose balance factor becomes ± 2 .

AVL insertion: സാധാരണ BST-യിലെപ്പോലെ key insert ചെയ്ത്, root-ലേക്കുള്ള path-യുടെ തിരിച്ചുകയറി balance factor-കൾ update ചെയ്ത്, balance factor ± 2 ആകുന്ന ആദ്യ node-ൽ rotation ചെയ്യൽ.

The new node goes in as a leaf, exactly as in a BST. The algorithm then travels back along the search path and checks the balance at each node. The first (lowest) node with $bf -2$ or $+2$ is the unbalanced node, and the rotation is chosen from where the new node went relative to it: LL, RR, LR or RL (next topics). After an insertion, one single or double rotation at that node rebalances the whole tree.

മലയാളത്തിൽ

ആദ്യം BST പോലെ തന്നെ insert ചെയ്യുക. പിന്നെ താഴെ നിന്നുമുകളിലേക്ക് ഓരോ node-ന്റെയും balance factor നോക്കുക. ± 2 ആയ ആദ്യ node കണ്ടാൽ, പുതിയ node അതിന്റെ ഏത് വശത്താണ് പോയത് എന്ന് നോക്കി (LL, RR, LR, RL) rotation ചെയ്യുക. Insertion-ൽ ഒരു rotation മതി, മുഴുവൻ tree-യും ശരിയാകും.

Path from unbalanced node A to the new node:

left, left $\rightarrow$ LL $\rightarrow$ single right rotation at A

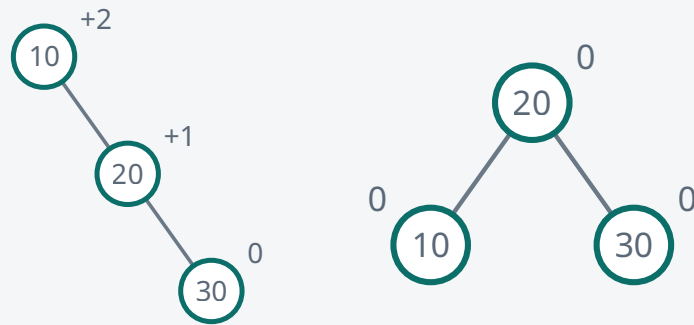
right, right $\rightarrow$ RR $\rightarrow$ single left rotation at A

left, right $\rightarrow$ LR $\rightarrow$ left rotation at A's left child, then right rotation at A

right, left $\rightarrow$ RL $\rightarrow$ right rotation at A's right child, then left rotation at A

EXAMPLE: INSERT 10, 20, 30, 25, 28, 27, 5, 4 INTO AN EMPTY AVL TREE

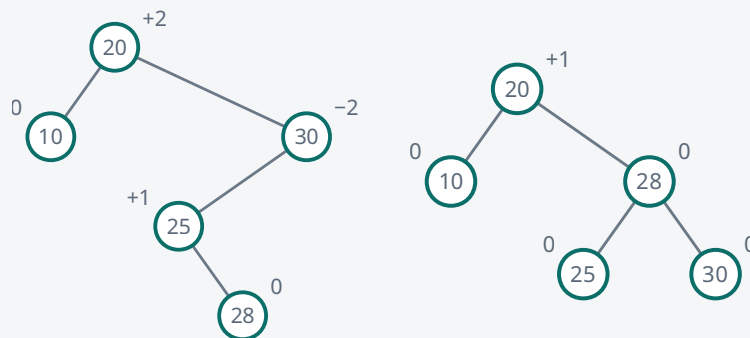
10, 20: 10 has bf +1. Insert 30: $\text{bf}(10) = +2$, path right-right → **RR**: left rotation at 10 gives 20 (10, 30).



Insert 30: +2 at 10 (RR)

After single left rotation

25 goes left of 30 ($\text{bf}(30) = -1$, $\text{bf}(20) = +1$). Insert 28: 28 goes right of 25. Going up, 30 is the first node with ± 2 ($\text{bf} -2$); the path from 30 is left (to 25) then right (to 28) → **LR**. Left rotation at 25, then right rotation at 30: 28 (25, 30). Node 20, also +2 before the fix, returns to +1.



Insert 28: -2 at 30 (LR)

After LR rotation at 30

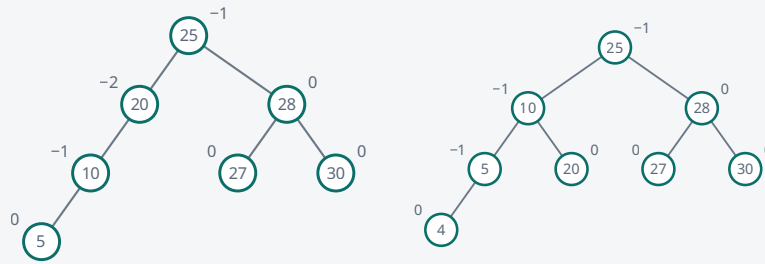
Insert 27: it goes 20 → 28 → 25 → right of 25. $\text{bf}(20) = +2$; path from 20 is right (28) then left (25) → **RL**. Right rotation at 28, then left rotation at 20: the new root is 25 with children 20 (10) and 28 (27, 30).



Insert 27: +2 at 20 (RL)

After RL rotation at 20

Insert 5: 25 → 20 → 10 → left of 10. $\text{bf}(20) = -2$, path left-left → **LL**: right rotation at 20 gives 10 (5, 20).
Insert 4: it goes left of 5; balance factors become -1 along the path, no rotation.



Insert 5: -2 at 20 (LL)

After LL at 20, then insert 4
(final)

All four rotations occurred. The program in the rotations topic prints the same rotations and the final preorder 25 10 5 4 20 28 27 30 with these balance factors.

Exam tip: Name the rotation by the path from the unbalanced node (the lowest node with ± 2) towards the new node, not from the root. Draw the tree after every insertion and write the balance factors; examiners give marks for the steps.

Deletion from an AVL tree

AVL deletion: deleting the key as in a BST (replacing a node that has two children by its inorder successor or predecessor), then checking balance factors on the path back to the root and rotating at every node whose balance factor becomes ± 2 .

AVL deletion: BST-യിലെപ്പോലെ key delete ചെയ്ത് (രണ്ട് child ഉള്ള node-നെ inorder successor അല്ലെങ്കിൽ predecessor കൊണ്ട് മാറ്റി), root വരെയുള്ള path-ൽ balance factor പരിശോധിച്ച് ± 2 ആകുന്ന എല്ലാ node-ലും rotation ചെയ്യൽ.

If the node to delete is a leaf, remove it. If it has one child, that child moves up. If it has two children, swap it with its inorder successor (or predecessor) and delete that node instead. Then, as in insertion, go back up the path to the root checking balance. Unlike insertion, a deletion may need rotations at **several** nodes on the path, because a rotation can reduce the height of the subtree and unbalance an ancestor.

മലയാളത്തിൽ

Delete ചെയ്തതിനു ശേഷം ഒരു വശം ചെറുതാകുന്നു; അപ്പോൾ മറ്റുവശം രണ്ട് level കൂടുതലാകാം. Rotation-നുള്ള case തിരഞ്ഞെടുക്കുന്നത് ഉയരം കൂടിയ വശത്തെ child-ന്റെ balance factor നോക്കിയാണ്. Insertion-ൽ നിന്ന് വ്യത്യസ്തമായി, deletion-ൽ root വരെ പല സ്ഥലത്തും rotation വേണ്ടിവന്നേക്കാം.

After deletion, at a node A with $bf(A) = -2$ (left side too tall), let B = left child:

$bf(B) = 0$ or $-1 \rightarrow$ single right rotation at A (LL)

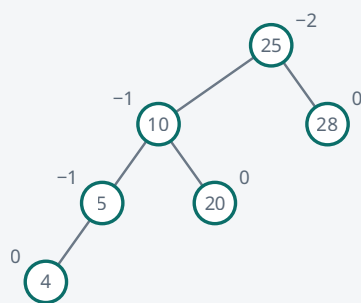
$bf(B) = +1 \rightarrow$ LR double rotation

(mirror image for $bf(A) = +2$)

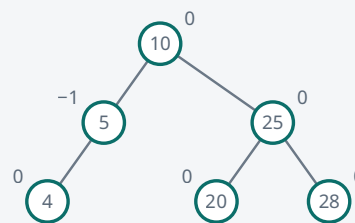
EXAMPLE

From the final tree of the insertion example, 25 (10 (5 (4), 20), 28 (27, 30)), delete 27 and then 30.

Delete 27 (a leaf): $bf(28)$ becomes $+1$; still balanced. Delete 30 (a leaf): 28 is now a leaf, so $h(\text{right of } 25) = 0$ while $h(\text{left}) = 2 \rightarrow bf(25) = -2$. The taller side is the left child 10 with $bf -1 \rightarrow$ LL case: right rotation at 25. Result: 10 (5 (4), 25 (20, 28)).



After deleting 27 and 30:
 $bf(25) = -2$



After single right rotation at
25

EXAMPLE (TWO CHILDREN)

Delete the root 25 from 25 (10 (5 (4), 20), 28 (27, 30)).

25 has two children; its inorder successor is 27. Copy 27 into the root and delete the leaf 27. Now $bf(28) = +1$ and $bf(27) = h(\text{right}) - h(\text{left}) = 1 - 2 = -1$. No rotation is needed: 27 (10 (5 (4), 20), 28 (-, 30)). (Checked with the program's delete function.)

Exam tip: The unit says a non-leaf node "has at least one child, then it must be swapped with its in-order successor or predecessor". For a node with only one child, it is simpler to let the child move up, exactly as in a BST; the swap is essential only when there are two children.

AVL tree rotations

Rotation: a local restructuring of a node and its child that changes which one is the parent while preserving the inorder (BST) order of all keys.

Single rotation: one rotation (LL or RR case). **Double rotation:** two rotations in opposite directions, first at the child and then at the unbalanced node (LR or RL case).

Rotation: എല്ലാ key-കളുടെയും inorder (BST) ക്രമം മാറ്റാതെ, ഒരു node-നെയും അതിന്റെ child-നെയും തമ്മിൽ ആർ parent എന്ന് മാറ്റുന്ന local restructuring.

Single rotation: ഒരൊറ്റ rotation (LL അല്ലെങ്കിൽ RR case). **Double rotation:** എതിർ ദിശകളിലുള്ള രണ്ട് rotation; ആദ്യം child-ൽ, പിന്നെ unbalanced node-ൽ (LR അല്ലെങ്കിൽ RL case).

There are four cases. In the figures, 7, 8 and 9 are nodes and A, B, C, D are whole subtrees. **RR** (unit's Figure 7.9): 7 was right-high; a node is inserted into C, the right subtree of its right child 8, and $bf(7)$ becomes +2. Make 8 the new root, move 7 down to the left with its left subtree A, and move B across to become 7's right child. **LL** is the mirror image. **RL** (unit's Figure 7.10): the new node is in B or C, below 8, the left child of 9, which is the right child of 7. First rotate right around 9, then rotate left around 7; node 8 comes up to the root with 7 and 9 as children. **LR** is the mirror image.

മലയാളത്തിൽ

ഒരു കയറിൽ തൂക്കിയിട്ട mobile toy ഒരു വശത്തേക്ക് ചരിഞ്ഞാൽ, നടുവിലെ പിടി മാറ്റി പിടിച്ച് balance ആക്കുന്നതുപോലെയാണ് rotation. Straight line ആയി ചരിഞ്ഞാൽ (LL, RR) ഒരു rotation മതി. വളഞ്ഞ് (zig-zag) ചരിഞ്ഞാൽ (LR, RL) ആദ്യം child-നെ നേരെയാക്കി, പിന്നെ ഒരു rotation കൂടി: അതാണ് double rotation. നടുവിലെ key മുകളിലേക്ക് വരുന്നു എന്ന് ഓർത്താൽ മതി.

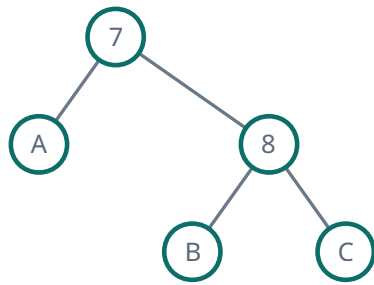
RR: $7(A, 8(B, C)) \rightarrow 8(7(A, B), C)$ (single left rotation)

LL: $8(7(A, B), C) \rightarrow 7(A, 8(B, C))$ (single right rotation)

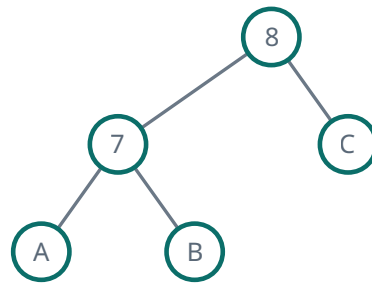
RL: $7(A, 9(8(B, C), D)) \rightarrow 7(A, 8(B, 9(C, D))) \rightarrow 8(7(A, B), 9(C, D))$

LR: $9(7(A, 8(B, C)), D) \rightarrow 9(8(7(A, B), C), D) \rightarrow 8(7(A, B), 9(C, D))$

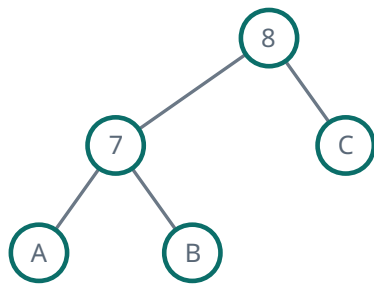
Inorder A 7 B 8 C 9 D never changes



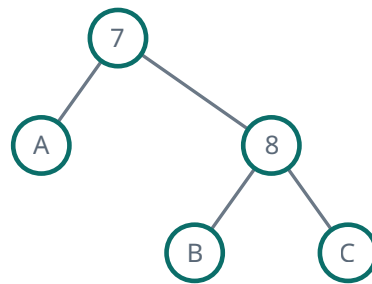
Before: C grew, $bf(7) = +2$



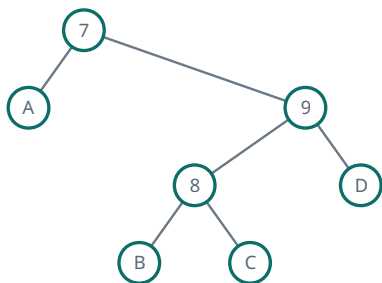
After single left rotation



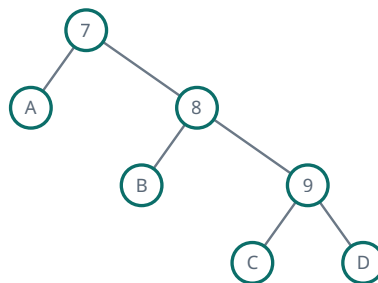
Before: A grew, $bf(8) = -2$



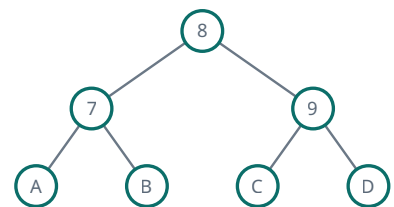
After single right rotation



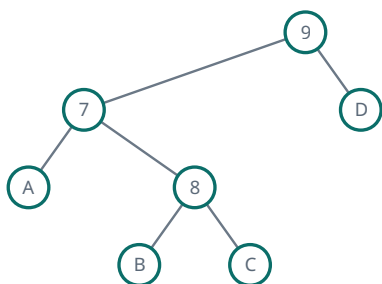
(a) node inserted in B or C



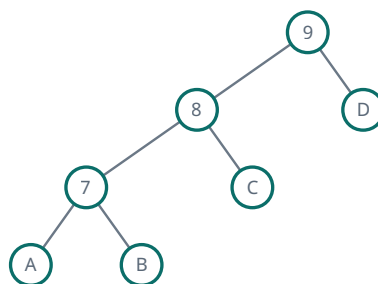
(b) right rotation at 9



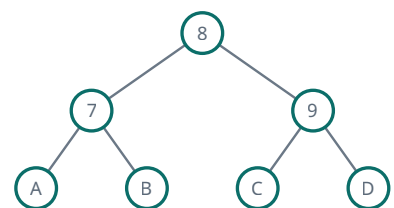
(c) left rotation at 7



(a) node inserted in B or C



(b) left rotation at 7



(c) right rotation at 9

EXAMPLE

Insert 30, 10, 20 into an empty AVL tree.

After 30 and 10, $bf(30) = -1$. 20 goes left of 30, then right of 10: $bf(30) = -2$, path left-right $\rightarrow$ LR. Left rotation at 10 gives 30 (20 (10)), then right rotation at 30 gives 20 (10, 30).

The unit represents an AVL node by the structure below. It has a bug: the pointers are declared as `struct node` * although the structure is named `struct avl`, so they would point to a different (undefined) type. Correct version:

```
struct avl {
    struct avl *left;
    int info;
    int bf;           /* balance factor */
    struct avl *right;
};
```

A complete program with insertion, deletion and all four rotations follows. It keeps a height field next to `bf` so that the balance factor can be recomputed simply as $h(right) - h(left)$ after each change.

```

#include <stdio.h>
#include <stdlib.h>

struct avl {
    struct avl *left;
    int info;
    int bf;           /* balance factor = height(right) - height(left) */
    int height;       /* height of this subtree (leaf = 0) */
    struct avl *right;
};

int ht(struct avl *t) { return t ? t->height : -1; }

void update(struct avl *t)
{
    int hl = ht(t->left), hr = ht(t->right);
    t->height = 1 + (hl > hr ? hl : hr);
    t->bf = hr - hl;
}

struct avl *rotate_right(struct avl *a) /* used for the LL case */
{
    struct avl *b = a->left;
    a->left = b->right;
    b->right = a;
    update(a); update(b);
    return b;
}

struct avl *rotate_left(struct avl *a) /* used for the RR case */
{
    struct avl *b = a->right;
    a->right = b->left;
    b->left = a;
    update(a); update(b);
    return b;
}

struct avl *rebalance(struct avl *t)
{
    update(t);
    if (t->bf == -2) { /* left subtree too tall */
        if (t->left->bf > 0) { /* LR: rotate child first */
            t->left = rotate_left(t->left);
            printf(" LR rotation at %d\n", t->info);
        } else
            printf(" LL rotation at %d\n", t->info);
        t = rotate_right(t);
    } else if (t->bf == 2) { /* right subtree too tall */
        if (t->right->bf < 0) { /* RL */
            t->right = rotate_right(t->right);

```

```

        printf(" RL rotation at %d\n", t->info);
    } else
        printf(" RR rotation at %d\n", t->info);
    t = rotate_left(t);
}
return t;
}

struct avl *insert(struct avl *t, int x)
{
    if (t == NULL) {
        t = malloc(sizeof *t);
        t->info = x;
        t->left = t->right = NULL;
        t->height = t->bf = 0;
        return t;
    }
    if (x < t->info)
        t->left = insert(t->left, x);
    else if (x > t->info)
        t->right = insert(t->right, x);
    return rebalance(t);
}

struct avl *delete(struct avl *t, int x)
{
    struct avl *s;
    if (t == NULL)
        return NULL;
    if (x < t->info)
        t->left = delete(t->left, x);
    else if (x > t->info)
        t->right = delete(t->right, x);
    else if (t->left && t->right) { /* two children */
        for (s = t->right; s->left; s = s->left)
            ; /* inorder successor */
        t->info = s->info;
        t->right = delete(t->right, s->info);
    } else { /* 0 or 1 child */
        s = t;
        t = t->left ? t->left : t->right;
        free(s);
        return t;
    }
    return rebalance(t);
}

void show(struct avl *t) /* preorder with bf */
{
    if (t == NULL) return;
    printf("%d(%+d) ", t->info, t->bf);
}

```

```

    show(t->left);
    show(t->right);
}

int main(void)
{
    int keys[] = {10, 20, 30, 25, 28, 27, 5, 4};
    int i, n = sizeof keys / sizeof keys[0];
    struct avl *root = NULL;

    for (i = 0; i < n; i++) {
        printf("insert %d\n", keys[i]);
        root = insert(root, keys[i]);
    }
    printf("Preorder: "); show(root); printf("\n");

    printf("delete 27\n"); root = delete(root, 27);
    printf("delete 30\n"); root = delete(root, 30);
    printf("Preorder: "); show(root); printf("\n");
    return 0;
}

```

```

insert 10
insert 20
insert 30
    RR rotation at 10
insert 25
insert 28
    LR rotation at 30
insert 27
    RL rotation at 20
insert 5
    LL rotation at 20
insert 4
Preorder: 25(-1) 10(-1) 5(-1) 4(+0) 20(+0) 28(+0) 27(+0) 30(+0)
delete 27
delete 30
    LL rotation at 25
Preorder: 10(+0) 5(-1) 4(+0) 25(+0) 20(+0) 28(+0)

```

Exam tip: Three mistakes in the unit's Section 7.3.3: (1) Figure 7.9 is labelled "LL Rotation", but the new node is in the right subtree of the right child, so it is the **RR case**, fixed by a single left rotation. (2) It says the height difference "became 7"; it becomes **2**. (3) For Figure 7.10 it says the left rotation brings "node 9" up; it is **node 8** that becomes the root, as panel (c) shows. Also use `struct avl *` for the pointers in the structure.

Applications of AVL trees

Height-balanced tree: a tree whose height is kept $O(\log n)$ by limiting the height difference of sibling subtrees, so that search, insertion and deletion take $O(\log n)$ time in the worst case.

Height-balanced tree: sibling subtree-കളുടെ height വ്യത്യാസം നിയന്ത്രിച്ച് height $O(\log n)$ ആയി നിലനിർത്തുന്ന tree; അതിനാൽ search, insertion, deletion എന്നിവ worst case-ലും $O(\log n)$ സമയം എടുക്കും.

AVL trees suit situations where (1) there are few insertions and deletions, (2) short search time is needed, and (3) input data is sorted or nearly sorted (which would turn a plain BST into a long chain). Uses given in the unit: dictionaries, telephone directories, compiler symbol tables and business records stored by key; applications that do many more searches and replacements than additions and deletions; and file systems. The cost of rebalancing on every update is the main drawback.

മലയാളത്തിൽ

Telephone directory പോലെ എപ്പോഴും തിരയുന്ന, വല്ലപ്പോഴും മാത്രം മാറ്റം വരുന്ന data-ക്ക് AVL tree നല്ലതാണ്. Sorted data ഒരു സാധാരണ BST-യിൽ ഇട്ടാൽ അത് ഒരു നീണ്ട വരി (linked list) പോലെയാകും; AVL അങ്ങനെ ആകാൻ അനുവദിക്കില്ല. പക്ഷേ ഓരോ insert-ലും delete-ലും rebalance ചെയ്യേണ്ടതിനാൽ update ചെലവ് കൂടുതലാണ്.

EXAMPLE

1000 keys arrive in sorted order. Compare the worst search in a plain BST and in an AVL tree.

Plain BST: every key goes to the right, height 999, up to 1000 comparisons. AVL: height at most about $1.44 \log_2 1002 \approx 14$, so at most about 15 comparisons.

B-trees

Multiway search tree of order m: an ordered tree in which each node has at most m children and $k - 1$ keys when it has k children, with keys and subtrees arranged in search-tree order.

B-tree of order m: a multiway search tree of order m in which all leaves are on the same level, every node except the root has at least $\lceil m/2 \rceil$ children (at least $\lceil m/2 \rceil - 1$ keys), and the root, if not a leaf, has at least 2 children.

Order m ഉള്ള multiway search tree: ഓരോ node-നും പരമാവധി m children ഉള്ളതും, k children ഉള്ള node-ൽ $k - 1$ key-കൾ ഉള്ളതും, key-കളും subtree-കളും search-tree ക്രമത്തിൽ ക്രമീകരിച്ചതുമായ ordered tree.

Order m ഉള്ള B-tree: എല്ലാ leaf-കളും ഒരേ level-ൽ വരുന്ന, root ഒഴികെയുള്ള ഓരോ node-നും കുറഞ്ഞത് $\lceil m/2 \rceil$ children (കുറഞ്ഞത് $\lceil m/2 \rceil - 1$ key-കൾ) ഉള്ള, leaf അല്ലാത്ത root-ന് കുറഞ്ഞത് 2 children ഉള്ള order m multiway search tree.

B-trees are balanced m -ary trees used in databases because they give guaranteed worst-case performance for insertion, deletion and retrieval. Each node holds many keys and has many subtrees, so the tree is very short. Since each node is usually one disk block, only a few nodes must be read from disk to find an item.

മലയാളത്തിൽ

ഒരു വലിയ office-ലെ file അലമാര ഓർക്കുക: ഓരോ shelf-ലും പല file-കൾ, ഓരോ shelf-ൽ നിന്നും പല ദിശകളിലേക്ക് വഴി. ഒരു node-ൽ ധാരാളം key-കൾ ഉള്ളതിനാൽ tree-യുടെ ഉയരം വളരെ കുറവാണ്; disk-ൽ നിന്ന് കുറച്ച് block-കൾ മാത്രം വായിച്ചാൽ മതി. എല്ലാ leaf-ഉം ഒരേ നിലയിലായതിനാൽ tree എപ്പോഴും balanced ആണ്.

Order m : at most m children and $m - 1$ keys per node

Non-root node: at least $\lceil m/2 \rceil$ children, at least $\lceil m/2 \rceil - 1$ keys

Root: at least 2 children (if not a leaf); a leaf root may have 1 key

All leaves on the same level

Order 5: 2 to 4 keys in each non-root node, 3 to 5 children in each non-root internal node



Figure 7.11: a B-tree of order 5. Every non-root node has 2 to 4 keys, internal nodes have 3 or 4 children, and all leaves are on the same level.

The unit's node structure:

```
struct btree {  
    int count;           /* number of keys stored in the node */  
    item_type key[3];     /* keys */  
    long branch[4];      /* child "pointers" (record numbers on disk) */  
};
```

EXAMPLE

For a B-tree of order 7, give the minimum and maximum number of keys in a non-root node.

Maximum = $m - 1 = 6$ keys (7 children). Minimum children = $\lceil 7/2 \rceil = 4$, so minimum keys = 3.

Exam tip: The unit writes "at least $m/2$ children" and "at least $(m/2) - 1$ keys" (and applies the key rule only to leaves). For odd m use the ceiling $\lceil m/2 \rceil$ and apply the key minimum to every non-root node. Also, the unit's `struct btree (key[3], branch[4])` is a node of order 4; for the order-5 tree of Figure 7.11 you would need `key[4]` and `branch[5]`.

Operations on B-trees

Node split: when a node is full ($m - 1$ keys, or $2t - 1$ keys for minimum degree t), dividing it into two nodes and moving its median key up into the parent.

Node split: ഒരു node നിറയുമ്പോൾ ($m - 1$ key-കൾ, അല്ലെങ്കിൽ minimum degree t -ക്ക് $2t - 1$ key-കൾ), അതിനെ രണ്ട് node-കളായി പിരിച്ച് നടുവിലെ (median) key-യെ parent-ലേക്ക് മുകളിലേക്ക് നീക്കൽ.

Operations: **search**, **create** and **insert** (deletion is also possible). Nodes live on secondary storage, so every node access needs a Disk-Read, and a modified node needs a Disk-Write. **Search** makes an n -way choice at each node: scan the keys linearly; stop at the first key $\geq$ the target; if equal, found; otherwise follow the child pointer just left of that key (or the last pointer). **Insert:** find the leaf, put the key there; if the node is full it is split and the median moves up. If the parent is also full it splits too, possibly up to the root, and a root split is the only way a B-tree grows taller.

മലയാളത്തിൽ

ഒരു bus നിറഞ്ഞാൽ രണ്ട് bus ആക്കി, നടുവിലെ ആളെ conductor-ന്റെ സ്ഥാനത്തേക്ക് (parent) കയറ്റുന്നതുപോലെയാണ് split. Search ചെയ്യുമ്പോൾ node-ലെ key-കൾ ഇടത്തുനിന്ന് വലത്തോട്ട് നോക്കി, ശരിയായ വിടവിലൂടെ താഴേക്ക് പോകുന്നു. Root split ആകുമ്പോൾ മാത്രമേ tree-യുടെ height കൂടുകയുള്ളൂ; അതുകൊണ്ടാണ് എല്ലാ leaf-കളും ഒരേ നിലയിൽ നിൽക്കുന്നത്.

B-TREE-SEARCH(x, k)

$i \leftarrow 1$

while $i \leq n[x]$ and $k > \text{key}_i[x]$ do $i \leftarrow i + 1$

if $i \leq n[x]$ and $k = \text{key}_i[x]$ then return (x, i)

if leaf $[x]$ then return NIL

else DISK-READ($c_i[x]$); return B-TREE-SEARCH($c_i[x], k$)

B-TREE-CREATE(T)

$x \leftarrow \text{ALLOCATE-NODE}()$; leaf $[x] \leftarrow \text{TRUE}$; $n[x] \leftarrow 0$

DISK-WRITE(x); root $[T] \leftarrow x$

B-TREE-INSERT(T, k)

$r \leftarrow \text{root}[T]$

if $n[r] = 2t - 1$ then

$s \leftarrow \text{ALLOCATE-NODE}()$; root $[T] \leftarrow s$; leaf $[s] \leftarrow \text{FALSE}$; $n[s] \leftarrow 0$; $c_1[s] \leftarrow r$

B-TREE-SPLIT-CHILD($s, 1, r$); B-TREE-INSERT-NONFULL(s, k)

else B-TREE-INSERT-NONFULL(r, k)

These algorithms (from Cormen et al.) use the **minimum degree t** : every non-root node has $t - 1$ to $2t - 1$ keys, i.e. order $m = 2t$. They split any full node met on the way down, before inserting.

EXAMPLE (FIGURE 7.12): INSERT 33, MINIMUM DEGREE $T = 3$ (AT MOST 5 KEYS)

Before:

```
      [ 10  20  30 ]
     /   |   |   \
[2 4 6] [12 15 17 19] [21 27] [32 35 36 41 53]
```

Step 1: in the root, $33 > 30$, so the path goes to the last child [32 35 36 41 53].

Step 2: that child is full ($5 = 2t - 1$ keys), so it is split: the median **36** moves up into the root, leaving [32 35] and [41 53].

Step 3: $33 < 36$, so 33 goes into [32 35], between 32 and 35.

After:

```
      [ 10  20  30  36 ]
     /   |   |   |   \
[2 4 6] [12 15 17 19] [21 27] [32 33 35] [41 53]
```

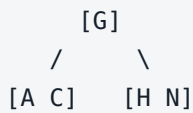
EXAMPLE (FIGURE 7.13): SEARCH FOR 21

Root [10 20 30]: $21 > 10$, $21 > 20$, $21 < 30$. Stop at 30 and follow the pointer just left of 30, to [21 27]. The first key there is 21: found at position 1.

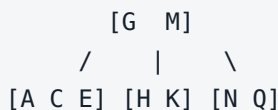
EXAMPLE (CHECK YOUR PROGRESS 3, Q1): B-TREE OF ORDER 5 FOR C N G A H E K Q M F W L T Z D P R X Y S

Order 5: at most 4 keys; a node with 5 keys is split and its middle (3rd) key moves up.

C N G A → [A C G N]. H → [A C G H N] overflows; G moves up.



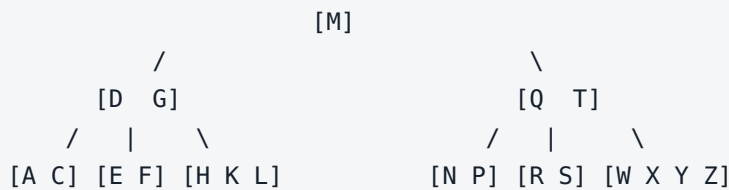
E → [A C E]; K → [H K N]; Q → [H K N Q]; M → [H K M N Q] overflows; M moves up.



F → [A C E F]; W → [N Q W]; L → [H K L]; T → [N Q T W]; Z → [N Q T W Z] overflows; T moves up: root [G M T], leaves [A C E F] [H K L] [N Q] [W Z].

D → [A C D E F] overflows; D moves up: root [D G M T], leaves [A C] [E F] [H K L] [N Q] [W Z].

P → [N P Q]; R → [N P Q R]; X → [W X Z]; Y → [W X Y Z]; S → [N P Q R S] overflows; Q moves up and the root becomes [D G M Q T], which overflows; M moves up into a new root.



This matches the unit's answer (checked by a program).

Deletion (described only in words in the unit): if removing a key leaves a node with fewer than the minimum number of keys, the violation is corrected by borrowing from a sibling or combining (merging) nodes, which may reduce the height of the tree. If the key is in an internal node, it is replaced by its predecessor or successor from a leaf first.

Exam tip: Three errors to know. (1) The key list printed in Check Your Progress 3 has S twice and no F, yet the answer key contains F; the question should read "...K Q M F W L...", which gives the tree above. With the list as printed (second S ignored as a duplicate) the result is a two-level tree: root [G M S X] with leaves [A C D E] [H K L] [N P Q R] [T W] [Y Z]. (2) In B-TREE-SEARCH, the test is $k = \text{key}_i[x]$, not $\text{key}_1[x]$. (3) In B-TREE-INSERT the old root becomes $c_1[s]$ and the call is SPLIT-CHILD(s, 1, r). Also note that Figure 7.12 is not of order 5 like Figure 7.11: its leaf [32 35 36 41 53] holds 5 keys, so read it as minimum degree $t = 3$ (order 6).

Applications of B-trees

Index: an auxiliary data structure, such as a B-tree on a key field, that lets a database locate records without scanning all of them.

Index: എല്ലാ record-ഉം scan ചെയ്യാതെ database-ന് record-കൾ കണ്ടെത്താൻ സഹായിക്കുന്ന, ഒരു key field-ലെ B-tree പോലുള്ള സഹായ data structure.

A database is a collection of data organised for easy updating, retrieval and management. Searching an unindexed database of n keys takes $O(n)$ time in the worst case. If the same data is indexed with a B-tree, the search takes $O(\log n)$ time, with only a handful of disk reads. B-trees (and variants such as B+ trees) are used in almost every database system and many file systems.

മലയാളത്തിൽ

ഒരു പുസ്തകത്തിന്റെ അവസാനത്തെ index പോലെയാണ് ഇത്: മുഴുവൻ പുസ്തകം മറിച്ചുനോക്കുന്നതിനു പകരം index നോക്കി നേരെ പേജിലേക്ക് പോകാം. Database-ൽ index ഇല്ലെങ്കിൽ $O(n)$; B-tree index ഉണ്ടെങ്കിൽ $O(\log n)$.

Height of a B-tree with n keys and minimum degree t : $h \leq \log_t((n + 1)/2)$

EXAMPLE

A B-tree with minimum degree $t = 100$ stores one million keys. At most how many levels below the root?

$h \leq \log_{100}((10^6 + 1)/2) \approx \log_{100}(500000) \approx 2.85$, so $h \leq 2$. At most 3 nodes (disk reads) are visited, compared with up to a million comparisons in an unindexed scan.

Splay trees and splaying steps

Splay tree: a self-adjusting binary search tree in which every accessed node (for search, insertion or deletion) is moved to the root by a sequence of rotations called splaying.

Zig / Zag: a single rotation bringing up a node one level, from the left / right of its parent. **Zig-zig, zag-zag, zig-zag, zag-zig:** two-level steps that move a node up past its parent and grandparent.

Splay tree: access ചെയ്യുന്ന ഓരോ node-നെയും (search, insertion, deletion) splaying എന്ന rotation-കളുടെ ഒരു പരമ്പര വഴി root-ലേക്ക് കൊണ്ടുവരുന്ന self-adjusting binary search tree.

Zig / Zag: parent-ന്റെ ഇടത് / വലത് വശത്തുള്ള ഒരു node-നെ ഒരു level മുകളിലേക്ക് കൊണ്ടുവരുന്ന single rotation. **Zig-zig, zag-zag, zig-zag, zag-zig:** parent-നെയും grandparent-നെയും കടന്ന് node-നെ രണ്ട് level മുകളിലേക്ക് കൊണ്ടുവരുന്ന step-കൾ.

New records in a BST are added as leaves, far from the root. If a record is used often, it should be near the root. A splay tree does this automatically: each access lifts the node to the root, so frequently used nodes stay near the top and rarely used ones drift down. A single access may be expensive (the tree can become unbalanced), but over a long sequence of operations the cost averages out to $O(\log n)$ per operation (amortized analysis). The key idea is to move the accessed node **two levels** at each step.

മലയാളത്തിൽ

അടുക്കളയിൽ ദിവസവും ഉപയോഗിക്കുന്ന ചായപ്പൊടി ടിൻ മുൻനിരയിലേക്ക് വരും, വർഷത്തിൽ ഒരിക്കൽ ഉപയോഗിക്കുന്ന പാത്രങ്ങൾ പിന്നിലേക്ക് പോകും. Splay tree-യും അങ്ങനെയാണ്: ഉപയോഗിച്ച node-നെ root-ലേക്ക് കയറ്റും. ഒരു operation ചിലപ്പോൾ ചെലവേറിയതാകാം, പക്ഷേ ഒരുപാട് operation-കളുടെ ശരാശരി $O(\log n)$ ആണ്.

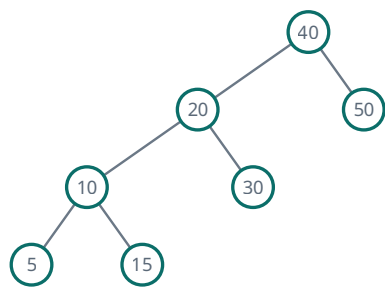
x = accessed node, p = its parent, g = grandparent

Zig: p is the root $\rightarrow$ one rotation at p

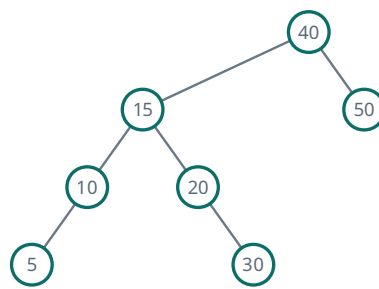
Zig-zig (x and p both left children; zag-zag: both right): rotate at g first, then at p

Zig-zag (x is a right child, p a left child; zag-zig: mirror): rotate at p , then at g

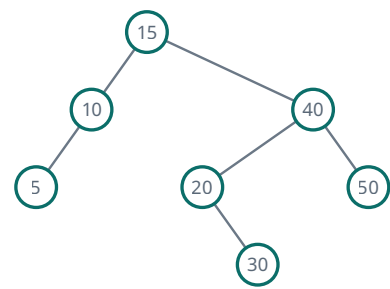
Splaying can be **bottom-up** (start at the accessed node and rotate it up to the root, as below) or **top-down** (split the tree while searching down from the root, next topic).



Before: splay at 15



After zig-zag (15, 10, 20)



After the final zig: 15 is root

EXAMPLE: BOTTOM-UP SPLAY AT 15

Tree 40 (20 (10 (5, 15), 30), 50). Splay at 15.

15 is the right child of 10, and 10 is the left child of 20 → **zig-zag**: rotate left at 10, then right at 20. 15 takes 20's place with children 10 (5) and 20 (–, 30).

Now 15's parent 40 is the root → **zig**: rotate right at 40. Result: 15 (10 (5), 40 (20 (–, 30), 50)).

EXAMPLE: ZIG-ZIG

In the same original tree, splay at 5.

5 and its parent 10 are both left children → **zig-zig**: rotate right at 20 first, then at 10: 5 (–, 10 (–, 20 (15, 30))) hangs below 40. Then **zig** at the root 40: result 5 (–, 40 (10 (–, 20 (15, 30)), 50)).

Exam tip: In zig-zig, rotate at the grandparent first, then at the parent. Doing two plain single rotations from the bottom (parent first) also brings the node to the root but loses the amortized $O(\log n)$ guarantee.

Top-down splaying and the splaying algorithm

Top-down splaying: splaying performed during the search from the root, by moving nodes smaller than the target into a left tree and larger ones into a right tree, then reassembling them around the target at the root.

Top-down splaying: root മുതൽ search ചെയ്യുന്നതിനിടെ തന്നെ, target-നേക്കാൾ ചെറിയ node-കളെ ഒരു left tree-യിലേക്കും വലിയവയെ ഒരു right tree-യിലേക്കും മാറ്റി, അവസാനം target-നെ root ആക്കി അവയെ ചേർത്തുവെക്കുന്ന splaying.

During top-down splaying the tree is kept in three parts. The **central subtree** starts as the whole tree and contains the target if present. The **left tree** (initially empty) collects nodes smaller than the target; the **right tree** (initially empty) collects larger nodes. Compare the target with the root of the central subtree: if target > root, the

root with its left subtree goes to the left tree; if $\text{target} < \text{root}$, the root with its right subtree goes to the right tree. Repeat until the target is found or a NULL is reached. Then **reassemble**: the target's left subtree becomes the right child of the largest node of the left tree, its right subtree becomes the left child of the smallest node of the right tree, and the left and right trees become the target's left and right subtrees.

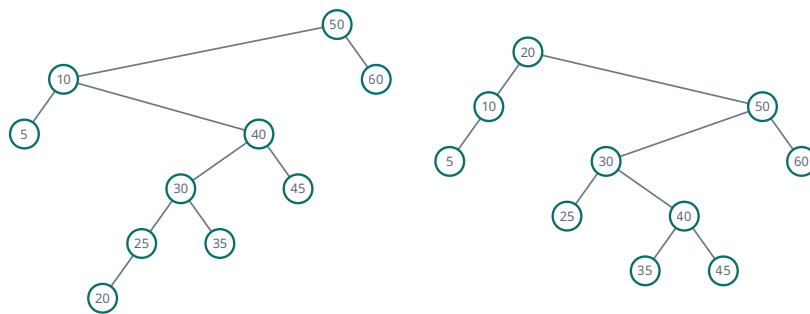
മലയാളത്തിൽ

ഒരു കൂട്ടം ആളുകളിൽ നിന്ന് ഒരാളെ തിരയുമ്പോൾ, അയാളേക്കാൾ ഉയരം കുറഞ്ഞവരെ ഇടത്തെ വരിയിലേക്കും കൂടിയവരെ വലത്തെ വരിയിലേക്കും മാറ്റിനിർത്തുന്നു എന്ന് കരുതുക. ആളെ കണ്ടെത്തിയാൽ അയാളെ നടുക്ക് നിർത്തി ഇടത്തെ വരി ഇടതുവശത്തും വലത്തെ വരി വലതുവശത്തും ചേർക്കുന്നു. ഇതാണ് top-down splaying: ഒരു യാത്രയിൽ search-ഉം splay-ഉം.

Insert(x): splay at x; if found $\rightarrow$ duplicate, keep the original; else x becomes the new root (the old root goes to its left or right)

Delete(x): splay at x; if found, remove the root, splay its left subtree at x (its maximum comes up, with no right child) and attach the old right subtree there

Reassemble: $\text{maxL.right} \leftarrow \text{target.left}$; $\text{minR.left} \leftarrow \text{target.right}$; $\text{target.left} \leftarrow L$; $\text{target.right} \leftarrow R$



Before: splay at 20

After reassembling: 20 is the root

EXAMPLE: TOP-DOWN SPLAY AT 20 (ZIG-ZAG, ZIG-ZIG, TERMINAL ZIG, AS IN THE UNIT'S FIGURE 7.15)

Step	Action	Left tree L	Right tree R	Central tree
start	–	empty	empty	whole tree (root 50)
zig-zag	20 < 50, 20 > 10: 50 (with 60) goes to R, 10 (with 5) goes to L	10 (5)	50 (–, 60)	40 (30 (25 (20), 35), 45)
zig-zig	20 < 40, 20 < 30: rotate right at 40, then 30 (with 40 (35, 45)) goes to R as the left child of 50	10 (5)	50 (30 (–, 40 (35, 45)), 60)	25 (20)
terminal zig	20 < 25 and 25's left child is 20: 25 goes to R as the left child of 30	10 (5)	50 (30 (25, 40 (35, 45)), 60)	20
reassemble	20.left ← L, 20.right ← R (20 has no subtrees to hand over)	20 (10 (5), 50 (30 (25, 40 (35, 45)), 60))		

The following C program implements top-down splay (the classic Sleator–Tarjan version), with insertion and deletion as described. It handles a zig-zag as two separate one-node links, which gives the same tree.

```

#include <stdio.h>
#include <stdlib.h>

struct node {
    int key;
    struct node *left, *right;
};

/* Top-down splay: brings key (or the last node on its search path) to the root */
struct node *splay(struct node *t, int key)
{
    struct node header, *L, *R, *y;
    if (t == NULL)
        return NULL;
    header.left = header.right = NULL;
    L = R = &header;                                /* L: max of left tree, R: min of right tree */
    for (;;) {
        if (key < t->key) {
            if (t->left == NULL) break;
            if (key < t->left->key) { /* zig-zig: rotate right first */
                y = t->left;
                t->left = y->right;
                y->right = t;
                t = y;
                if (t->left == NULL) break;
            }
            R->left = t; /* link t into the right tree */
            R = t;
            t = t->left;
        } else if (key > t->key) {
            if (t->right == NULL) break;
            if (key > t->right->key) { /* zag-zag: rotate left first */
                y = t->right;
                t->right = y->left;
                y->left = t;
                t = y;
                if (t->right == NULL) break;
            }
            L->right = t; /* link t into the left tree */
            L = t;
            t = t->right;
        } else
            break; /* found */
    }
    L->right = t->left; /* reassemble the three trees */
    R->left = t->right;
    t->left = header.right;
    t->right = header.left;
    return t;
}

```

```

struct node *insert(struct node *t, int key)
{
    struct node *n;
    t = splay(t, key);
    if (t != NULL && t->key == key)
        return t;                                /* duplicate: keep original */
    n = malloc(sizeof *n);
    n->key = key;
    if (t == NULL) {
        n->left = n->right = NULL;
    } else if (key < t->key) {
        n->left = t->left; n->right = t; t->left = NULL;
    } else {
        n->right = t->right; n->left = t; t->right = NULL;
    }
    return n;                                    /* new key becomes the root */
}

struct node *delete(struct node *t, int key)
{
    struct node *x;
    if (t == NULL)
        return NULL;
    t = splay(t, key);
    if (t->key != key)
        return t;                                /* not found */
    if (t->left == NULL) {
        x = t->right;
    } else {
        x = splay(t->left, key);                  /* max of left subtree comes up */
        x->right = t->right;
    }
    free(t);
    return x;
}

void show(struct node *t)                        /* preorder, - for empty */
{
    if (t == NULL) { printf("- "); return; }
    printf("%d ", t->key);
    if (t->left || t->right) { show(t->left); show(t->right); }
}

int main(void)
{
    /* build the example BST as a plain BST, then splay at 20 */
    int keys[] = {50, 10, 60, 5, 40, 30, 45, 25, 35, 20};
    int i;
    struct node *root = NULL, **p;
    for (i = 0; i < 10; i++) {
        p = &root;

```

```

while (*p) p = (keys[i] < (*p)->key) ? &(*p)->left : &(*p)->right;
*p = calloc(1, sizeof **p);
(*p)->key = keys[i];
}
printf("Before : "); show(root); printf("\n");
root = splay(root, 20);
printf("Splay 20: "); show(root); printf("\n");
root = insert(root, 42);
printf("Insert 42: "); show(root); printf("\n");
root = delete(root, 30);
printf("Delete 30: "); show(root); printf("\n");
return 0;
}

```

```

Before : 50 10 5 40 30 25 20 - 35 45 60
Splay 20: 20 10 5 - 50 30 25 40 35 45 60
Insert 42: 42 20 10 5 - 40 30 25 35 - 45 - 50 - 60
Delete 30: 25 20 10 5 - - 42 40 35 - 45 - 50 - 60

```

EXAMPLE: READING THE OUTPUT

The output prints each tree in preorder, with "-" for an empty child. After splaying at 20 it matches the table. Inserting 42: splaying for 42 brings 45 (the last node on the search path) to the root; $42 < 45$, so 42 becomes the new root with the old root's left subtree on its left and 45 on its right. Deleting 30: 30 is splayed to the root and removed; its left subtree is splayed so that its largest key 25 comes up, and the old right subtree is attached as 25's right child.

Exam tip: The unit's sentence on reassembly is garbled. Write it as: the largest node of the left tree gets the target's left subtree as its right child; the smallest node of the right tree gets the target's right subtree as its left child.

Amortized algorithm analysis

Amortized analysis: finding the average cost per operation over a worst-case sequence of operations, which bounds the total cost even when a single operation is expensive.

Amortized analysis: ഏറ്റവും മോശം ക്രമത്തിലുള്ള operation-കളുടെ ഒരു പരമ്പരയിൽ ഓരോ operation-ന്റെയും ശരാശരി ചെലവ് കണ്ടെത്തൽ; ഒരു operation ചെലവേറിയതാണെങ്കിലും മൊത്തം ചെലവിന് ഇത് ഒരു bound നൽകുന്നു.

Amortized analysis looks at a long sequence of operations instead of one, and gives a worst-case bound on the average. It is not an average over random inputs; it is a guarantee for every sequence. There are three methods.

Aggregate analysis: find the total worst-case cost $T(n)$ of n operations; each operation's amortized cost is $T(n)/n$ (the same for all). **Accounting method:** charge different amortized costs to different operations; the extra

charged on cheap operations is stored as prepaid credit to pay for expensive ones later. **Potential method:** like accounting, but the saved credit is stored as "potential energy" of the whole data structure. For splay trees, every operation takes $O(\log n)$ amortized time.

മലയാളത്തിൽ

മാസ ശമ്പളം കിട്ടുന്ന ഒരാൾ ഓരോ ദിവസവും കുറച്ച് പണം മാറ്റിവെച്ച് മാസാവസാനം വലിയ ബില്ലിന് അടയ്ക്കുന്നതുപോലെയാണ് accounting method. ചില ദിവസം ചെലവ് കൂടും, പക്ഷേ ശരാശരി നോക്കിയാൽ ഓരോ ദിവസത്തെയും ചെലവ് ചെറുതാണ്. Splay tree-യിൽ ഒരു operation ചിലപ്പോൾ $O(n)$ ആകാം, പക്ഷേ m operation-കൾക്ക് മൊത്തം $O(m \log n)$ മാത്രം.

Aggregate: amortized cost = $T(n)/n$

Accounting: amortized cost = actual cost + credit stored (credit never negative)

Potential: amortized cost $c'_i = c_i + \Phi(D_i) - \Phi(D_{i-1})$, with $\Phi \geq \Phi(D_0)$

Splay tree: m operations on n nodes cost $O(m \log n)$

EXAMPLE (STACK WITH MULTIPOP, AGGREGATE METHOD)

A stack supports push, pop and multipop(k), which pops up to k items. A single multipop can cost $O(n)$. Show that any sequence of n operations costs $O(n)$.

Each item is pushed once and can be popped (by pop or multipop) at most once. With at most n pushes, the total number of pops is at most n . Total cost $\leq 2n$, so $T(n)/n \leq 2$: each operation costs $O(1)$ amortized.

EXAMPLE (SAME STACK, ACCOUNTING METHOD)

Charge 2 units for push, 0 for pop and multipop.

Push uses 1 unit and leaves 1 unit of credit on the pushed item. Every pop is paid for by the credit on the item it removes, so credit never goes negative. Total amortized cost $\leq 2n = O(n)$.

Red-black trees: properties and rotations

Red-black tree: a binary search tree with one extra colour bit per node (red or black) such that the root and all NIL leaves are black, a red node has only black children, and every path from a node to its descendant leaves has the same number of black nodes.

Black height $bh(x)$: the number of black nodes on any path from, but not including, node x down to a leaf.

Red-black tree: ഓരോ node-നും ഒരു അധിക colour bit (red അല്ലെങ്കിൽ black) ഉള്ള binary search tree; root-ഉം എല്ലാ NIL leaf-കളും black ആണ്, red node-ന്റെ children black മാത്രം, ഒരു node-ൽ നിന്ന് താഴെയുള്ള leaf-കളിലേക്കുള്ള എല്ലാ path-ലും black node-കളുടെ എണ്ണം തുല്യം.

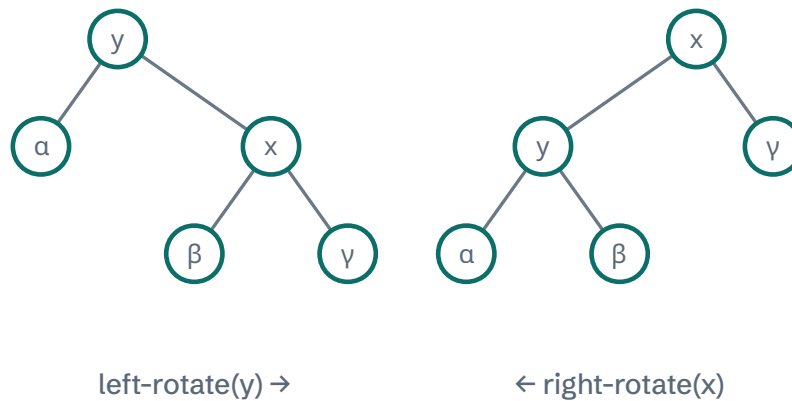
Black height $bh(x)$: node x -നെ ഉൾപ്പെടുത്താതെ, x മുതൽ ഒരു leaf വരെയുള്ള ഏത് path-ലെയും black node-കളുടെ എണ്ണം.

Each node has the fields color, key, left, right and p (parent); a missing child or parent is NIL. The colour rules keep the tree roughly balanced: no path is more than twice as long as any other, so the height is at most $2 \log_2(n + 1)$ and search is $O(\log n)$ in the worst case. INSERT and DELETE may break the rules; they are restored by **recoloring** and by **rotations**, which change the pointer structure but keep the inorder key order. **Left rotation** on y (whose right child x is not NIL): x becomes the root of the subtree, y becomes x 's left child, and x 's old left child becomes y 's right child. **Right rotation** is the mirror image.

മലയാളത്തിൽ

Red-black tree-യിൽ ഓരോ node-നും ഒരു നിറം ഉണ്ട്. പ്രധാന നിയമങ്ങൾ: root black, രണ്ട് red അടുത്തടുത്ത് (parent-child) വരുത്, എല്ലാ വഴിയിലും ഒരേ എണ്ണം black. ഇതുകൊണ്ട് ഏറ്റവും നീളമുള്ള path ഏറ്റവും ചെറിയതിന്റെ ഇരട്ടിയിൽ കൂടില്ല. നിയമം തെറ്റിയാൽ നിറം മാറ്റിയും rotation ചെയ്തും ശരിയാക്കും. AVL-നെക്കാൾ അല്പം അയഞ്ഞ balance ആണ്, പക്ഷേ update-ന് കുറച്ച് rotation മതി.

1. Every node is red or black
 2. The root is black
 3. Every leaf (NIL) is black
 4. If a node is red, both its children are black
 5. For every node, all paths from it to descendant leaves contain the same number of black nodes
- $\text{Height} \leq 2 \log_2(n + 1)$



In the rotation figure, α, β, γ are subtrees; their inorder $\alpha \ y \ \beta \ x \ \gamma$ is the same before and after.

EXAMPLE

In the red-black tree 7 (2 (1, 5 (4)), 11 (8, 14 (-, 15))) with 7, 1, 5, 8, 14 black and 2, 4, 11, 15 red, find $bh(7)$ and $bh(11)$.

From 7 down to a NIL leaf, e.g. $7 \rightarrow 2 \rightarrow 1 \rightarrow \text{NIL}$: black nodes not counting 7 are 1 and NIL, so $bh(7) = 2$.

From 11: $11 \rightarrow 8 \rightarrow \text{NIL}$ gives 8 and NIL, so $bh(11) = 2$. Every other path gives the same count (property 5).

Exam tip: The unit lists only four properties and leaves out "every leaf (NIL) is black". Write all five. In diagrams in this chapter, **black nodes are drawn with a double ring** and red nodes with a single ring.

Insertion into a red-black tree

RB insertion: inserting the key as in a BST, colouring the new node z red, and then running a fix-up that recolours and rotates until no red node has a red parent and the root is black.

RB insertion: BST-യിലെപ്പോലെ key insert ചെയ്ത്, പുതിയ node z -ന് red നിറം നൽകി, ഒരു red node-നും red parent ഇല്ലാതാക്കുകയും root black ആക്കുകയും ചെയ്യുന്നതുവരെ recolour-ഉം rotation-ഉം ചെയ്യുന്ന fix-up നടത്തൽ.

Procedure 1 inserts z as in a BST; Procedure 2 (fix-up) repairs the colours. Because z is red, black heights do not change; the only possible violations are: the root is red (z is the root), or z and its parent $P(Z)$ are both red. When $P(Z)$ is a left child, let y be z 's uncle (the other child of the grandparent):

Case 1 (uncle y red): colour $P(Z)$ and y black, colour the grandparent $P(P(Z))$ red, and move z up to the grandparent; repeat. **Case 2 (uncle black, z a right child):** set $z = P(Z)$ and left-rotate at z ; this turns it into case 3. **Case 3 (uncle black, z a left child):** colour $P(Z)$ black and $P(P(Z))$ red, then right-rotate at $P(P(Z))$; done. When $P(Z)$ is a right child, use the mirror images (swap left and right). Finally colour the root black.

മലയാളത്തിൽ

പുതിയ node എപ്പോഴും red ആയി വരുന്നു. Parent-ഉം red ആണെങ്കിൽ പ്രശ്നം. അപ്പോൾ uncle-നെ നോക്കുക: uncle red ആണെങ്കിൽ നിറം മാത്രം മാറ്റുക (case 1), പ്രശ്നം മുകളിലേക്ക് നീങ്ങും. Uncle black ആണെങ്കിൽ rotation വേണം: z വളഞ്ഞ സ്ഥാനത്താണെങ്കിൽ (case 2) ആദ്യം നേരെയൊക്കുക, പിന്നെ case 3-ൽ നിറം മാറ്റി grandparent-ൽ rotate ചെയ്യുക. Insertion-ൽ പരമാവധി രണ്ട് rotation.

Case 1: uncle red $\rightarrow$ recolour, $z \leftarrow$ grandparent (repeat)

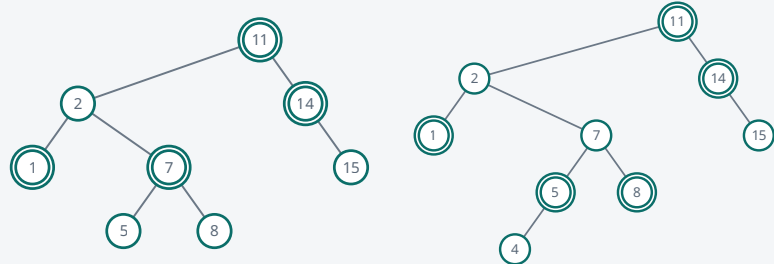
Case 2: uncle black, z is an "inner" child $\rightarrow z \leftarrow$ parent, rotate at z (becomes case 3)

Case 3: uncle black, z is an "outer" child $\rightarrow$ parent black, grandparent red, rotate at grandparent (stop)

At the end: root $\leftarrow$ black

EXAMPLE (ALL THREE CASES): INSERT 4

Tree: 11 (2 (1, 7 (5, 8)), 14 (–, 15)); black: 11, 1, 7, 14; red: 2, 5, 8, 15. Insert 4: it goes left of 5 and is red; its parent 5 is red, a violation.



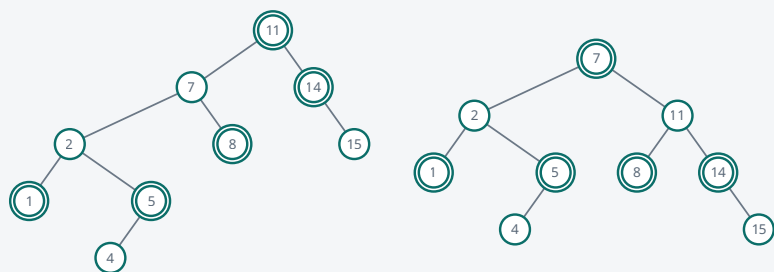
(a) valid RB tree; insert 4
below 5

(b) Case 1 at $z = 4$: now $z = 7$

Case 1: $z = 4$, parent 5 is a left child, uncle 8 is red. Colour 5 and 8 black, 7 red; $z = 7$. Now 7 and its parent 2 are both red.

Case 2: parent 2 is a left child, uncle 14 is black, and $z = 7$ is a right child. Set $z = 2$ and left-rotate at 2: 7 moves up with 2 as its left child. Now $z = 2$ is a left child with red parent 7.

Case 3: colour 7 black and 11 red, then right-rotate at 11. Final tree: 7 (2 (1, 5 (4)), 11 (8, 14 (–, 15))), with 7, 1, 5, 8, 14 black and 2, 4, 11, 15 red. Every path has black height 2.



(c) Case 2: $z = 2$, left-rotate(2)

(d) Case 3: recolour, right-rotate(11)

EXAMPLE: BUILD FROM EMPTY, INSERT 10, 20, 30

10: root, coloured black. 20: red right child of 10, no violation. 30: red child of red 20; the parent 20 is a right child, the uncle (left of 10) is NIL, which is black, and 30 is an outer child → mirror case 3: colour 20 black, 10 red, left-rotate at 10. Result: 20 black with red children 10 and 30.

Exam tip: The unit's wording has slips: in case 1 it is the **grandparent** $P(P(Z))$ that is made red and becomes the new z ; in case 3 the right rotation is done on the **grandparent** $P(P(Z))$, not on $P(Z)$. The unit's worked example (Figures 7.19 to 7.23) is lost in the extraction; the example above goes through the same three cases and was checked by a program.

Deletion from a red-black tree

RB deletion: deleting the node as in a BST and, if the node actually removed was black, running a fix-up that pushes the resulting "extra black" up the tree with recolouring and rotations until the properties hold again.

RB deletion: BST-യിലെപ്പോലെ node delete ചെയ്ത്, യഥാർത്ഥത്തിൽ നീക്കം ചെയ്ത node black ആയിരുന്നെങ്കിൽ, ഉണ്ടാകുന്ന "extra black"-നെ recolour-ഉം rotation-ഉം വഴി മുകളിലേക്ക് നീക്കി properties വീണ്ടും ശരിയാക്കുന്ന fix-up നടത്തൽ.

Procedure 1 deletes as in a BST. Let y be the node actually removed (z itself, or z 's successor) and x the node that moves into y 's place. If y was **red**, nothing breaks: no red nodes become adjacent, no black heights change, and y cannot be the root. If y was **black**, Procedure 2 runs: x carries an "extra black". The loop runs **while x is not the root and x is black**; let w be x 's sibling. For x a left child:

Case 1 (w red): colour w black and $p(x)$ red, left-rotate $p(x)$, and let w be the new sibling; this gives case 2, 3 or 4.
Case 2 (w black, both children black): colour w red and move x up to $p(x)$. **Case 3 (w black, w 's left child red, right child black):** colour w 's left child black and w red, right-rotate w , and take the new sibling; this gives case 4.
Case 4 (w black, w 's right child red): colour w with $p(x)$'s colour, make $p(x)$ and w 's right child black, left-rotate $p(x)$, and set x to the root; the loop ends. Finally colour x black. If x is a right child, use the mirror images.

മലയാളത്തിൽ

ഒരു black node നീക്കിയാൽ ആ path-ൽ ഒരു black കുറയും. ഈ കുറവ് നികത്താൻ x -ന് ഒരു "extra black" ഉണ്ടെന്ന് കരുതുന്നു. Sibling w -നെ നോക്കി നാല് case: w red ആണെങ്കിൽ rotate ചെയ്ത് black sibling ഉണ്ടാക്കുക (case 1); w -ന്റെ രണ്ട് കുട്ടികളും black ആണെങ്കിൽ w -നെ red ആക്കി പ്രശ്നം മുകളിലേക്ക് (case 2); w -ന്റെ അകത്തെ കുട്ടി red ആണെങ്കിൽ (case 3) case 4 ആക്കുക; w -ന്റെ പുറത്തെ കുട്ടി red ആണെങ്കിൽ ഒരു rotation-ൽ തീരും (case 4).

Only if the removed node y is black: fix-up on x, w = sibling of x

Case 1: w red → recolour, rotate p(x) → case 2, 3 or 4

Case 2: w black, both children black → w red, x ← p(x)

Case 3: w black, inner child red, outer child black → recolour, rotate w → case 4

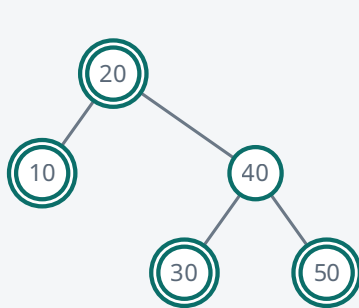
Case 4: w black, outer child red → recolour, rotate p(x), x ← root (stop)

Deletion needs at most 3 rotations

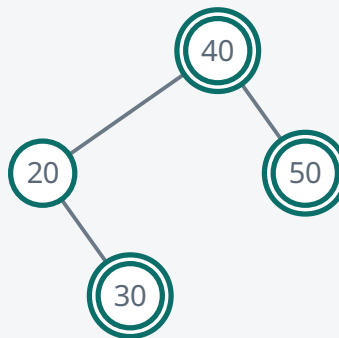
EXAMPLE: CASE 1 FOLLOWED BY CASE 2

Tree 20 (10, 40 (30, 50)) with 40 red and all others black. Delete 10.

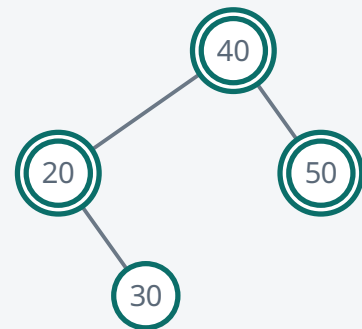
10 is black, so x = NIL (left of 20) carries an extra black, and w = 40 is red → case 1: colour 40 black, 20 red, left-rotate at 20. Now the new sibling w = 30 is black with black (NIL) children → case 2: colour 30 red, x = 20. 20 is red, so the loop stops and 20 is coloured black.



Delete 10: w = 40 is red



Case 1: recolour, left-rotate(20)



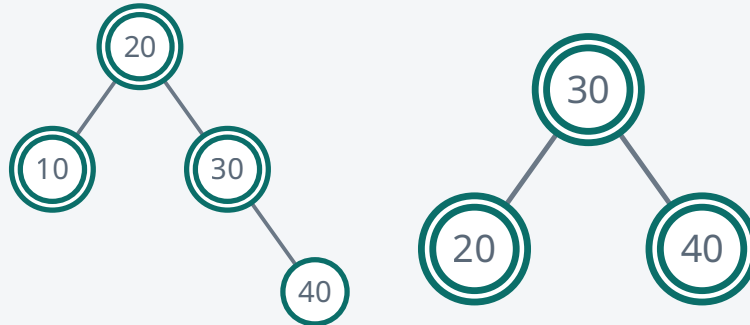
Case 2: 30 red, x = 20 → black

Result: 40 (20 (–, 30), 50), with 30 red and 40, 20, 50 black. Black height is 2 on every path.

EXAMPLE: CASE 4 (AND CASE 3)

Tree 20 (10, 30 (–, 40)) with only 40 red. Delete 10.

$x = \text{NIL}$, $w = 30$ is black and its right (outer) child 40 is red $\rightarrow$ case 4: 30 gets 20's colour (black), 20 and 40 become black, left-rotate at 20. Result: 30 (20, 40), all black.



Delete 10: $w = 30$ black, right child red

Case 4: left-rotate(20)

If instead the red child were 25, the left (inner) child of 30, case 3 applies first: colour 25 black and 30 red, right-rotate at 30, so 25 becomes the sibling with red right child 30. Then case 4 gives 25 (20, 30), all black.

Exam tip: The unit says the loop runs "until x becomes root and its color is red"; the correct condition is: continue **while** $x \neq \text{root}$ and **colour**(x) = **black** (stop when x is the root or x is red). Also, fix-up is needed only when the node physically removed is black. The deletion examples here were checked by a program that verifies all five properties.

AA-trees

AA-tree: a balanced binary search tree that stores a level in each node instead of a colour, where a left child is exactly one level below its parent, a right child is at the same level or one below, and no two consecutive horizontal links occur.

Skew / Split: skew is a right rotation that removes a left horizontal link; split is a left rotation (raising the middle node one level) that removes two consecutive right horizontal links.

AA-tree: ഓരോ node-ലും colour-ന് പകരം level സൂക്ഷിക്കുന്ന balanced binary search tree; left child parent-നെക്കാൾ കൃത്യം ഒരു level താഴെ, right child അതേ level-ലോ ഒന്ന് താഴെയോ, തുടർച്ചയായ രണ്ട് horizontal link-കൾ പാടില്ല.

Skew / Split: skew എന്നത് left horizontal link നീക്കുന്ന right rotation; split എന്നത് തുടർച്ചയായ രണ്ട് right horizontal link-കൾ നീക്കുന്ന left rotation (നടുവിലെ node ഒരു level മുകളിലേക്ക്).

Red-black deletion has many cases. An AA-tree simplifies it: instead of a colour it stores a small integer **level**. The level of a node is: 1 if it is a leaf; the same as its parent if it is red; one less than its parent if it is black. A right child at the same level as its parent forms a **horizontal link** (it corresponds to a red node). Rules: horizontal links go only to the right, and there are never two consecutive ones. Search is the ordinary BST search. Insertion always adds a new node at level 1, which can create (a) a left horizontal link or (b) two consecutive right horizontal links. On the way back up, each node calls `skew()` to fix (a) and then `split()` to fix (b). A skew can create a (b) problem and a split can create an (a) problem higher up, so they are repeated up the path.

മലയാളത്തിൽ

ഒരു കെട്ടിടത്തിന്റെ നിലകൾ പോലെ ഓരോ node-നും ഒരു level ഉണ്ട്. ഒരേ നിലയിൽ ഉള്ള രണ്ട് node-കൾ horizontal link വഴി ചേരും, പക്ഷേ അത് വലത്തോട്ട് മാത്രം, അതും ഒരു നിലയിൽ മൂന്നിൽ കൂടുതൽ node-കൾ നിരന്ന് വരരുത്. ഇടത്തോട്ട് horizontal link വന്നാൽ skew (right rotation); വലത്തോട്ട് തുടർച്ചയായി രണ്ടെണ്ണം വന്നാൽ split (left rotation, നടുവിലെ node ഒരു നില മുകളിലേക്ക്).

$\text{level}(\text{leaf}) = 1$; $\text{level}(\text{left child}) = \text{level}(\text{parent}) - 1$

$\text{level}(\text{right child}) = \text{level}(\text{parent})$ or $\text{level}(\text{parent}) - 1$

$\text{level}(\text{right grandchild}) < \text{level}(\text{node})$ (no two consecutive horizontal links)

Insert: BST insert at level 1, then on the way up: $t \leftarrow \text{skew}(t)$; $t \leftarrow \text{split}(t)$

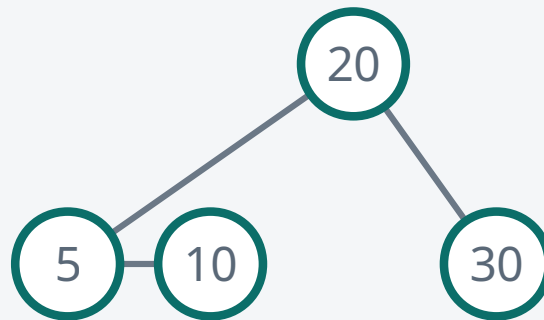
In the figures below each row is one level; a horizontal line is a horizontal link.

EXAMPLE: INSERT 30, 20, 10, 5, 15, 25, 35

30: a single node at level 1. 20: left of 30 at the same level, a left horizontal link $\rightarrow$ skew at 30: 20 $(-, 30)$.

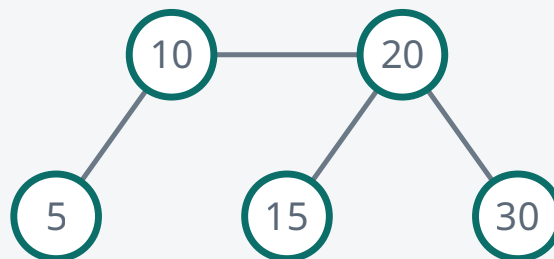
10: left of 20 $\rightarrow$ skew at 20, giving 10 $\rightarrow$ 20 $\rightarrow$ 30, two right horizontal links $\rightarrow$ split at 10: 20 rises to level 2 with children 10 and 30.

5: left of 10, same level $\rightarrow$ skew at 10: 5 $\rightarrow$ 10 horizontally.



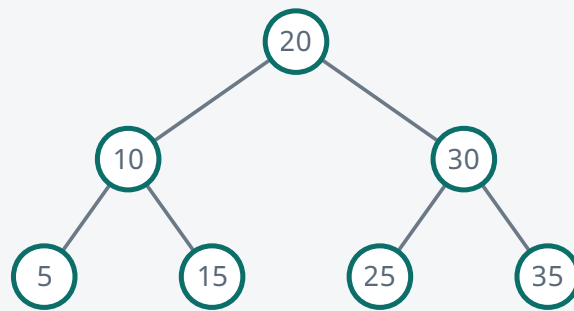
After inserting 5 (skew at 10)

15: right of 10, so 5 $\rightarrow$ 10 $\rightarrow$ 15 are consecutive horizontal links $\rightarrow$ split at 5: 10 rises to level 2. Now 10 is the left child of 20 at the same level (a left horizontal link) $\rightarrow$ skew at 20: 10 (5, 20 (15, 30)) with 10 and 20 at level 2.



After 15: split at 5, skew at 20

25: left of 30 $\rightarrow$ skew at 30: 25 $\rightarrow$ 30. 35: right of 30, giving 25 $\rightarrow$ 30 $\rightarrow$ 35 $\rightarrow$ split at 25: 30 rises to level 2. Now 10 $\rightarrow$ 20 $\rightarrow$ 30 are consecutive horizontal links at level 2 $\rightarrow$ split at 10: 20 rises to level 3.



After 35: split at 25, split at 10

Final: 20 (level 3) with children 10 and 30 (level 2), and leaves 5, 15, 25, 35 (level 1). Total: 5 skews and 4 splits (checked by a program).

Exam tip: The unit writes "right rotations through function `show()`"; it means `skew()`. Remember the pair: skew = right rotation (fixes a left horizontal link), split = left rotation plus level + 1 (fixes two right horizontal links). The unit's figures for inserting into Figure 7.28 are missing from the extraction; the steps above show the same skew/split process on a small tree.

Treaps

Treap: a binary search tree in which each node also has a randomly assigned priority, arranged so that it is a BST by keys and a heap by priorities (each node's priority is at least as large as its parent's).

Treap: ഓരോ node-നും random ആയി നൽകുന്ന ഒരു priority കൂടി ഉള്ള binary search tree; key-കളുടെ അടിസ്ഥാനത്തിൽ BST-യും priority-കളുടെ അടിസ്ഥാനത്തിൽ heap-ഉം ആയിരിക്കും (ഓരോ node-ന്റെയും priority അതിന്റെ parent-ന്റേതിനെക്കാൾ കുറഞ്ഞത് തുല്യമെങ്കിലും).

Each node stores an item, left and right pointers, and a priority chosen at random when the node is created. The name is tree + heap. To insert, place the new node as a leaf by the BST rule, then rotate it up while its priority is smaller than its parent's (to restore heap order). Because priorities are random, the expected height is $O(\log n)$, so operations take $O(\log n)$ expected time with simple code.

മലയാളത്തിൽ

Tree + heap = treap. Key നോക്കിയാൽ BST, priority നോക്കിയാൽ heap. പുതിയ node ആദ്യം BST പോലെ leaf ആയി ഇടുന്നു; അതിന്റെ priority parent-നെക്കാൾ ചെറുതാണെങ്കിൽ rotation ചെയ്ത് മുകളിലേക്ക് കയറ്റുന്നു. Priority random ആയതിനാൽ tree ശരാശരി balanced ആയിരിക്കും, ലോട്ടറി നറുക്കെടുപ്പ് പോലെ.

BST order on keys: left keys < node key < right keys

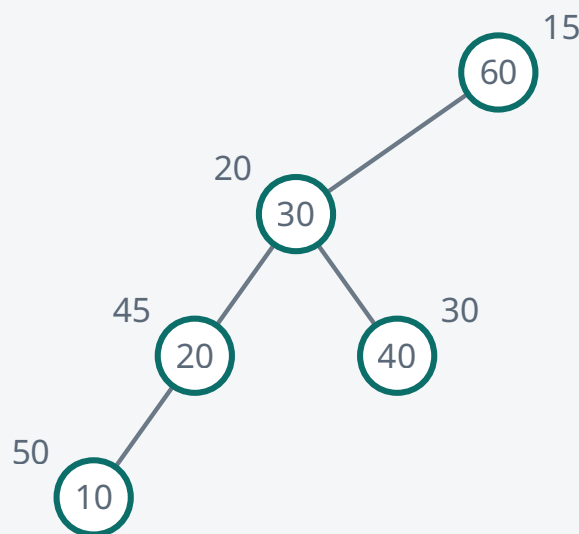
Heap order on priorities: priority(parent) $\leq$ priority(child)

Insert: BST insert, then rotate up while priority(node) < priority(parent)

EXAMPLE: INSERT (KEY, PRIORITY) = (40, 30), (20, 45), (60, 15), (10, 50), (30, 20)

40 is the root. 20 goes left of 40; 45 $\geq$ 30, no rotation. 60 goes right of 40; 15 < 30 $\rightarrow$ rotate left at 40: 60 becomes root with 40 (20) on its left. 10 goes left of 20; 50 $\geq$ 45, fine.

30 goes 60 $\rightarrow$ 40 $\rightarrow$ 20 $\rightarrow$ right of 20. 20 < 45 $\rightarrow$ rotate left at 20 (30 above 20); 20 < 30 $\rightarrow$ rotate right at 40 (30 above 40); 20 $\geq$ 15, stop. Final: 60 (30 (20 (10), 40), -).



Final treap (small numbers = priorities)

Quick recap

- BST: all left-subtree keys < node < all right-subtree keys; inorder traversal gives sorted keys.
- Preorder NLR, inorder LNR, postorder LRN; Figure 7.1 gives K J F G S M L P U / F G J K L M P S U / G F J L P M U S K.
- BST insertion always adds a leaf; deletion has 3 cases (leaf, one child, two children $\rightarrow$ inorder successor).
- AVL tree: at every node $|h(\text{right}) - h(\text{left})| \leq 1$; unit's bf = $h(R) - h(L)$, left-high = -1.

- Rotations: LL $\rightarrow$ right rotation, RR $\rightarrow$ left rotation, LR and RL $\rightarrow$ double rotations; the middle key rises.
- Insertion needs at most one (single or double) rotation; deletion may need rotations up to the root.
- B-tree of order m : $\leq m$ children, all leaves on one level, non-root nodes $\geq \lceil m/2 \rceil$ children; overflow $\rightarrow$ split, median moves up.
- B-trees minimise disk reads; used for database indexes ($O(\log n)$ search instead of $O(n)$).
- Splay tree: accessed node moved to the root by zig, zig-zig, zig-zag steps; $O(\log n)$ amortized per operation.
- Top-down splaying uses left, right and central trees and reassembles them around the target.
- Amortized analysis: aggregate ($T(n)/n$), accounting (credits), potential (Φ).
- Red-black tree: root and NIL leaves black, no red-red parent-child, equal black height; height $\leq 2 \log_2(n + 1)$.
- RB insert: new node red; case 1 recolour (uncle red), cases 2 and 3 rotate (uncle black). RB delete: 4 cases on sibling w.
- AA-tree: levels instead of colours; skew = right rotation, split = left rotation + level up. Treap: BST by key, heap by random priority.

Check yourself

Q. 1. Insert 45, 20, 60, 10, 30, 50, 70, 25 into an empty BST and write the three traversals.

Tree: 45 (20 (10, 30 (25, -)), 60 (50, 70)).

Preorder: 45 20 10 30 25 60 50 70. Inorder: 10 20 25 30 45 50 60 70. Postorder: 10 25 30 20 50 70 60 45.

Q. 2. What are the major features of a Binary Search Tree? (CYP 1, Q2)

All keys less than the root are in the left subtree, all keys greater than the root are in the right subtree, and the same rule holds for every subtree. Hence inorder traversal gives sorted order, and search, insert and delete take $O(h)$ time.

Q. 3. Define the structure of an AVL tree node in C. (CYP 2)

```
struct avl {
    struct avl *left;
    int info;
    int bf;        /* -1 left-high, 0 balanced, +1 right-high */
    struct avl *right;
};
```

The unit's answer declares the pointers as `struct node *`; they must point to `struct avl`.

Q. 4. Insert 1, 2, 3, 4, 5, 6, 7 into an empty AVL tree. Name each rotation.

3: RR at 1 $\rightarrow$ 2 (1, 3). 4: no rotation. 5: RR at 3 $\rightarrow$ 2 (1, 4 (3, 5)). 6: $bf(2) = +2$, RR at 2 $\rightarrow$ 4 (2 (1, 3), 5 (-, 6)). 7: RR at 5 $\rightarrow$ 4 (2 (1, 3), 6 (5, 7)).

The result is a perfectly balanced tree of height 2 (checked with the AVL program).

Q. 5. Define a multiway search tree of order m. (CYP 3, Q2)

An ordered tree in which each node has at most m children; a node with k children has k - 1 keys, and the keys and subtrees are arranged in search-tree order (keys in a node sorted, keys in the i-th subtree lying between the (i-1)-th and i-th keys). A B-tree is a multiway search tree with the extra balance rules.

Q. 6. In the order-5 B-tree of Figure 7.11, trace the search for K and for R.

K: root [M], $K < M \rightarrow$ child [E H]; $K > H \rightarrow$ last child [I K L]; K found. Three nodes read.

R: root [M], $R > M \rightarrow$ [P T X]; $P < R < T \rightarrow$ child [Q S]; R is not there and it is a leaf $\rightarrow$ not found (NIL).

Q. 7. Splay the tree 40 (20 (10 (5, 15), 30), 50) at 5 (bottom-up). Name the steps.

5 and its parent 10 are both left children $\rightarrow$ zig-zig: rotate at 20, then at 10, giving 5 (-, 10 (-, 20 (15, 30))) under 40. Then zig at the root 40.

Result: 5 (-, 40 (10 (-, 20 (15, 30)), 50)).

Q. 8. Insert 10, 20, 30, 15, 25, 5, 1 into an empty red-black tree.

10 black. 20 red. 30 $\rightarrow$ mirror case 3 $\rightarrow$ 20 black (10 red, 30 red). 15 $\rightarrow$ parent 10 red, uncle 30 red $\rightarrow$ case 1: 10, 30 black, 20 red, then root recoloured black. 25 $\rightarrow$ red child of black 30, fine. 5 $\rightarrow$ red child of black 10, fine. 1 $\rightarrow$ parent 5 red, uncle 15 red $\rightarrow$ case 1: 5, 15 black, 10 red; 10's parent 20 is black, stop.

Final: 20 (10 (5 (1), 15), 30 (25)); black: 20, 5, 15, 30; red: 10, 1, 25. Black height of the root is 2 (checked by program).

Q. 9. Insert 10, 20, 30, 40, 50, 60, 70 into an empty AA-tree.

30: split at 10 $\rightarrow$ 20 (10, 30), 20 at level 2. 50: 30 $\rightarrow$ 40 $\rightarrow$ 50 $\rightarrow$ split at 30 $\rightarrow$ 40 rises to level 2 (20 $\rightarrow$ 40 horizontal). 70: 50 $\rightarrow$ 60 $\rightarrow$ 70 $\rightarrow$ split at 50 $\rightarrow$ 60 rises to level 2; now 20 $\rightarrow$ 40 $\rightarrow$ 60 at level 2 $\rightarrow$ split at 20 $\rightarrow$ 40 rises to level 3.

Final: 40 (level 3), children 20 and 60 (level 2), leaves 10, 30, 50, 70 (level 1).

Q. 10. Distinguish the three methods of amortized analysis.

Aggregate: total cost $T(n)$ of n operations, each gets the same amortized cost $T(n)/n$. Accounting: different amortized costs; overcharges are stored as credit on specific objects and pay for later expensive operations. Potential: like accounting, but the credit is a potential function Φ of the whole data structure; amortized cost = actual cost + $\Delta\Phi$.

UNIT 8

Graphs

Vertices-ഉം അവയെ ബന്ധിപ്പിക്കുന്ന edges-ഉം ചേർന്ന graph എന്ന data structure ആണ് ഈ unit-ൽ. Representation, shortest path (Dijkstra, Floyd-Warshall), minimum cost spanning tree (Kruskal, Prim), BFS, DFS, strongly connected components എന്നിവ C programs സഹിതം പഠിക്കുന്നു.

Graph, directed and undirected graphs

Graph terminology

Adjacency list and adjacency matrix

Graph connectivity

Shortest path problem and relaxation

Dijkstra's algorithm

Graphs with negative edge costs

Acyclic graphs and topological order

All pairs shortest paths (Floyd-Warshall)

Minimum cost spanning trees

Kruskal's algorithm

Prim's algorithm and comparison with Kruskal

Applications of minimum cost spanning trees

Breadth first search (BFS)

Depth first search (DFS)

Strongly connected components

Graph, directed and undirected graphs

Graph: a graph $G = (V, E)$ consists of a finite set V of vertices and a set E of edges, where each edge is a pair of vertices.

Directed graph (digraph): a graph in which every edge (a, b) has a direction from a to b , so (a, b) and (b, a) are different edges.

Undirected graph: a graph whose edges have no direction, so (a, b) and (b, a) are the same edge.

Graph: $G = (V, E)$ എന്ന graph-ൽ vertices-ന്റെ ഒരു finite set V -യും edges-ന്റെ ഒരു set E -യും ഉണ്ട്; ഓരോ edge-ഉം രണ്ട് vertices-ന്റെ ഒരു pair ആണ്.

Directed graph (digraph): ഓരോ edge (a, b) -ക്കും a -യിൽ നിന്ന് b -യിലേക്ക് ഒരു direction ഉള്ള graph; അതിനാൽ (a, b) -ഉം (b, a) -ഉം വ്യത്യസ്ത edges ആണ്.

Undirected graph: edges-ന് direction ഇല്ലാത്ത graph; അതിനാൽ (a, b) -ഉം (b, a) -ഉം ഒരേ edge ആണ്.

IGNOU calls a graph "a general tree with no parent-child relationship". Vertices are the objects (cities, computers, people) and edges are the connections between them (roads, cables, friendships). In a tree every node except the root has exactly one parent; a graph has no such rule, so any vertex may be joined to any other.

മലയാളത്തിൽ

Kerala-യുടെ road map ഓർക്കുക: ഓരോ town-ഉം ഒരു vertex, രണ്ട് town-കളെ ചേർക്കുന്ന റോഡ് ഒരു edge. രണ്ട് ദിശയിലും പോകാവുന്ന സാധാരണ റോഡ് undirected edge ആണ്; ഒരു ദിശയിലേക്ക് മാത്രമുള്ള one-way റോഡ് directed edge ആണ്. Tree-യിൽ ഉള്ളതുപോലെ parent, child എന്ന ബന്ധം graph-ൽ ഇല്ല.

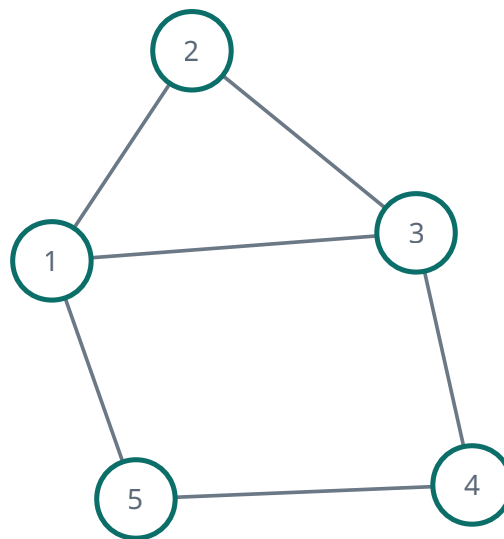


Figure 8.1: $V = \{1, 2, 3, 4, 5\}$, $E = \{(1,2), (1,5), (1,3), (5,4), (4,3), (2,3)\}$.

$G = (V, E)$

Undirected: $(a, b) = (b, a)$

Directed: $(a, b) \neq (b, a)$

EXAMPLE 1 (FIGURE 8.1)

Write V and E for the graph of Figure 8.1. Are vertices 1 and 4 directly related?

$V = \{1, 2, 3, 4, 5\}$; $E = \{(1,2), (1,5), (1,3), (5,4), (4,3), (2,3)\}$, 6 edges.

There is no edge $(1, 4)$, so 1 and 4 are not directly related. They are connected only through other vertices, for example $1 - 3 - 4$ or $1 - 5 - 4$.

EXAMPLE 2

A digraph has edges (1,5) and (5,1). How many edges is that? What if the graph is undirected?

Directed: 2 different edges ($1 \rightarrow 5$ and $5 \rightarrow 1$). Undirected: (1,5) and (5,1) name the same edge, so there is only 1 edge.

Graph terminology

Path: a sequence of vertices $p = \langle v_0, v_1, \dots, v_k \rangle$ in which each vertex is adjacent to the next; its length k is the number of edges on it.

Cycle: a path of non-zero length whose first and last vertices are the same ($v_0 = v_k$).

Spanning tree: a subgraph of a connected graph G that is a tree and contains all vertices of G ; it has $|V| - 1$ edges.

Path: ഓരോ vertex-ഉം അടുത്തതിനോട് adjacent ആയ $p = \langle v_0, v_1, \dots, v_k \rangle$ എന്ന vertices-ന്റെ sequence; അതിലെ edges-ന്റെ എണ്ണം k ആണ് length.

Cycle: ആദ്യ vertex-ഉം അവസാന vertex-ഉം ഒന്നായ ($v_0 = v_k$) non-zero length ഉള്ള path.

Spanning tree: connected graph G -യുടെ എല്ലാ vertices-ഉം ഉൾക്കൊള്ളുന്നതും ഒരു tree ആയതുമായ subgraph; അതിൽ $|V| - 1$ edges ഉണ്ട്.

These words appear in every graph question, so learn them exactly. Two vertices are **adjacent** if an edge joins them. A **simple path** has all its vertices distinct (except possibly the first and last). An **edge weight** is a cost attached to an edge, and a **loop** is an edge from a vertex to itself.

മലയാളത്തിൽ

ഒരു bus route ഓർക്കുക: നിർത്തുന്ന stops ക്രമത്തിൽ എഴുതിയാൽ അതാണ് path; എത്ര റോഡ് കഷണങ്ങൾ കടന്നു എന്നതാണ് path length. തുടങ്ങിയ stop-ൽ തന്നെ തിരിച്ചെത്തുന്ന circular bus ഒരു cycle ആണ്. ഓരോ റോഡിനുമുള്ള ticket charge ആണ് edge weight.

Term	Meaning	In Figure 8.1
Adjacent vertices	joined by an edge	5 and 4
Path	sequence of adjacent vertices	1, 5, 4 (edges (1,5), (5,4))
Path length	number of edges on the path	path 1, 5, 4 has length 2
Cycle	path with $v_0 = v_k$, length > 0	1, 2, 3, 1
Simple path	all vertices distinct (except possibly first = last)	2, 3, 4, 5
Loop	edge of the form (v, v)	none
Edge weight	cost associated with an edge	(unweighted graph)
Spanning tree	$ V - 1$ edges connecting all vertices, no cycle	(1,2), (1,3), (1,5), (5,4): 4 edges

EXAMPLE 1

In Figure 8.1, list all simple paths from 1 to 4 and their lengths.

1, 5, 4 (length 2); 1, 3, 4 (length 2); 1, 2, 3, 4 (length 3).

No other exists: from 2 the only way on is 3, and from 5 the only way on is 4. So there are three simple paths, two of length 2 and one of length 3.

EXAMPLE 2

How many edges does a spanning tree of Figure 8.1 have? Give one.

$|V| = 5$, so a spanning tree has $5 - 1 = 4$ edges. One example: (1,2), (1,3), (3,4), (4,5). It reaches every vertex and has no cycle.

Exam tip: "path length" counts edges, not vertices: $\langle v_0, \dots, v_k \rangle$ has $k + 1$ vertices but length k . A graph with no cycle is called acyclic.

Adjacency list and adjacency matrix

Adjacency list: an array adj of $|V|$ lists, where $\text{adj}[u]$ contains all vertices adjacent to u .

Adjacency matrix: a $|V| \times |V|$ matrix A with $a_{ij} = 1$ if edge $(i, j) \in E$ and $a_{ij} = 0$ otherwise.

Adjacency list: $|V|$ lists ഉള്ള adj എന്ന array; $\text{adj}[u]$ -ൽ u -വിനോട് adjacent ആയ എല്ലാ vertices-ഉം ഉണ്ട്.

Adjacency matrix: edge $(i, j) \in E$ ആണെങ്കിൽ $a_{ij} = 1$, അല്ലെങ്കിൽ $a_{ij} = 0$ ആയ $|V| \times |V|$ matrix A .

The matrix answers "is there an edge $i \rightarrow j$?" in one step but always uses n^2 cells, so it suits dense graphs. The list stores only the edges that exist (space proportional to $n + e$), so it suits sparse graphs. For an undirected graph the matrix is symmetric about the main diagonal: $A = A^T$. For a weighted graph, store the weight instead of 1.

മലയാളത്തിൽ

Matrix ഒരു school attendance register പോലെയാണ്: എല്ലാ row-column കളിലും ഉണ്ട്, ബന്ധം ഉണ്ടെങ്കിൽ 1, ഇല്ലെങ്കിൽ 0. List ഒരു phone contact list പോലെ: ഓരോ ആൾക്കും അവർ വിളിക്കുന്ന ആളുകളുടെ പേര് മാത്രം. Edges കുറവുള്ള graph-ന് list-ന് സ്ഥലം കുറച്ച് മതി.

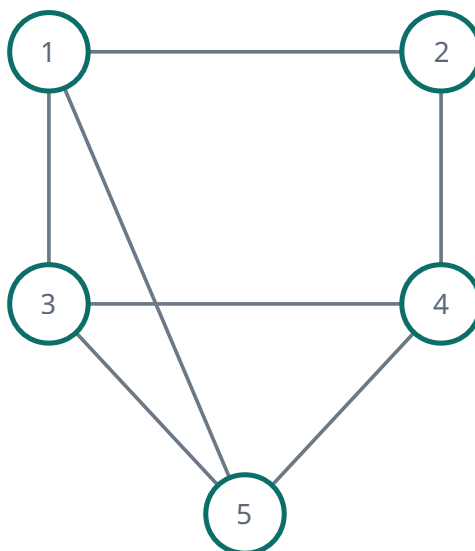


Figure 8.2: undirected graph with edges 1-2, 1-3, 1-5, 2-4, 3-4, 3-5, 4-5.

EXAMPLE 1 (FIGURE 8.2)

Write the adjacency list and adjacency matrix of Figure 8.2.

$\text{adj}[1] = \{2, 3, 5\}$

$\text{adj}[2] = \{1, 4\}$

$\text{adj}[3] = \{1, 4, 5\}$

$\text{adj}[4] = \{2, 3, 5\}$

$\text{adj}[5] = \{1, 3, 4\}$

	1	2	3	4	5
1	0	1	1	0	1
2	1	0	0	1	0
3	1	0	0	1	1
4	0	1	1	0	1
5	1	0	1	1	0

Check: every row has as many 1s as the vertex's list, and $a_{ij} = a_{ji}$ everywhere.

EXAMPLE 2 (C PROGRAM)

Build both representations of Figure 8.2 from its edge list.

```
#include <stdio.h>

#define N 5

int main(void)
{
    int edges[][2] = { {1,2}, {1,3}, {1,5}, {2,4}, {3,4}, {3,5}, {4,5} };
    int m = sizeof edges / sizeof edges[0];
    int a[N + 1][N + 1] = {{0}};    /* vertices 1..N, row/column 0 unused */
    int i, j;

    for (i = 0; i < m; i++) {        /* undirected: set both a[u][v] and a[v][u] */
        a[edges[i][0]][edges[i][1]] = 1;
        a[edges[i][1]][edges[i][0]] = 1;
    }
    printf("Adjacency matrix:\n");
    for (i = 1; i <= N; i++) {
        for (j = 1; j <= N; j++)
            printf("%d ", a[i][j]);
        printf("\n");
    }
    printf("Adjacency lists:\n");
    for (i = 1; i <= N; i++) {
        printf("adj[%d] =", i);
        for (j = 1; j <= N; j++)
            if (a[i][j])
                printf(" %d", j);
        printf("\n");
    }
    return 0;
}
```

Adjacency matrix:

```
0 1 1 0 1
1 0 0 1 0
1 0 0 1 1
0 1 1 0 1
1 0 1 1 0
```

Adjacency lists:

```
adj[1] = 2 3 5
adj[2] = 1 4
adj[3] = 1 4 5
adj[4] = 2 3 5
adj[5] = 1 3 4
```

Exam tip: the unit's matrix for Figure 8.2 has row 2 as 1 0 0 1 1, which puts an edge 2–5. That contradicts both the list $\text{adj}[2] = \{1, 4\}$ and row 5 (whose column 2 is 0), and makes the matrix non-symmetric. The correct row 2 is 1 0 0 1 0. Quick check in the exam: an undirected graph's matrix must equal its transpose.

Graph connectivity

Connected graph: an undirected graph in which a path exists between every pair of vertices.

Strongly connected graph: a directed graph in which, for every pair of vertices u, v , there is a directed path from u to v and from v to u .

Weakly connected graph: a directed graph that is not strongly connected but whose underlying undirected graph is connected. **Complete graph:** a graph with an edge between every pair of vertices.

Connected graph: എല്ലാ vertex pair-നും ഇടയിൽ ഒരു path ഉള്ള undirected graph.

Strongly connected graph: ഏത് vertices u, v എടുത്താലും u -ൽ നിന്ന് v -യിലേക്കും v -യിൽ നിന്ന് u -യിലേക്കും directed path ഉള്ള directed graph.

Weakly connected graph: strongly connected അല്ലാത്തതും, directions ഒഴിവാക്കിയാൽ connected ആകുന്നതുമായ directed graph. **Complete graph:** എല്ലാ vertex pair-നും ഇടയിൽ edge ഉള്ള graph.

For directed graphs we must respect the arrows. If every vertex can reach every other along the arrows, the digraph is strongly connected. If that fails but the graph becomes connected once you ignore the arrows (the "underlying graph"), it is weakly connected. A complete undirected graph on n vertices has $n(n - 1)/2$ edges.

മലയാളത്തിൽ

ഒരു നഗരത്തിൽ എല്ലാം one-way റോഡുകളാണെന്ന് കരുതുക. ഏത് junction-ൽ നിന്നും ഏത് junction-ലേക്കും നിയമം തെറ്റിക്കാതെ എത്താമെങ്കിൽ strongly connected. നിയമം നോക്കാതെ (arrow മറന്ന്) നടന്നാൽ മാത്രമേ എത്തൂ എങ്കിൽ weakly connected.

Complete undirected graph K_n : $n(n - 1)/2$ edges

Complete digraph on n vertices: $n(n - 1)$ edges

EXAMPLE 1

Digraph with edges $1 \rightarrow 2, 2 \rightarrow 3, 3 \rightarrow 1$. Strongly or weakly connected? What if $3 \rightarrow 1$ is reversed to $1 \rightarrow 3$?

With $3 \rightarrow 1$: $1 \rightarrow 2 \rightarrow 3 \rightarrow 1$ is a cycle through all vertices, so each reaches each: **strongly connected**.

With $1 \rightarrow 3$ instead: nothing leaves 3, so 3 cannot reach 1. Ignoring directions it is still connected, so it is **weakly connected**.

EXAMPLE 2

How many edges does a complete undirected graph on 5 vertices have?

$5 \times 4 / 2 = 10$ edges.

Exam tip: the unit says a strongly connected graph has "every pair of distinct vertices connected with each other". Write the precise version: a directed path in both directions between every pair.

Shortest path problem and relaxation

Single source shortest path problem: given a weighted graph $G = (V, E)$ and a source $s \in V$, find a shortest (minimum total weight) path from s to every vertex $v \in V$.

Relaxation: for an edge (u, v) , if $d[u] + w(u, v) < d[v]$, set $d[v] = d[u] + w(u, v)$ and $pi[v] = u$.

Single source shortest path problem: weighted graph $G = (V, E)$ -ഉം source $s \in V$ -യും തന്നാൽ, s -ൽ നിന്ന് ഓരോ vertex $v \in V$ -യിലേക്കും ഏറ്റവും കുറഞ്ഞ total weight ഉള്ള path കണ്ടെത്തുക.

Relaxation: edge (u, v) -ക്ക് $d[u] + w(u, v) < d[v]$ ആണെങ്കിൽ $d[v] = d[u] + w(u, v)$ -ഉം $pi[v] = u$ -ഉം ആക്കുക.

The weight of a path is the sum of its edge weights; it can be distance, time or cost. Every shortest path algorithm keeps an estimate $d[v]$ (an upper bound on the true distance) and lowers it by relaxation until it equals the shortest path weight. A useful fact: any part of a shortest path is itself a shortest path.

മലയാളത്തിൽ

Thrissur-ൽ നിന്ന് Kochi-ക്ക് ഇപ്പോൾ അറിയാവുന്ന ഏറ്റവും നല്ല വഴി 80 km എന്ന് കരുതുക. Chalakudy വഴി പോയാൽ 75 km മതി എന്ന് കണ്ടാൽ, estimate 75 ആക്കി "Chalakudy വഴി" എന്ന് എഴുതിവയ്ക്കുന്നു. ഇതാണ് relaxation: d കുറയ്ക്കുക, predecessor (pi) മാറ്റുക.

relax(u, v):

if $d[v] > d[u] + w(u, v)$ then

$d[v] = d[u] + w(u, v)$; $pi[v] = u$

EXAMPLE

$d[u] = 5$, $d[v] = 14$, $w(u, v) = 3$. Relax (u, v). Then relax again with $d[v] = 7$.

First: $5 + 3 = 8 < 14$, so $d[v]$ becomes 8 and $pi[v] = u$.

Second: $5 + 3 = 8$ is not < 7 , so nothing changes.

Exam tip: the unit's answer key says "Node relaxation" is the basis of Dijkstra's algorithm. The standard term is edge relaxation (we relax an edge (u, v)); writing "relaxation" alone is safe.

Dijkstra's algorithm

Dijkstra's algorithm: a greedy algorithm that solves the single source shortest path problem for a graph with non-negative edge weights by repeatedly adding the closest unfinished vertex to S and relaxing its outgoing edges.

Dijkstra's algorithm: non-negative edge weights ഉള്ള graph-ൽ single source shortest path problem പരിഹരിക്കുന്ന greedy algorithm; ഏറ്റവും അടുത്തുള്ള, തീരാത്ത vertex-നെ S-ൽ ചേർത്ത് അതിന്റെ outgoing edges relax ചെയ്യുന്നത് ആവർത്തിക്കുന്നു.

The algorithm keeps two sets: S, the vertices whose shortest distance is final, and $Q = V - S$. It also keeps $d[v]$ (best estimate) and $pi[v]$ (predecessor). Start with $d[s] = 0$ and every other $d = \infty$. Each round, pick the vertex u in Q with the smallest d (Extract-Min), move it to S, and relax every edge (u, v). Because no weight is negative, no later path can beat $d[u]$, so it is final. Following pi backwards from any vertex gives its shortest path.

മലയാളത്തിൽ

ഒരു കല്യാണ വീട്ടിലേക്ക് ക്ഷണക്കത്ത് കൊടുക്കാൻ പോകുന്നു എന്ന് കരുതുക: ഇതുവരെ എത്തിയ സ്ഥലങ്ങളിൽ നിന്ന് ഏറ്റവും അടുത്ത വീട് ആദ്യം. ആ വീട്ടിലെത്തിയാൽ അവിടെ നിന്നുള്ള റോഡുകൾ വഴി മറ്റ് വീടുകളിലേക്കുള്ള ദൂരം പുതുക്കുന്നു (relax). ഓരോ step-ലും ഏറ്റവും അടുത്തത് എടുക്കുന്നതുകൊണ്ടാണ് ഇത് greedy.

for each $v \in V$: $d[v] = \infty$, $pi[v] = \text{nil}$

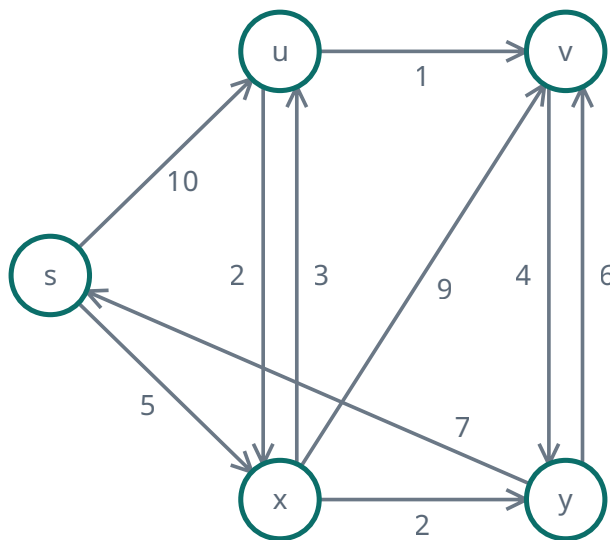
$d[s] = 0$; $S = \emptyset$; $Q = V$

while Q is not empty:

$u = \text{Extract-Min}(Q)$; $S = S \cup \{u\}$

for each edge (u, v) : $\text{relax}(u, v)$

Time with an array (linear search for Extract-Min): $\Theta(n^2)$



The unit's worked digraph: $s \rightarrow u$ 10, $s \rightarrow x$ 5, $u \rightarrow v$ 1, $u \rightarrow x$ 2, $x \rightarrow u$ 3, $x \rightarrow v$ 9, $x \rightarrow y$ 2, $v \rightarrow y$ 4, $y \rightarrow v$ 6, $y \rightarrow s$ 7.

EXAMPLE 1 (THE UNIT'S STEP-BY-STEP EXAMPLE)

Run Dijkstra from s on the graph above.

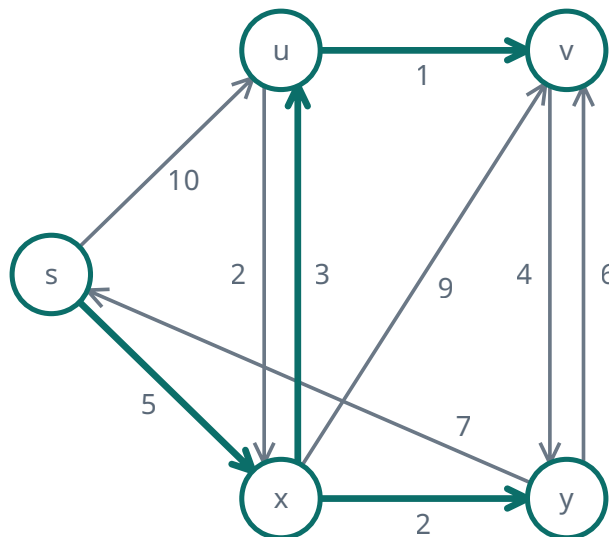
Step	Vertex added to S	d(s)	d(u)	d(v)	d(x)	d(y)
Start	—	0	∞	∞	∞	∞
1	s	0	10	∞	5	∞
2	x (5)	0	8	14	5	7
3	y (7)	0	8	13	5	7
4	u (8)	0	8	9	5	7
5	v (9)	0	8	9	5	7

Step 2: from x, u improves $10 \rightarrow 5 + 3 = 8$, v becomes $5 + 9 = 14$, y becomes $5 + 2 = 7$.

Step 3: from y, v improves $14 \rightarrow 7 + 6 = 13$.

Step 4: from u, v improves $13 \rightarrow 8 + 1 = 9$.

Predecessors: $\text{pi}[x] = s$, $\text{pi}[y] = x$, $\text{pi}[u] = x$, $\text{pi}[v] = u$. Shortest paths: $s \rightarrow x$ (5), $s \rightarrow x \rightarrow y$ (7), $s \rightarrow x \rightarrow u$ (8), $s \rightarrow x \rightarrow u \rightarrow v$ (9).



Highlighted edges: the shortest path tree given by pi.

EXAMPLE 2 (C PROGRAM, ARRAY VERSION)

```

#include <stdio.h>

#define N 5
#define INF 1000000

int main(void)
{
    char name[N] = {'s', 'u', 'v', 'x', 'y'};
    /* w[i][j] = weight of edge i -> j, 0 = no edge */
    int w[N][N] = {
        /*      s   u   v   x   y */
        /* s */ { 0, 10, 0, 5, 0 },
        /* u */ { 0, 0, 1, 2, 0 },
        /* v */ { 0, 0, 0, 0, 4 },
        /* x */ { 0, 3, 9, 0, 2 },
        /* y */ { 7, 0, 6, 0, 0 }
    };

    int d[N], pi[N], inS[N];
    int i, k, u, v, src = 0;

    for (v = 0; v < N; v++) {
        d[v] = INF;
        pi[v] = -1;
        inS[v] = 0;
    }
    d[src] = 0;

    for (k = 0; k < N; k++) {
        u = -1; /* Extract-Min by linear search */
        for (i = 0; i < N; i++)
            if (!inS[i] && (u == -1 || d[i] < d[u]))
                u = i;
        if (d[u] == INF)
            break; /* rest are unreachable */
        inS[u] = 1; /* add u to S */
        printf("Add %c (d = %d)\n", name[u], d[u]);
        for (v = 0; v < N; v++) /* relax edges leaving u */
            if (w[u][v] && !inS[v] && d[u] + w[u][v] < d[v]) {
                d[v] = d[u] + w[u][v];
                pi[v] = u;
            }
    }

    printf("\nVertex  d  pi\n");
    for (v = 0; v < N; v++)
        printf("  %c    %2d  %c\n", name[v], d[v], pi[v] == -1 ? '-' : name[pi[v]]);
    return 0;
}

```

```
Add s (d = 0)
Add x (d = 5)
Add y (d = 7)
Add u (d = 8)
Add v (d = 9)
```

Vertex	d	pi
s	0	-
u	8	x
v	9	u
x	5	s
y	7	x

Exam tip: the unit says Dijkstra's algorithm "finds shortest path between all pairs of vertices". That is wrong: it finds shortest paths from one source to all vertices. For all pairs you run it once per source or use Floyd-Warshall. Also, its pseudocode sets $S := \{0\}$; it should be $S := \emptyset$ (the empty set), and $d[s] := 0$ belongs after the initialisation loop, not inside it.

Graphs with negative edge costs

Negative weight cycle: a cycle whose edge weights add up to a negative number; if it is reachable from the source, shortest paths through it are not defined.

Negative weight cycle: edge weights കൂട്ടിയാൽ negative ആകുന്ന cycle; source-ൽ നിന്ന് അതിൽ എത്താമെങ്കിൽ അതുവഴിയുള്ള shortest paths define ചെയ്യാനാവില്ല.

Negative edges can appear in two situations: (1) a negative edge but every cycle has non-negative total weight, and (2) a negative edge that creates a negative weight cycle reachable from the source. In case (1) shortest paths exist; in case (2) you can go round the cycle again and again, lowering the cost without limit, so there is no shortest path. Dijkstra's algorithm works only when all weights are non-negative.

മലയാളത്തിൽ

ഓരോ തവണ ഒരു റൗണ്ട് ഓടുമ്പോഴും കൈയ്യിൽ 3 രൂപ കൂടുതൽ കിട്ടുന്ന ഒരു വഴി ഉണ്ടെന്ന് കരുതുക. എത്ര റൗണ്ട് ഓടിയാലും ഇനിയും ലാഭം കൂട്ടാം, അതിനാൽ "ഏറ്റവും കുറഞ്ഞ cost" എന്ന ഒരു ഉത്തരം ഇല്ല. ഇതാണ് negative weight cycle.

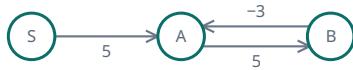


Figure 8.6: cycle $A \rightarrow B \rightarrow A$ has weight $5 + (-3) = 2$ (non-negative).



Figure 8.7: cycle $A \rightarrow B \rightarrow A$ has weight $5 + (-8) = -3$ (negative).

EXAMPLE 1 (FIGURES 8.6 AND 8.7)

Find shortest distances from S in both graphs.

Figure 8.6: $d(S) = 0$, $d(A) = 5$, $d(B) = 10$. Going round the cycle adds +2, so it never helps; shortest paths are well defined.

Figure 8.7: path S, A, B costs 10; S, A, B, A, B costs $5 + 5 - 8 + 5 = 7$; each extra round lowers it by 3 more (4, 1, -2, ...). No minimum exists, so the shortest paths to A and B are not defined ($-\infty$).

EXAMPLE 2 (DIJKSTRA FAILS EVEN WITHOUT A NEGATIVE CYCLE)

Edges $S \rightarrow A$ (2), $S \rightarrow B$ (3), $B \rightarrow A$ (-2). Run Dijkstra from S.

After S: $d(A) = 2$, $d(B) = 3$. A is closest, so A is fixed with $d(A) = 2$. Then B is added and relaxing $B \rightarrow A$ gives $3 - 2 = 1$, but A is already final. Dijkstra reports 2; the true shortest distance is 1 ($S \rightarrow B \rightarrow A$).

Exam tip: the unit lists both cases as "negative edge with non-negative weight cycle"; the second should read "negative edge with negative weight cycle reachable from the source". For graphs with negative edges (outside this unit), the Bellman-Ford algorithm is used; it also detects negative cycles.

Acyclic graphs and topological order

Acyclic graph: a graph with no cycle; a directed acyclic graph is called a DAG.

Topological order: a linear ordering of the vertices of a DAG such that for every edge (u, v) , u comes before v .

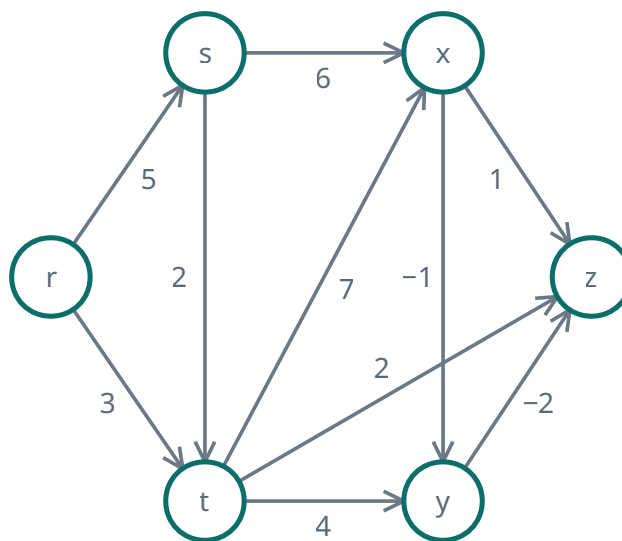
Acyclic graph: ഒരു cycle-ഉം ഇല്ലാത്ത graph; directed acyclic graph-നെ DAG എന്ന് വിളിക്കുന്നു.

Topological order: ഓരോ edge (u, v) -ക്കും u, v -ക്ക് മുമ്പ് വരുന്ന രീതിയിൽ DAG-ന്റെ vertices-നെ ഒരു വരിയിൽ നിരത്തുന്നത്.

A path $(A, B, C, \dots, P)$ forms a cycle if $A = P$; a graph with no such path is acyclic. Every DAG has at least one topological order. One way to get it: run DFS and list vertices in decreasing order of finishing time. In a DAG, shortest paths can be found even with negative edges: relax the edges of each vertex in topological order, in $O(n + e)$ time.

മലയാളത്തിൽ

Course prerequisites ഓർക്കുക: C പഠിച്ചിട്ടേ Data Structures പഠിക്കാവൂ, അതിനു ശേഷമേ Algorithms. ഈ നിബന്ധനകൾ തെറ്റിക്കാതെ subjects ഒരു വരിയിൽ എഴുതുന്നതാണ് topological order. ഇവിടെ cycle ഉണ്ടെങ്കിൽ ("A-ക്ക് മുമ്പ് B, B-ക്ക് മുമ്പ് A") ഒരു ക്രമവും സാധ്യമല്ല.



A weighted DAG: $r \rightarrow s$ 5, $r \rightarrow t$ 3, $s \rightarrow t$ 2, $s \rightarrow x$ 6, $t \rightarrow x$ 7, $t \rightarrow y$ 4, $t \rightarrow z$ 2, $x \rightarrow y$ -1, $x \rightarrow z$ 1, $y \rightarrow z$ -2.

EXAMPLE 1

Find a topological order of the DAG above, then shortest distances from s.

DFS from r (neighbours in alphabetical order) visits r, s, t, x, y, z; they finish in the order z, y, x, t, s, r.

Reversing: **r, s, t, x, y, z**. Every edge goes left to right in this list.

$d(s) = 0$, all others ∞ . Relax in that order:

r: $d(r) = \infty$, skip.

s: $t = 2$, $x = 6$.

t: x stays 6 ($2 + 7 = 9$), $y = 2 + 4 = 6$, $z = 2 + 2 = 4$.

x: $y = 6 - 1 = 5$, z stays 4 ($6 + 1 = 7$).

y: $z = 5 - 2 = 3$.

Result: $d(r) = \infty$, $d(s) = 0$, $d(t) = 2$, $d(x) = 6$, $d(y) = 5$, $d(z) = 3$.

EXAMPLE 2 (C PROGRAM)

```

#include <stdio.h>

#define N 6
#define INF 1000000
#define NOEDGE 0      /* no zero-weight edges in this graph */

char name[N] = {'r', 's', 't', 'x', 'y', 'z'};
int w[N][N] = {
    /*      r   s   t   x   y   z */
    /* r */ { 0, 5, 3, 0, 0, 0 },
    /* s */ { 0, 0, 2, 6, 0, 0 },
    /* t */ { 0, 0, 0, 7, 4, 2 },
    /* x */ { 0, 0, 0, 0, -1, 1 },
    /* y */ { 0, 0, 0, 0, 0, -2 },
    /* z */ { 0, 0, 0, 0, 0, 0 }
};

int visited[N], order[N], top = N;

void dfs(int u)
{
    int v;
    visited[u] = 1;
    for (v = 0; v < N; v++)
        if (w[u][v] != NOEDGE && !visited[v])
            dfs(v);
    order[--top] = u;          /* place u in front when it finishes */
}

int main(void)
{
    int d[N], i, u, v, src = 1;      /* source s */

    for (u = 0; u < N; u++)
        if (!visited[u])
            dfs(u);

    printf("Topological order: ");
    for (i = 0; i < N; i++)
        printf("%c ", name[order[i]]);
    printf("\n");

    for (v = 0; v < N; v++)
        d[v] = INF;
    d[src] = 0;
    for (i = 0; i < N; i++) {        /* relax in topological order */
        u = order[i];
        if (d[u] == INF)
            continue;
        for (v = 0; v < N; v++)
            if (w[u][v] != NOEDGE && d[u] + w[u][v] < d[v])
                d[v] = d[u] + w[u][v];
    }
}

```

```

}
for (v = 0; v < N; v++) {
    if (d[v] == INF)
        printf("d[%c] = infinity\n", name[v]);
    else
        printf("d[%c] = %d\n", name[v], d[v]);
}
return 0;
}

```

```

Topological order: r s t x y z
d[r] = infinity
d[s] = 0
d[t] = 2
d[x] = 6
d[y] = 5
d[z] = 3

```

Exam tip: a topological order is usually not unique; any order that respects every edge gets full marks. If a graph has a cycle, no topological order exists.

All pairs shortest paths (Floyd-Warshall)

All pairs shortest paths problem: find the shortest path between every pair of vertices (v, w) of a weighted graph.

Floyd-Warshall algorithm: a dynamic programming algorithm that solves it in $O(n^3)$ time by allowing vertices $1, 2, \dots, k$ in turn as intermediate vertices: $d_{ij} = \min(d_{ij}, d_{ik} + d_{kj})$.

All pairs shortest paths problem: weighted graph-ലെ ഓരോ vertex pair (v, w) -ക്കും ഇടയിലുള്ള shortest path കണ്ടെത്തുക.

Floyd-Warshall algorithm: vertices $1, 2, \dots, k$ -യെ ഓരോന്നായി intermediate vertices ആയി അനുവദിച്ച് $d_{ij} = \min(d_{ij}, d_{ik} + d_{kj})$ ഉപയോഗിച്ച് $O(n^3)$ time-ൽ ഇത് പരിഹരിക്കുന്ന dynamic programming algorithm.

Use the cost matrix C : $C_{ij} = 0$ if $i = j$, the weight of edge (i, j) if it exists, and ∞ otherwise. Start with $D = C$. In round k , for every pair (i, j) , check whether going through vertex k is cheaper: either k is not on the best path (keep d_{ij}) or it is, and then the path splits into $i \rightarrow k$ and $k \rightarrow j$. After n rounds D holds all shortest distances. The path matrix P records the k that last improved each entry, so paths can be rebuilt.

മലയാളത്തിൽ

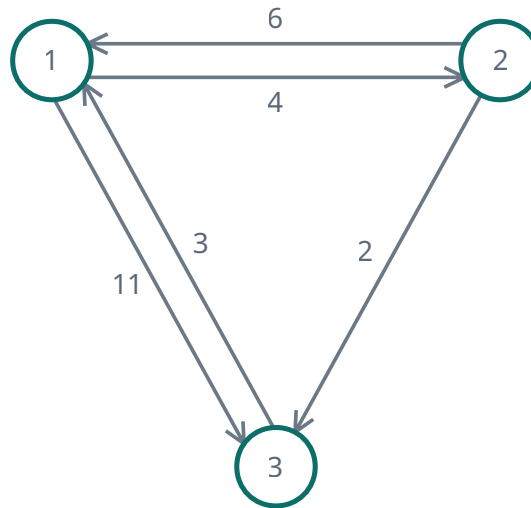
ഓരോ round-ലും ഒരു പുതിയ "transit town" അനുവദിക്കുന്നു എന്ന് കരുതുക. ആദ്യം direct bus മാത്രം; പിന്നെ town 1 വഴി മാറിക്കയറാം; പിന്നെ 1-ഉം 2-ഉം വഴി... ഓരോ ടിക്കറ്റിനും "k വഴി പോയാൽ ലാഭമുണ്ടോ?" എന്ന് നോക്കുന്നു. എല്ലാ towns-ഉം അനുവദിച്ചു കഴിയുമ്പോൾ table-ലെ എല്ലാ ദൂരങ്ങളും ശരിയാകും.

$C_{ij} = 0$ if $i = j$; $w(i, j)$ if $(i, j) \in E$; ∞ if $i \neq j$ and $(i, j) \notin E$

$D^{(0)} = C$

$d_{ij}^{(k)} = \min(d_{ij}^{(k-1)}, d_{ik}^{(k-1)} + d_{kj}^{(k-1)})$, $k = 1 \dots n$

Three nested loops over $k, i, j \Rightarrow O(n^3)$



Digraph: $1 \rightarrow 2$ (4), $2 \rightarrow 1$ (6), $1 \rightarrow 3$ (11), $3 \rightarrow 1$ (3), $2 \rightarrow 3$ (2).

EXAMPLE 1

Apply Floyd-Warshall to the graph above.

$$D^{(0)} = C = \begin{pmatrix} 0 & 4 & 11 \\ 6 & 0 & 2 \\ 3 & \infty & 0 \end{pmatrix}$$

$$k = 1: d_{32} = \min(\infty, d_{31} + d_{12}) = 3 + 4 = 7. D^{(1)} = \begin{pmatrix} 0 & 4 & 11 \\ 6 & 0 & 2 \\ 3 & 7 & 0 \end{pmatrix}$$

$$k = 2: d_{13} = \min(11, d_{12} + d_{23}) = 4 + 2 = 6. D^{(2)} = \begin{pmatrix} 0 & 4 & 6 \\ 6 & 0 & 2 \\ 3 & 7 & 0 \end{pmatrix}$$

$$k = 3: d_{21} = \min(6, d_{23} + d_{31}) = 2 + 3 = 5. D^{(3)} = \begin{pmatrix} 0 & 4 & 6 \\ 5 & 0 & 2 \\ 3 & 7 & 0 \end{pmatrix}$$

Path matrix: $P[1][3] = 2$ ($1 \rightarrow 2 \rightarrow 3$), $P[2][1] = 3$ ($2 \rightarrow 3 \rightarrow 1$), $P[3][2] = 1$ ($3 \rightarrow 1 \rightarrow 2$); other pairs use the direct edge.

EXAMPLE 2 (PROGRAM 8.1, CORRECTED AND COMPLETE)

```

#include <stdio.h>

#define N 3
#define INF 1000000 /* "infinity": large, but INF + INF still fits in an int */

void AllPairsShortestPaths(int n, int C[][N], int P[][N], int D[][N])
{
    int i, j, k;

    for (i = 0; i < n; i++) {
        for (j = 0; j < n; j++) {
            D[i][j] = C[i][j];
            P[i][j] = -1;
        }
        D[i][i] = 0; /* cost from a vertex to itself */
    }

    for (k = 0; k < n; k++)
        for (i = 0; i < n; i++)
            for (j = 0; j < n; j++)
                if (D[i][k] + D[k][j] < D[i][j]) {
                    D[i][j] = D[i][k] + D[k][j];
                    P[i][j] = k;
                }
}

int main(void)
{
    int C[N][N] = {
        { 0, 4, 11 },
        { 6, 0, 2 },
        { 3, INF, 0 }
    };
    int D[N][N], P[N][N], i, j;

    AllPairsShortestPaths(N, C, P, D);

    printf("D (shortest costs):\n");
    for (i = 0; i < N; i++) {
        for (j = 0; j < N; j++)
            printf("%4d", D[i][j]);
        printf("\n");
    }
    printf("P (intermediate vertex, -1 = direct edge):\n");
    for (i = 0; i < N; i++) {
        for (j = 0; j < N; j++)
            printf("%4d", P[i][j] == -1 ? -1 : P[i][j] + 1);
        printf("\n");
    }
}

```

```

return 0;
}

```

```

D (shortest costs):
0  4  6
5  0  2
3  7  0
P (intermediate vertex, -1 = direct edge):
-1 -1  2
 3 -1 -1
-1  1 -1

```

Exam tip: Program 8.1 in the unit does not compile and has a real bug. (1) `int i, j, k` has no semicolon. (2) `if i = j then C[i][j] = 0` is not `C` and sits outside any loop. (3) `D[i][j] = 0`; after the inner loop runs with `j = N`, writing outside the row; it must be `D[i][i] = 0`; (4) The inner loop uses `J` (capital) instead of `j`. Also, if ∞ is stored as `INT_MAX`, `D[i][k] + D[k][j]` overflows; use a large value such as 1000000 as above. In the text, $d_{ik}^{(k-1)}$ is the shortest path from `i` (not `j`) to `k`.

Minimum cost spanning trees

Minimum cost spanning tree (MST): a spanning tree of a connected weighted graph whose total edge weight is the smallest among all its spanning trees.

Minimum cost spanning tree (MST): connected weighted graph-ന്റെ എല്ലാ spanning trees-ലും വച്ച് edge weights-ന്റെ ആകെ തുക ഏറ്റവും കുറഞ്ഞ spanning tree.

A spanning tree is a subgraph that contains all the vertices and is a tree (connected, no cycle); a graph can have many. The weight of a tree is the sum of its edge weights, so different spanning trees have different weights. IGNOU's picture: link a group of islands with bridges so every island can reach every other, at minimum total cost.

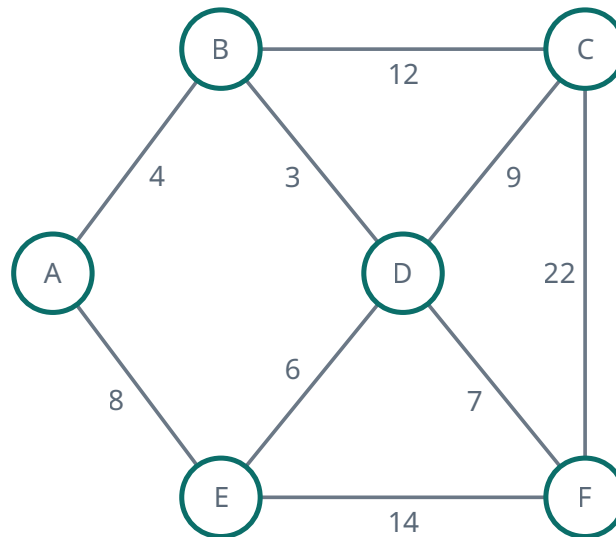
മലയാളത്തിൽ

Kuttanad-ലെ ചെറിയ ദ്വീപുകളെ പാലം കൊണ്ട് ബന്ധിപ്പിക്കണം എന്ന് കരുതുക. എല്ലാ ദ്വീപിൽ നിന്നും എല്ലാ ദ്വീപിലേക്കും എത്തണം, പക്ഷേ ചെലവ് ഏറ്റവും കുറവ് വേണം. ആവശ്യമില്ലാത്ത ഒരു പാലവും (cycle ഉണ്ടാക്കുന്നത്) പണിയില്ല. ഇങ്ങനെ കിട്ടുന്ന പാലങ്ങളുടെ set ആണ് MST.

Spanning tree of a graph with n vertices: exactly $n - 1$ edges, no cycle

Weight of a tree = sum of its edge weights

MST = spanning tree of minimum weight



The graph used below for Kruskal and Prim (6 vertices, 9 edges; weights as in the unit's Figure 8.10).

EXAMPLE

A triangle P-Q (1), Q-R (2), P-R (3). List its spanning trees and find the MST.

Remove any one edge to break the cycle: {PQ, QR} weight 3, {PQ, PR} weight 4, {QR, PR} weight 5. The MST is {PQ, QR} with weight 3.

Exam tip: the unit's summary says a spanning tree connects all vertices "with minimum cost". That describes a minimum spanning tree; an ordinary spanning tree only needs all vertices, $n - 1$ edges and no cycle.

Kruskal's algorithm

Kruskal's algorithm: builds an MST by taking edges in increasing order of weight and adding each edge that joins two different trees of the forest, rejecting any edge that would form a cycle, until $n - 1$ edges are added.

Kruskal's algorithm: edges weight കൂടുന്ന ക്രമത്തിൽ എടുത്ത്, forest-ലെ രണ്ട് വ്യത്യസ്ത trees-നെ ചേർക്കുന്ന edge ചേർക്കുകയും cycle ഉണ്ടാക്കുന്ന edge ഒഴിവാക്കുകയും ചെയ്ത്, $n - 1$ edges ആകുന്നത് വരെ MST നിർമ്മിക്കുന്നു.

Start with a forest of n single-vertex trees and put all edges in a priority queue (or sort them). Repeatedly take the cheapest edge: if its ends are already in the same tree it would form a cycle, so reject it; otherwise add it, joining two trees into one. The forest is a partition of the vertex set; each accepted edge unions two parts until only one remains. Running time $O(E \log E)$, mostly for sorting.

മലയാളത്തിൽ

ഏറ്റവും വില കുറഞ്ഞ പാലം ആദ്യം പണിയുക; പക്ഷേ ഇതിനകം തന്നെ ബന്ധിപ്പിക്കപ്പെട്ട രണ്ട് ദ്വീപുകൾക്കിടയിലാണെങ്കിൽ ആ പാലം വെറുതെ, വിടുക. ചെറിയ കൂട്ടങ്ങൾ (trees) ചേർന്ന് ചേർന്ന് അവസാനം ഒരൊറ്റ കൂട്ടമാകും.

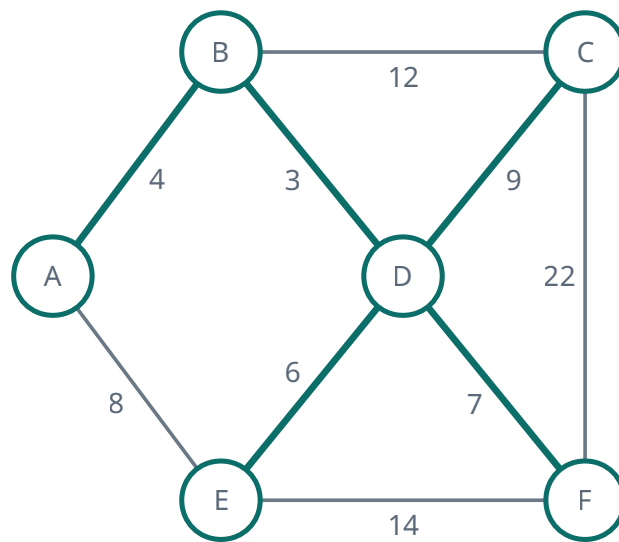
EXAMPLE 1 (THE UNIT'S FIGURE 8.10 WEIGHTS)

Find the MST of the graph above with Kruskal's algorithm.

Sorted edges: BD 3, AB 4, DE 6, DF 7, AE 8, DC 9, BC 12, EF 14, CF 22.

Step	Edge	Action	Trees after the step
1	B-D (3)	add	{B, D}, {A}, {C}, {E}, {F}
2	A-B (4)	add	{A, B, D}, {C}, {E}, {F}
3	D-E (6)	add	{A, B, D, E}, {C}, {F}
4	D-F (7)	add	{A, B, D, E, F}, {C}
5	A-E (8)	reject: A and E already in the same tree (cycle A-B-D-E-A)	unchanged
6	D-C (9)	add; $5 = n - 1$ edges, stop	{A, B, C, D, E, F}

MST edges: BD, AB, DE, DF, DC. Weight = $3 + 4 + 6 + 7 + 9 = 29$. This matches the unit's steps: 3, 4, 6, 7, then 8 rejected and 9 added.



Highlighted: the MST, total weight 29.

EXAMPLE 2 (C PROGRAM WITH UNION-FIND)

```

#include <stdio.h>
#include <stdlib.h>

#define V 6
#define E 9

struct edge { int u, v, w; };

int parent[V];

int find(int x)                                /* root of x's tree in the forest */
{
    while (parent[x] != x)
        x = parent[x];
    return x;
}

int cmp(const void *a, const void *b)
{
    return ((const struct edge *)a)->w - ((const struct edge *)b)->w;
}

int main(void)
{
    char name[V] = {'A', 'B', 'C', 'D', 'E', 'F'};
    struct edge e[E] = {
        {0, 1, 4}, {0, 4, 8}, {1, 2, 12}, {1, 3, 3}, {3, 2, 9},
        {3, 4, 6}, {3, 5, 7}, {2, 5, 22}, {4, 5, 14}
    };
    int i, count = 0, total = 0;

    for (i = 0; i < V; i++)
        parent[i] = i;                /* n single-node trees */
    qsort(e, E, sizeof e[0], cmp);    /* cheapest edge first */

    for (i = 0; i < E && count < V - 1; i++) {
        int ru = find(e[i].u), rv = find(e[i].v);
        if (ru == rv) {
            printf("%c-%c (%d) rejected: forms a cycle\n", name[e[i].u], name[e[i].v], e[i].w);
        } else {
            parent[ru] = rv;           /* union: join the two trees */
            count++;
            total += e[i].w;
            printf("%c-%c (%d) added\n", name[e[i].u], name[e[i].v], e[i].w);
        }
    }
    printf("Total cost of MST = %d\n", total);
    return 0;
}

```

```
B-D (3) added
A-B (4) added
D-E (6) added
D-F (7) added
A-E (8) rejected: forms a cycle
D-C (9) added
Total cost of MST = 29
```

Exam tip: always show the sorted edge list and write "rejected: forms a cycle" for skipped edges. Stop as soon as $n - 1$ edges are in; later edges need not be examined.

Prim's algorithm and comparison with Kruskal

Prim's algorithm: builds an MST one vertex at a time: start with a single vertex in tree T , and repeatedly add the smallest edge that joins a vertex in T to a vertex not in T , until T has all n vertices.

Prim's algorithm: ഒരു vertex മാത്രമുള്ള tree T -യിൽ തുടങ്ങി, T -യിലെ ഒരു vertex-നെ T -ക്ക് പുറത്തുള്ള vertex-മായി ചേർക്കുന്ന ഏറ്റവും ചെറിയ edge ആവർത്തിച്ച് ചേർത്ത്, n vertices-ഉം ആകുന്നത് വരെ ഒരു സമയം ഒരു vertex എന്ന രീതിയിൽ MST നിർമ്മിക്കുന്നു.

Prim keeps two disjoint sets of vertices: A (already in the tree) and $\bar{A} = V - A$. Each step picks the cheapest edge crossing from A to $\bar{A}$ and moves its outside end into A . So the tree always stays one connected piece, unlike Kruskal's forest. It works like Dijkstra, except the key of a vertex is the weight of one edge, not a path length.

മലയാളത്തിൽ

ഒരു ദ്വീപിൽ നിന്ന് തുടങ്ങുക. ഇതുവരെ എത്തിയ ദ്വീപുകളിൽ നിന്ന് പുറത്തേക്കുള്ള ഏറ്റവും വില കുറഞ്ഞ പാലം പണിയുക; അങ്ങനെ ഓരോ step-ലും ഒരു പുതിയ ദ്വീപ് കൂട്ടത്തിൽ ചേരുന്നു. Kruskal-ൽ പല കഷണങ്ങൾ ഒരുമിക്കുന്നു; Prim-ൽ ഒരൊറ്റ tree വളരുന്നു.

$T = \{x\}$

while T has fewer than n vertices:

find the smallest edge (a, b) with $a \in T, b \notin T$

add edge (a, b) and vertex b to T

EXAMPLE 1 (SAME GRAPH, START AT A)

Step	T before	Crossing edges (T → outside)	Chosen
1	{A}	AB 4, AE 8	A–B (4)
2	{A, B}	AE 8, BC 12, BD 3	B–D (3)
3	{A, B, D}	AE 8, BC 12, DC 9, DE 6, DF 7	D–E (6)
4	{A, B, D, E}	BC 12, DC 9, DF 7, EF 14 (AE 8 is now inside T)	D–F (7)
5	{A, B, D, E, F}	BC 12, DC 9, CF 22	D–C (9)

Edges added in order 4, 3, 6, 7, 9; weight 29, the same MST as Kruskal. Edge AE (8) is never used because both its ends are already in T (the unit says it "forms a cycle").

EXAMPLE 2 (C PROGRAM, $O(N^2)$ ARRAY VERSION)

```
#include <stdio.h>

#define V 6
#define INF 1000000

int main(void)
{
    char name[V] = {'A', 'B', 'C', 'D', 'E', 'F'};
    int w[V][V] = { /* 0 = no edge */
        /*      A  B  C  D  E  F */
        /* A */ { 0,  4,  0,  0,  8,  0 },
        /* B */ { 4,  0, 12,  3,  0,  0 },
        /* C */ { 0, 12,  0,  9,  0, 22 },
        /* D */ { 0,  3,  9,  0,  6,  7 },
        /* E */ { 8,  0,  0,  6,  0, 14 },
        /* F */ { 0,  0, 22,  7, 14,  0 }
    };
    int inT[V] = {0}, key[V], from[V];
    int i, k, u, total = 0;

    for (i = 0; i < V; i++) {
        key[i] = INF; /* cheapest edge from T to i */
        from[i] = -1;
    }
    key[0] = 0; /* start T with vertex A */

    for (k = 0; k < V; k++) {
        u = -1;
        for (i = 0; i < V; i++)
            if (!inT[i] && (u == -1 || key[i] < key[u]))
                u = i;
        inT[u] = 1;
        if (from[u] != -1) {
            printf("Add %c-%c (%d)\n", name[from[u]], name[u], key[u]);
            total += key[u];
        }
        for (i = 0; i < V; i++) /* update edges from T to G - T */
            if (w[u][i] && !inT[i] && w[u][i] < key[i]) {
                key[i] = w[u][i];
                from[i] = u;
            }
    }
    printf("Total cost of MST = %d\n", total);
    return 0;
}
```

```
Add A-B (4)
Add B-D (3)
Add D-E (6)
Add D-F (7)
Add D-C (9)
Total cost of MST = 29
```

	Kruskal's algorithm	Prim's algorithm
Principle	generic MST algorithm; grows a forest	special case of the generic algorithm; grows one tree, like Dijkstra
Works on	a single sorted set of edges	two disjoint sets of vertices, A and $V - A$
Running time	$O(E \log E)$	$O(E \log V)$ with a heap; $O(n^2)$ with an array
Best for	sparse graphs	dense graphs

Exam tip: in the unit's Prim steps, Step 4 says "edge with cost 73"; it is cost 7. The unit's comparison says Prim works on "two disjoint sets of edges"; they are sets of vertices (A and $\bar{A}$). It also says Prim "processes the edges randomly"; it actually always picks the minimum crossing edge. "For a dense graph Prim's is faster than Kruskal's" is **True**.

Applications of minimum cost spanning trees

Use of an MST: an MST connects all n points with $n - 1$ links of least total cost, so it models any "connect everything as cheaply as possible" network problem.

MST-ന്റെ ഉപയോഗം: n points-നെയും ഏറ്റവും കുറഞ്ഞ ആകെ ചെലവിൽ $n - 1$ links കൊണ്ട് MST ബന്ധിപ്പിക്കുന്നു; അതിനാൽ "എല്ലാം ഏറ്റവും ചുരുങ്ങിയ ചെലവിൽ ബന്ധിപ്പിക്കുക" എന്ന network problems-ന് ഇത് model ആകുന്നു.

IGNOU lists three applications: (1) travelling to all cities (travelling salesman problem), (2) electronic circuit design, connecting n pins with $n - 1$ wires of least total length, and (3) finding an independent set of circuit equations for an electrical network. Other common ones are laying cables, pipelines and road or computer networks.

മലയാളത്തിൽ

KSEB ഒരു പുതിയ colony-യിലെ എല്ലാ വീടുകളിലേക്കും ഏറ്റവും കുറഞ്ഞ നീളം wire വലിക്കണം എന്ന് കരുതുക. അത് MST തന്നെ. Circuit board-ൽ pins ബന്ധിപ്പിക്കുന്നതും ഇതേ ആശയം.

EXAMPLE

Five offices must be linked by cable; each possible link has a cost. Why is the answer a tree?

Every office must be reachable (connected), and any cycle contains a link whose removal keeps everything connected while saving money. So the cheapest network has no cycle: a spanning tree with 4 links, found by Kruskal or Prim.

Exam tip: an MST does not itself solve the travelling salesman problem (a tour must return to the start and visit each city once). The MST gives a lower bound and a basis for a good approximate tour. You can still list TSP as IGNOU does, but do not claim the MST is the tour.

Breadth first search (BFS)

Breadth first search: a graph traversal that starts at a source vertex, visits all its unvisited adjacent vertices first, then their unvisited neighbours, and so on level by level, using a queue.

Breadth first search: ഒരു source vertex-ൽ തുടങ്ങി, അതിന്റെ എല്ലാ unvisited adjacent vertices-നെയും ആദ്യം, പിന്നെ അവയുടെ unvisited neighbours-നെ എന്നിങ്ങനെ level by level ഒരു queue ഉപയോഗിച്ച് സന്ദർശിക്കുന്ന graph traversal.

BFS divides vertices into visited and not visited. Mark the source visited and put it in a queue. Repeatedly delete a vertex from the queue and visit it; add each of its unvisited neighbours to the queue (in the order they are listed) and mark them. Vertices are reached in increasing number of edges from the source. Time: $O(n^2)$ with a matrix, $O(n + e)$ with lists.

മലയാളത്തിൽ

ഒരു കളത്തിൽ കല്ലിട്ടാൽ ഓളങ്ങൾ വട്ടത്തിൽ പരക്കുന്നത് പോലെ: ആദ്യം തൊട്ടടുത്തുള്ളവർ, പിന്നെ അവരുടെ അടുത്തവർ. ഒരു വാർത്ത ആദ്യം നേരിട്ടുള്ള സുഹൃത്തുക്കളെ, പിന്നെ അവരുടെ സുഹൃത്തുക്കളെ അറിയിക്കുന്നതും BFS ആണ്. Queue ഉപയോഗിക്കുന്നത് കൊണ്ട് ആദ്യം വന്നവർ ആദ്യം.

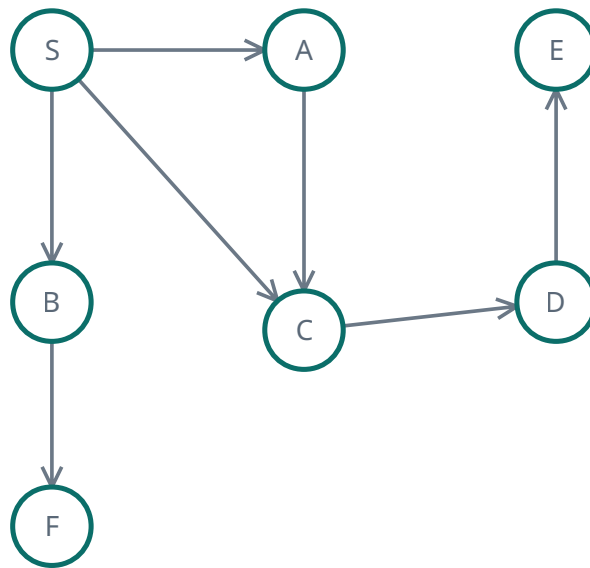


Figure 8.13 (as reconstructed): $S \rightarrow A$, $S \rightarrow B$, $S \rightarrow C$, $A \rightarrow C$, $C \rightarrow D$, $D \rightarrow E$, $B \rightarrow F$.

EXAMPLE 1 (THE UNIT'S ORDER)

BFS from S, taking S's neighbours in the order B, C, A as the unit does.

Delete	Unvisited neighbours added	Queue after
—	S	S
S	B, C, A	B C A
B	F	C A F
C	D	A F D
A	— (C already visited)	F D
F	—	D
D	E	E
E	—	empty

BFS order: **S, B, C, A, F, D, E** (as in the unit).

EXAMPLE 2 (MATRIX ORDER)

With an adjacency matrix in the order S, A, B, C, D, E, F, the neighbours of S are scanned as A, B, C. Find the BFS order.

Queue: $S \rightarrow A B C \rightarrow$ (A adds nothing) $\rightarrow$ B adds F $\rightarrow$ C adds D $\rightarrow F \rightarrow$ D adds E.

BFS order: **S, A, B, C, F, D, E**. The program in the next topic prints exactly this.

Exam tip: BFS uses a **queue**; DFS uses recursion (a stack). The visiting order depends on the order in which neighbours are listed, so state the order you use. Mark a vertex visited when it enters the queue, so it is never added twice.

Depth first search (DFS)

Depth first search: a graph traversal that goes deeper whenever possible: from the current vertex it visits an unvisited adjacent vertex, and when none is left it backtracks to the previous vertex.

Depth first search: സാധ്യമാകുമ്പോഴൊക്കെ ആഴത്തിലേക്ക് പോകുന്ന graph traversal: നിലവിലെ vertex-ൽ നിന്ന് ഒരു unvisited adjacent vertex സന്ദർശിക്കുന്നു, ഒന്നും ബാക്കിയില്ലെങ്കിൽ മുൻത്തെ vertex-ലേക്ക് backtrack ചെയ്യുന്നു.

DFS is recursive by nature and is similar to pre-order tree traversal. Each vertex gets a discovery time d (first visit) and a finishing time f (when all its neighbours are done and we backtrack). If some vertices are still unvisited when the source finishes, DFS restarts from any unvisited vertex; the result is a DFS forest. Time: $O(n^2)$ with an adjacency matrix, $O(n + e)$ with adjacency lists.

മലയാളത്തിൽ

ഒരു പഴയ തറവാട്ടിലെ മുറികൾ നോക്കുന്നത് ഓർക്കുക: ഒരു വാതിൽ കണ്ടാൽ ഉടൻ അകത്തേക്ക് കയറി അവസാനം വരെ പോകുന്നു; മുന്നോട്ട് വഴിയില്ലെങ്കിൽ തിരിച്ചു വന്ന് അടുത്ത തുറക്കാത്ത വാതിൽ നോക്കുന്നു. ഇതാണ് backtracking. Recursion (stack) ആണ് ഇത് ഓർത്തുവയ്ക്കുന്നത്.

```
DFS(u): mark u visited; d[u] = ++time
for each v adjacent to u: if v not visited, DFS(v)
f[u] = ++time
Main: for each vertex u: if u not visited, DFS(u)
```

EXAMPLE 1 (FIGURE 8.13)

DFS from S with neighbours in order A, B, C, ...; give d/f times.

S (1) → A (2) → C (3) → D (4) → E (5). E has no unvisited neighbour: E finishes (6), back to D, finishes (7), C (8), A (9). Back at S, next neighbour B (10) → F (11). F finishes (12), B (13), S (14).

Vertex	S	A	C	D	E	B	F
d/f	1/14	2/9	3/8	4/7	5/6	10/13	11/12

DFS order: **S, A, C, D, E, B, F** (the unit's order).

EXAMPLE 2 (C PROGRAM FOR BFS AND DFS)

```

#include <stdio.h>

#define N 7

char name[N] = {'S', 'A', 'B', 'C', 'D', 'E', 'F'};
int adj[N][N];
int visited[N], d[N], f[N], timer = 0;

void addEdge(char a, char b)          /* directed edge a -> b */
{
    int i, u = 0, v = 0;
    for (i = 0; i < N; i++) {
        if (name[i] == a) u = i;
        if (name[i] == b) v = i;
    }
    adj[u][v] = 1;
}

void bfs(int s)
{
    int queue[N], front = 0, rear = 0, u, v;

    for (v = 0; v < N; v++)
        visited[v] = 0;
    visited[s] = 1;
    queue[rear++] = s;
    while (front < rear) {
        u = queue[front++];          /* delete from queue */
        printf("%c ", name[u]);
        for (v = 0; v < N; v++)
            if (adj[u][v] && !visited[v]) {
                visited[v] = 1;      /* mark when added, so no vertex enters twice */
                queue[rear++] = v;
            }
    }
    printf("\n");
}

void dfs(int u)
{
    int v;

    visited[u] = 1;
    d[u] = ++timer;                  /* discovery time */
    printf("%c ", name[u]);
    for (v = 0; v < N; v++)
        if (adj[u][v] && !visited[v])
            dfs(v);
    f[u] = ++timer;                  /* finishing time */
}

```

```

int main(void)
{
    int v;

    addEdge('S', 'A'); addEdge('S', 'B'); addEdge('S', 'C');
    addEdge('A', 'C'); addEdge('C', 'D'); addEdge('D', 'E');
    addEdge('B', 'F');

    printf("BFS from S: ");
    bfs(0);

    printf("DFS from S: ");
    for (v = 0; v < N; v++)
        visited[v] = 0;
    for (v = 0; v < N; v++)          /* restart from any unvisited vertex */
        if (!visited[v])
            dfs(v);
    printf("\n");
    for (v = 0; v < N; v++)
        printf("%c %d/%d\n", name[v], d[v], f[v]);
    return 0;
}

```

```

BFS from S: S A B C F D E
DFS from S: S A C D E B F
S 1/14
A 2/9
B 10/13
C 3/8
D 4/7
E 5/6
F 11/12

```

Exam tip: the unit's Figure 8.14 gives $S = 1/10$ and $B = 11/14$, i.e. S finishing before B is discovered and B starting a new DFS tree. That is impossible here: $S \rightarrow B$ is an edge (BFS visits B straight from S), so B is discovered while S is still open. The correct times are S 1/14, B 10/13, F 11/12. Remember: if $u \rightarrow v$ is an edge and v is unvisited when u starts, then $d[u] < d[v] < f[v] < f[u]$.

Strongly connected components

Strongly connected component (SCC): a maximal set $U \subseteq V$ of a directed graph such that for every $u, v \in U$ there is a path from u to v and from v to u .

Transpose G^T : the graph (V, E^T) with $E^T = \{(u, v) : (v, u) \in E\}$, i.e. G with every edge reversed.

Strongly connected component (SCC): directed graph-ൽ ഏത് $u, v \in U$ എടുത്താലും u -ൽ നിന്ന് v -യിലേക്കും v -യിൽ നിന്ന് u -യിലേക്കും path ഉള്ള ഏറ്റവും വലിയ (maximal) set $U \subseteq V$.

Transpose G^T : $E^T = \{(u, v) : (v, u) \in E\}$ ആയ (V, E^T) എന്ന graph; അതായത് എല്ലാ edges-ന്റെയും ദിശ തിരിച്ച G .

Finding SCCs is a neat application of DFS. G^T has the same vertices and edges with directions reversed; its adjacency matrix is the transpose of G 's. Reversing edges does not change which vertices are mutually reachable, so G and G^T have the same SCCs. Running DFS on G^T in decreasing order of finishing times from G cuts the graph exactly at SCC boundaries.

മലയാളത്തിൽ

ഒരു WhatsApp group-ൽ ആർക്കൊക്കെ പരസ്പരം (രണ്ടു ദിശയിലും) message എത്തിക്കാം എന്ന കൂട്ടങ്ങളാണ് SCCs. ആദ്യം G -യിൽ DFS ഓടിച്ച് finishing times കുറിക്കുന്നു; പിന്നെ എല്ലാ arrows-ഉം തിരിച്ചിട്ട് (G^T), ഏറ്റവും വലിയ finishing time ഉള്ള vertex-ൽ നിന്ന് DFS തുടങ്ങുന്നു. ഓരോ DFS tree-ഉം ഓരോ SCC.

Strongly-Connected-Components(G):

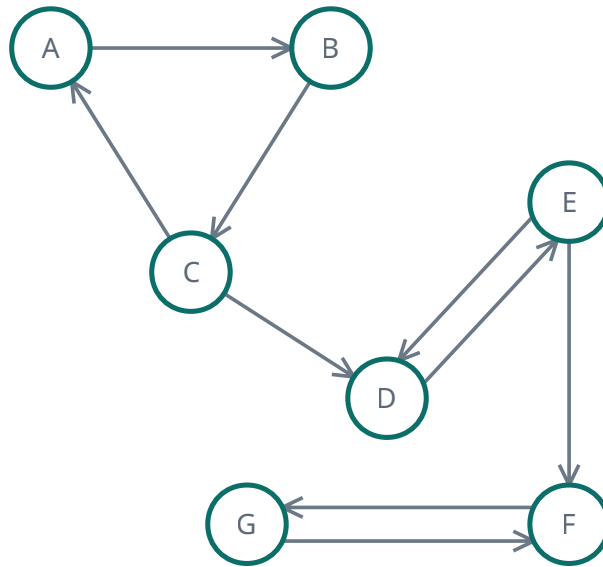
Step 1: DFS(G) to compute $f[u]$ for all $u \in V$

Step 2: compute G^T

Step 3: DFS(G^T), taking vertices in decreasing order of $f[u]$

Step 4: output the vertices of each tree of the Step 3 forest as one SCC

Time: $O(n + e)$ with adjacency lists



Digraph: $A \rightarrow B, B \rightarrow C, C \rightarrow A, C \rightarrow D, D \rightarrow E, E \rightarrow D, E \rightarrow F, F \rightarrow G, G \rightarrow F$.

EXAMPLE 1

Find the SCCs of the digraph above.

Step 1 (DFS on G from A , alphabetical order): A 1/14, B 2/13, C 3/12, D 4/11, E 5/10, F 6/9, G 7/8.

Step 2: G^T edges: $B \rightarrow A, C \rightarrow B, A \rightarrow C, D \rightarrow C, E \rightarrow D, D \rightarrow E, F \rightarrow E, G \rightarrow F, F \rightarrow G$.

Step 3: highest f is A (14). DFS on G^T from A : $A \rightarrow C \rightarrow B$, then stuck. Tree $\{A, C, B\}$. Next highest unvisited: D (11): $D \rightarrow E$ ($D \rightarrow C$ is visited). Tree $\{D, E\}$. Next: F (9): $F \rightarrow G$. Tree $\{F, G\}$.

SCCs: $\{A, B, C\}, \{D, E\}, \{F, G\}$.

EXAMPLE 2 (C PROGRAM)

```

#include <stdio.h>

#define N 7

char name[N] = {'A', 'B', 'C', 'D', 'E', 'F', 'G'};
int g[N][N], gt[N][N];
int visited[N], d[N], f[N], timer = 0;
int byFinish[N], cnt = 0;

void dfs1(int u)                                /* DFS on G, records d[] and f[] */
{
    int v;
    visited[u] = 1;
    d[u] = ++timer;
    for (v = 0; v < N; v++)
        if (g[u][v] && !visited[v])
            dfs1(v);
    f[u] = ++timer;
    byFinish[cnt++] = u;                        /* increasing order of finishing time */
}

void dfs2(int u)                                /* DFS on the transpose GT */
{
    int v;
    visited[u] = 1;
    printf("%c ", name[u]);
    for (v = 0; v < N; v++)
        if (gt[u][v] && !visited[v])
            dfs2(v);
}

int main(void)
{
    int edges[][2] = { {0,1}, {1,2}, {2,0}, {2,3}, {3,4}, {4,3}, {4,5}, {5,6}, {6,5} };
    int m = sizeof edges / sizeof edges[0];
    int i, u, v, k = 0;

    for (i = 0; i < m; i++)
        g[edges[i][0]][edges[i][1]] = 1;

    for (u = 0; u < N; u++)                    /* Step 1: DFS(G) */
        if (!visited[u])
            dfs1(u);
    for (u = 0; u < N; u++)
        printf("%c %d/%d\n", name[u], d[u], f[u]);

    for (u = 0; u < N; u++)                    /* Step 2: GT = transpose of G */
        for (v = 0; v < N; v++)
            gt[u][v] = g[v][u];

    for (u = 0; u < N; u++)

```

```

    visited[u] = 0;
    for (i = N - 1; i >= 0; i--) {    /* Step 3: DFS(GT), decreasing f[u] */
        u = byFinish[i];
        if (!visited[u]) {
            printf("SCC %d: ", ++k); /* Step 4: each tree is one SCC */
            dfs2(u);
            printf("\n");
        }
    }
    return 0;
}

```

```

A 1/14
B 2/13
C 3/12
D 4/11
E 5/10
F 6/9
G 7/8
SCC 1: A C B
SCC 2: D E
SCC 3: F G

```

Exam tip: DFS is the traversal used for SCCs. In the definition write "maximal": {A, B} is also mutually reachable here, but it is not an SCC because C can be added. The unit's own Figures 8.15/8.16 did not survive in this text, so learn the method with the example above.

Quick recap

- $G = (V, E)$; directed edges have a direction $((a, b) \neq (b, a))$, undirected edges do not.
- Path length = number of edges; a cycle is a path with $v_0 = v_k$; a graph with no cycle is acyclic.
- A spanning tree has all n vertices and $n - 1$ edges and no cycle.
- Adjacency matrix: n^2 space, symmetric for undirected graphs. Adjacency list: space proportional to $n + e$.
- Strongly connected: directed path both ways between every pair; weakly connected: connected only if directions are ignored.
- Relaxation: if $d[u] + w(u, v) < d[v]$, set $d[v] = d[u] + w(u, v)$, $pi[v] = u$.
- Dijkstra: single source, non-negative weights, greedy (Extract-Min), $\Theta(n^2)$ with an array.
- A reachable negative weight cycle makes shortest paths undefined; Dijkstra can fail with any negative edge.
- In a DAG, relax vertices in topological order (DFS: decreasing finish time); negative edges are fine.
- Floyd-Warshall: $d_{ij} = \min(d_{ij}, d_{ik} + d_{kj})$ for $k = 1 \dots n$, $O(n^3)$.
- Kruskal: cheapest edge first, reject cycles, forest of trees merges; $O(E \log E)$.
- Prim: grow one tree from a vertex by the cheapest crossing edge; better for dense graphs.

- BFS uses a queue (level by level); DFS is recursive (go deep, backtrack) and records d/f times.
- SCCs: DFS(G) for f[u], then DFS(G^T) in decreasing f[u]; each tree is one SCC.

Check yourself

Q. 1. A graph with no cycle is called _____ graph. The adjacency matrix of an undirected graph is _____ about the main diagonal.

An **acyclic** graph. The matrix is **symmetric** ($A = A^T$), because edge (i, j) is the same as (j, i).

Q. 2. Write the adjacency matrices of (a) the digraph with edges $1 \rightarrow 2, 1 \rightarrow 3, 3 \rightarrow 4, 4 \rightarrow 1, 4 \rightarrow 2$ (Figure 8.3) and (b) the undirected graph with edges 1-2, 1-3, 1-4, 2-4, 3-4 (Figure 8.4).

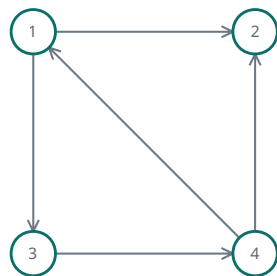


Figure 8.3

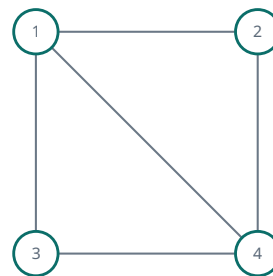


Figure 8.4

$$(a) \begin{pmatrix} 0 & 1 & 1 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 1 & 1 & 0 & 0 \end{pmatrix} \quad (b) \begin{pmatrix} 0 & 1 & 1 & 1 \\ 1 & 0 & 0 & 1 \\ 1 & 0 & 0 & 1 \\ 1 & 1 & 1 & 0 \end{pmatrix}$$

(a) is not symmetric (directed); (b) is symmetric. (The unit's answer key labels both matrices "Figure 8.3"; the second one is for Figure 8.4.)

Q. 3. What is the basis of Dijkstra's algorithm, and what is the time complexity of the all pairs shortest paths (Floyd-Warshall) algorithm?

Dijkstra is based on (edge) **relaxation**: $d[v]$ is lowered to $d[u] + w(u, v)$ whenever that is smaller. Floyd-Warshall has three nested loops over n vertices, so it runs in $O(n^3)$.

Q. 4. Run Dijkstra's algorithm from A on the digraph $A \rightarrow B$ 4, $A \rightarrow C$ 1, $C \rightarrow B$ 2, $B \rightarrow D$ 1, $C \rightarrow D$ 5, $D \rightarrow E$ 3. Give the order in which vertices are added and the final distances.

Start: $d(A) = 0$, rest ∞ .

Add A: $d(B) = 4$, $d(C) = 1$.

Add C (1): $d(B) = \min(4, 1 + 2) = 3$, $d(D) = 1 + 5 = 6$.

Add B (3): $d(D) = \min(6, 3 + 1) = 4$.

Add D (4): $d(E) = 4 + 3 = 7$.

Add E (7).

Order: A, C, B, D, E. Distances: A 0, C 1, B 3, D 4, E 7. Path to E: $A \rightarrow C \rightarrow B \rightarrow D \rightarrow E$.

Q. 5. Why can't Dijkstra's algorithm be used on graphs with negative edge weights? What happens if a negative weight cycle is reachable from the source?

Dijkstra fixes a vertex's distance when it is extracted, assuming no later path can be cheaper. A negative edge found later can make a cheaper path, e.g. $S \rightarrow A$ 2, $S \rightarrow B$ 3, $B \rightarrow A$ -2: Dijkstra gives $d(A) = 2$ but the true value is 1. If a negative weight cycle is reachable, going round it again lowers the cost each time, so the shortest path is not defined at all.

Q. 6. Find the MST of the undirected graph $P-Q$ 1, $Q-R$ 3, $P-R$ 4, $R-S$ 2, $Q-S$ 5, $S-T$ 7, $R-T$ 6 using Kruskal's algorithm. Then give the order in which Prim's algorithm (from P) adds edges.

Kruskal, sorted: PQ 1 add; RS 2 add; QR 3 add; PR 4 reject (P, R already joined via Q); QS 5 reject (cycle Q-R-S); RT 6 add. Now $4 = n - 1$ edges, stop (ST 7 is not needed).

MST = {PQ, RS, QR, RT}, weight $1 + 2 + 3 + 6 = 12$.

Prim from P: PQ (1), QR (3), RS (2), RT (6). Same tree, weight 12.

Q. 7. Which graph traversal uses a queue? Which is recursive by nature? Which is used to find strongly connected components? True or false: for a dense graph Prim's algorithm is faster than Kruskal's.

BFS uses a queue. **DFS** is recursive. **DFS** (run on G and then on G^T) finds SCCs. **True**: with an array Prim runs in $O(n^2)$, while Kruskal needs $O(E \log E)$, and E is close to n^2 in a dense graph.

Q. 8. Give a topological order of the DAG $1 \rightarrow 2$, $1 \rightarrow 3$, $2 \rightarrow 4$, $3 \rightarrow 4$, $4 \rightarrow 5$. Is it unique?

DFS from 1: $1 \rightarrow 2 \rightarrow 4 \rightarrow 5$; 5 finishes, then 4, then 2; back at 1, visit 3 (4 already visited); 3 finishes, then 1. Decreasing finish time: **1, 3, 2, 4, 5**.

Not unique: 1, 2, 3, 4, 5 is also valid, because 2 and 3 have no edge between them.

Q. 9. Apply one round ($k = 1$) of Floyd-Warshall to $C = [[0, 3, \infty], [\infty, 0, 1], [2, \infty, 0]]$ (vertices 1, 2, 3).

With $k = 1$, $d_{ij} = \min(d_{ij}, d_{i1} + d_{1j})$. Only row 3 has d_{31} finite (2): $d_{32} = \min(\infty, 2 + 3) = 5$; d_{33} stays 0. Row 2 has $d_{21} = \infty$, so nothing changes.

$$D^{(1)} = \begin{pmatrix} 0 & 3 & \infty \\ \infty & 0 & 1 \\ 2 & 5 & 0 \end{pmatrix}$$

Q. 10. Find the strongly connected components of the digraph $1 \rightarrow 2, 2 \rightarrow 3, 3 \rightarrow 1, 3 \rightarrow 4, 4 \rightarrow 5$.

DFS on G from 1: 1 (1), 2 (2), 3 (3), 4 (4), 5 (5); 5 finishes 6, 4 finishes 7, 3 finishes 8, 2 finishes 9, 1 finishes 10.

G^T : $2 \rightarrow 1, 3 \rightarrow 2, 1 \rightarrow 3, 4 \rightarrow 3, 5 \rightarrow 4$. DFS in decreasing f : from 1 reach 3, $2 \rightarrow \{1, 2, 3\}$; from 4 (3 is visited) $\rightarrow \{4\}$; from 5 $\rightarrow \{5\}$.

SCCs: $\{1, 2, 3\}, \{4\}, \{5\}$.

UNIT 9

Searching and Sorting Techniques

ഒരു വലിയ data ശേഖരത്തിൽ നിന്ന് വേണ്ട item വേഗത്തിൽ കണ്ടെത്താനുള്ള linear search, binary search എന്നിവയും, data-യെ ക്രമത്തിൽ അടുക്കാനുള്ള insertion, bubble, quick, merge, heap sort എന്നിവയും ഈ unit-ൽ പഠിക്കുന്നു. ഓരോ method-ന്റെയും C program, pass-by-pass trace, time complexity എന്നിവ ഇവിടെയുണ്ട്.

Searching: basic terms

Linear (sequential) search

Binary search

Applications of searching

Internal sorting

Insertion sort

Bubble sort

Quick sort

2-way merge sort

Heap: definition, array form and construction

Heap sort

Sorting on several keys

Searching: basic terms

Searching: the process of finding whether a given value (the search argument) is present in a collection of data and, if it is, where it is stored.

Key: a field of a record used to distinguish one record from the others; a record is located by matching the search argument with its key.

Searching: തനിരിക്കുന്ന ഒരു value (search argument) ഒരു data ശേഖരത്തിൽ ഉണ്ടോ എന്നും, ഉണ്ടെങ്കിൽ അത് എവിടെയാണ് സൂക്ഷിച്ചിരിക്കുന്നത് എന്നും കണ്ടെത്തുന്ന പ്രക്രിയ.

Key: ഒരു record-നെ മറ്റ് records-ൽ നിന്ന് വേർതിരിച്ചറിയാൻ ഉപയോഗിക്കുന്ന field; search argument-നെ key-യുമായി ഒത്തുനോക്കിയാണ് record കണ്ടെത്തുന്നത്.

A **file** is a collection of **records**, and a record is a collection of **fields**. In a telephone directory, the whole directory is the file, each line (name + phone number) is a record, and name and phone number are fields. The field we search on is the **key**. Usually that is the name, but in a reverse lookup it could be the phone number. A **search algorithm** takes an argument *a* and finds the record whose key matches *a*. The unit covers two methods: **linear (sequential) search** and **binary search**.

മലയാളത്തിൽ

അലമാരയിൽ എല്ലാം വാരിവലിച്ചിട്ടാൽ ഒരു ഷർട്ട് കണ്ടെത്താൻ എല്ലാം ഓരോന്നായി നോക്കണം; അടുക്കി വെച്ചാൽ പെട്ടെന്ന് കിട്ടും. Computer-ലും ഇതുതന്നെ: data ക്രമത്തിലാണെങ്കിൽ search വേഗത്തിലാകും. Telephone directory ഒരു file, അതിലെ ഓരോ വരിയും ഒരു record, പേരും നമ്പറും fields. ഏത് field വെച്ചാണോ നമ്മൾ തിരയുന്നത്, അതാണ് key.

File = collection of records; Record = collection of fields

Key = the field used to identify / search a record

Search argument (a, el) = the value we are looking for

Successful search: a record with key = argument is found; unsuccessful: no such record

EXAMPLE

A student file stores (Enrolment no., Name, Programme) for each student. Identify file, record, field and a suitable key.

File = the whole list of students. Record = one student's row. Fields = Enrolment no., Name, Programme.

Key = Enrolment no., because it is unique for every student (two students may share a name).

Linear (sequential) search

Linear search: a method that compares the search key with the elements of a list one by one, from the first, until a match is found or the list ends.

Linear search: search key-യെ list-ലെ elements-മായി ആദ്യം മുതൽ ഓരോന്നായി താരതമ്യം ചെയ്ത്, match കിട്ടുന്നതുവരെയോ list തീരുന്നതുവരെയോ തുടരുന്ന method.

Linear search is the simplest search. It works on any array, sorted or not, and when the elements are in random order it is the only reasonable method. It is slow for big arrays, but for small arrays the time is not even noticed, so it is a perfectly valid choice there.

മലയാളത്തിൽ

Bus stand-ൽ കൂട്ടുകാരനെ അന്വേഷിക്കുമ്പോൾ ഓരോ ആളുടെയും മുഖം ഒന്നൊന്നായി നോക്കുന്നതുപോലെ. Array sorted ആകണമെന്നില്ല. Element ആദ്യം തന്നെ ഉണ്ടെങ്കിൽ ഒരു comparison മതി; ഏറ്റവും അവസാനമോ ഇല്ലെങ്കിലോ n comparisons വേണം.

Algorithm (m = array of n keys, el = value to be searched):

Step 1: [Initialise] k = 0, flag = 1

Step 2: Repeat Step 3 for k = 0, 1, 2, ..., n - 1

Step 3: if m[k] = el then flag = 0; print "Search is successful", element found at location k + 1; stop

Step 4: if flag = 1 then print "Search is unsuccessful"

Step 5: stop

Program 9.1 in the unit has several errors. The corrected program below compiles with gcc and stops at the first match, as the algorithm says.

```
#include <stdio.h>

int linear_search(int m[], int n, int el)
{
    int k;
    for (k = 0; k < n; k++)
        if (m[k] == el)
            return k;          /* found: stop at once */
    return -1;                 /* not found */
}

int main(void)
{
    int m[] = {42, 17, 8, 30, 95, 61, 23};
    int n = 7, el, pos;

    el = 30;
    pos = linear_search(m, n, el);
    if (pos != -1)
        printf("Search is successful: %d found at location %d\n", el, pos + 1);
    else
        printf("Search is unsuccessful: %d not in list\n", el);

    el = 50;
    pos = linear_search(m, n, el);
    if (pos != -1)
        printf("Search is successful: %d found at location %d\n", el, pos + 1);
    else
        printf("Search is unsuccessful: %d not in list\n", el);
    return 0;
}
```

```
Search is successful: 30 found at location 4
Search is unsuccessful: 50 not in list
```

EXAMPLE (TRACE)

Search for $el = 30$ in $m = [42, 17, 8, 30, 95, 61, 23]$.

k	m[k]	m[k] = 30?
0	42	no
1	17	no
2	8	no
3	30	yes, stop

Found at index 3, i.e. location $3 + 1 = 4$, after 4 comparisons. Searching for 50 compares all 7 elements and then reports "unsuccessful".

EXAMPLE (IGNOU'S TELEPHONE DIRECTORY)

A directory has $n = 10$ records with Name as key: Nitin Kumar, Preeti Jain, Sandeep Singh, Sapna Chowdhary, Hitesh Somal, R.S.Singh, Radha Sharma, S.N.Singh, Arvind Chittora, Anil Rawat. Search for "Radha Sharma".

The names are compared in order, $m[0], m[1], \dots$. The first six do not match; $m[6] = \text{"Radha Sharma"}$ matches. The search stops at index 6, so the record is at position $6 + 1 = 7$, and the phone number returned is **26150880**. It took 7 comparisons.

Efficiency. The number of comparisons depends on where the key is. If every position is equally likely, a successful search needs on average $(1 + 2 + \dots + n)/n = (n + 1)/2$ comparisons, and an unsuccessful search needs n .

Best case: key at the first position $\rightarrow 1$ comparison $\rightarrow O(1)$

Average case (successful): $(n + 1)/2$ comparisons $\rightarrow O(n)$

Worst case (last position or not present): n comparisons $\rightarrow O(n)$

Exam tip: Program 9.1 in the unit will not compile. It uses undeclared names (`element` should be `el`, and `number` is never declared), it has missing brackets in `if (m[k]==el` and `printf("%d", m[i];`, its display loop is hard-wired to 20 instead of `n`, and it uses `rand()` without `<stdlib.h>`. It also keeps looping after a match, although the algorithm says "stop". Use `int main(void)`, not `void main()` and `<conio.h>`, which gcc does not provide.

Binary search

Binary search: a search on a sorted array that compares the key with the middle element and, depending on the result, discards the half that cannot contain the key, until the key is found or no elements remain.

Binary search: sorted array-ൽ key-യെ നടുവിലെ element-മായി താരതമ്യം ചെയ്ത്, key ഉണ്ടാകാൻ സാധ്യതയില്ലാത്ത പകുതി ഒഴിവാക്കി, key കിട്ടുന്നതുവരെയോ elements തീരുന്നതുവരെയോ തുടരുന്ന search.

If the array is sorted, one comparison with the middle element tells us which half the key must be in. If $\text{key} < \text{middle}$, it can only be in the left half; if $\text{key} > \text{middle}$, only in the right half. The other half is "thrown out" with a single comparison. The search space halves at every step, so even 100,000 elements need at most 17 comparisons. The array must be ordered (ascending or descending).

മലയാളത്തിൽ

Dictionary-യിൽ "Mango" എന്ന വാക്ക് തിരയുമ്പോൾ നമ്മൾ ആദ്യ പേജ് മുതൽ വായിക്കില്ല; നടുവിൽ തുറക്കും, "M" മുന്നിലാണോ പിന്നിലാണോ എന്ന് നോക്കി ഒരു പകുതി ഉപേക്ഷിക്കും. ഇതാണ് binary search. ഓരോ step-ലും search space പകുതിയാകുന്നു. പക്ഷേ ഒരു നിബന്ധനയുണ്ട്: array sorted ആയിരിക്കണം.

Algorithm (k = sorted array of n keys, ascending):

```
Step 1: low  $\leftarrow$  0, high  $\leftarrow$  n - 1
Step 2: while (low  $\leq$  high) do
    mid  $\leftarrow$  (low + high)/2 (integer division)
    if key = k[mid] then write "record is at position", mid + 1; stop
    else if key < k[mid] then high  $\leftarrow$  mid - 1
    else low  $\leftarrow$  mid + 1
endwhile
Step 3: write "Sorry, key value not found"
Step 4: stop
```

```

#include <stdio.h>

int binary_search(int k[], int n, int key)
{
    int low = 0, high = n - 1, mid;
    while (low <= high) {
        mid = (low + high) / 2;
        printf("low=%d mid=%d high=%d k[mid]=%d\n", low, mid, high, k[mid]);
        if (key == k[mid])
            return mid;          /* found: stop */
        else if (key < k[mid])
            high = mid - 1;
        else
            low = mid + 1;
    }
    return -1;
}

int main(void)
{
    int k[] = {11, 22, 33, 44, 55};
    int keys[] = {55, 25};
    int i, pos;
    for (i = 0; i < 2; i++) {
        printf("Looking for %d\n", keys[i]);
        pos = binary_search(k, 5, keys[i]);
        if (pos != -1)
            printf("Record is at position %d\n\n", pos + 1);
        else
            printf("Sorry, key value not found\n\n");
    }
    return 0;
}

```

```

Looking for 55
low=0 mid=2 high=4 k[mid]=33
low=3 mid=3 high=4 k[mid]=44
low=4 mid=4 high=4 k[mid]=55
Record is at position 5

```

```

Looking for 25
low=0 mid=2 high=4 k[mid]=33
low=0 mid=0 high=1 k[mid]=11
low=1 mid=1 high=1 k[mid]=22
Sorry, key value not found

```

EXAMPLE (IGNOU'S, SUCCESSFUL SEARCH)

k = [11, 22, 33, 44, 55], n = 5, key = 55.

Iteration	low	high	mid	k[mid]	Decision
1	0	4	$(0+4)/2 = 2$	33	$55 > 33 \rightarrow \text{low} = 3$
2	3	4	$(3+4)/2 = 3$	44	$55 > 44 \rightarrow \text{low} = 4$
3	4	4	$(4+4)/2 = 4$	55	equal $\rightarrow$ found

The record is at position $\text{mid} + 1 = 5$, after 3 comparisons.

EXAMPLE (UNSUCCESSFUL SEARCH)

Same array, key = 25.

mid = 2, 33: $25 < 33 \rightarrow \text{high} = 1$. mid = $(0+1)/2 = 0$, 11: $25 > 11 \rightarrow \text{low} = 1$. mid = 1, 22: $25 > 22 \rightarrow \text{low} = 2$.
Now $\text{low} = 2 > \text{high} = 1$, the loop ends, and the answer is **"key value not found"** after 3 comparisons.

Descending order (Check Your Progress 2, Q2). In a descending array the big keys are on the left, so only the two direction tests swap: if $\text{key} > k[\text{mid}]$, go left ($\text{high} = \text{mid} - 1$), else go right ($\text{low} = \text{mid} + 1$).

```
int binary_search_desc(int k[], int n, int key)
{
    int low = 0, high = n - 1, mid;
    while (low <= high) {
        mid = (low + high) / 2;
        if (key == k[mid])
            return mid;
        else if (key > k[mid])    /* bigger keys are on the left */
            high = mid - 1;
        else
            low = mid + 1;
    }
    return -1;
}
```

EXAMPLE (DESCENDING ARRAY)

k = [90, 75, 60, 45, 30, 15], key = 30.

low = 0, high = 5, mid = 2, k[2] = 60. $30 < 60$, so 30 lies to the right: low = 3.

low = 3, high = 5, mid = 4, k[4] = 30 $\rightarrow$ found at position $4 + 1 = 5$.

Efficiency. Each comparison halves the number of candidates, so after about $\log_2 n$ comparisons only one is left.

Best case: key at the first mid $\rightarrow$ 1 comparison $\rightarrow O(1)$

Average and worst case: $O(\log_2 n)$; worst case = $\lfloor \log_2 n \rfloor + 1$ comparisons

Condition: the array must be sorted and allow direct access by index (an array, not a linked list)

Array size n	Linear search (average , successful) $\approx$ $n/2$	Binary search (worst case) = $\lfloor \log_2 n \rfloor + 1$
8	4	4
128	64	8
256	128	9
1000	500	10
100,000	50,000	17

For very small arrays linear search can be as fast, but as n grows binary search wins clearly. Its drawback is that the data must be sorted: one element out of order can make it miss the key. If you will search an unsorted list only once, a linear search is cheaper than sorting first and then doing a binary search.

Exam tip: The unit's table gives 11 comparisons for $n = 1000$ and 18 for $n = 100,000$. The correct worst-case values are 10 and 17 ($\lfloor \log_2 1000 \rfloor + 1 = 9 + 1$, $\lfloor \log_2 100000 \rfloor + 1 = 16 + 1$). We checked this by running binary search on every hit and miss for those sizes. The unit's algorithm also forgets to **stop** after "record is at position": as printed, it would loop for ever on a match and then print "not found". Program 9.2 has more bugs: a missing `)` in `if (found==1, the misspelt call binary_searchy, a void function printed with %d, and scanf("%d", array[i]) missing the &.`

Applications of searching

Trie: a tree-based data structure that stores a collection of strings character by character, so that searching for a string (or prefix) is fast.

Trie: strings-നെ അക്ഷരം അക്ഷരമായി സൂക്ഷിക്കുന്ന, ഒരു string-ഓ prefix-ഓ വേഗത്തിൽ search ചെയ്യാൻ സഹായിക്കുന്ന tree-based data structure.

The main use of searching is to find a particular record in a large file quickly. The unit gives three applications:

- **Spell checker** (word processors): each word you type is looked up in a dictionary word list. A word found in the list is taken as correctly spelt, and a word not found is flagged as wrong. The unit says this uses linear (sequential) search.

- **Search engines:** software robots (also called spiders or crawlers) follow hyperlinks around the Web, send new pages back to be **indexed** by keywords, and revisit old pages because the Web keeps changing. Your query is searched against these keyword indices, and the best matches are returned as "hits".
- **String pattern matching:** document processing (HTML, XML, editors, web search) involves searching text for patterns. A **trie** makes searching a collection of strings fast.

മലയാളത്തിൽ

Word-ൽ തെറ്റായ spelling-ന് താഴെ ചുവന്ന വര വരുന്നത് spell checker ആ വാക്ക് dictionary list-ൽ തിരഞ്ഞ് കിട്ടാത്തതുകൊണ്ടാണ്. Google പോലുള്ള search engines-ന്റെ crawlers web മുഴുവൻ ചുറ്റിനടന്ന് pages-നെ keywords അനുസരിച്ച് index ചെയ്തുവെക്കുന്നു; നമ്മൾ query കൊടുക്കുമ്പോൾ ആ index-ൽ search ചെയ്യുന്നു. Online railway enquiry, database-ൽ നിന്ന് ഒരു record എടുക്കൽ ഇവയെല്ലാം searching-ന്റെ ഉപയോഗങ്ങളാണ്.

EXAMPLE

A trie stores the words CAT, CAR and COW. How is "CAR" found?

From the root follow C → A → R. Each step matches one letter, so the search takes 3 steps (the length of the word) no matter how many words are stored. CAT and CAR share the path C → A, so common prefixes are stored only once.

Internal sorting

Sorting: arranging a list of records in ascending or descending order of their keys.

Internal sorting: sorting in which all the data to be sorted is held in the main memory of the computer (external sorting uses secondary storage).

Sorting: records-ന്റെ list-നെ അവയുടെ keys-ന്റെ ascending അല്ലെങ്കിൽ descending order-ൽ ക്രമീകരിക്കൽ.

Internal sorting: sort ചെയ്യേണ്ട data മുഴുവൻ computer-ന്റെ main memory-യിൽ തന്നെ ഉള്ള sorting (external sorting-ൽ secondary storage ഉപയോഗിക്കുന്നു).

We sort data mainly so that later searches are fast (binary search needs sorted data). The unit studies five internal sorting methods: insertion sort, bubble sort, quick sort, two-way merge sort and heap sort. When comparing them we count key comparisons (and sometimes moves) as a function of n , the number of elements.

മലയാളത്തിൽ

Class-ലെ കട്ടികളെ ഉയരം അനുസരിച്ച് നിരത്തി നിർത്തുന്നത് ഒരു sorting ആണ്. എല്ലാവരും ഒരേ മുറിയിലുണ്ടെങ്കിൽ (main memory) അത് internal sorting. വളരെ വലിയ data ആണെങ്കിൽ, ഒരു ഭാഗം disk-ൽ വെച്ച് കുറച്ചു കുറച്ചായി sort ചെയ്യേണ്ടിവരും; അത് external sorting.

In-place sort: needs only $O(1)$ extra memory (insertion, bubble, heap; quick sort needs only its recursion stack)

Stable sort: equal keys keep their original relative order (insertion, bubble, merge are stable; quick and heap are not)

EXAMPLE

Records (Anu, 70), (Binu, 85), (Cini, 70) are sorted on marks, ascending. Which output does a stable sort give?

(Anu, 70), (Cini, 70), (Binu, 85). Anu and Cini have equal keys, and a stable sort keeps Anu before Cini, as in the input. This matters when you sort on several keys (see the last topic).

All the sorting traces in this unit use the same list so that you can compare the methods: 25, 57, 48, 37, 12, 92, 86, 33. Every trace was produced by running the C code shown.

Insertion sort

Insertion sort: a sorting method that takes the elements one at a time and inserts each into its proper place among the elements already sorted, shifting larger elements one position right.

Insertion sort: elements-നെ ഓരോന്നായി എടുത്ത്, ഇതിനകം sort ചെയ്ത elements-ന്റെ ഇടയിൽ ശരിയായ സ്ഥാനത്ത് insert ചെയ്യുന്ന, വലിയ elements-നെ ഒരു സ്ഥാനം വലത്തേക്ക് നീക്കുന്ന sorting method.

A card player picks up cards one at a time and slides each new card into its right place in his hand. Insertion sort does exactly that. The left part of the array is always sorted. To place the next item (the "target"), scan the sorted part for items greater than the target, shift them one place down, and put the target in the vacated slot. Repeat for every element.

മലയാളത്തിൽ

ചിട്ടകളിക്കുമ്പോൾ കിട്ടുന്ന ഓരോ ചിട്ടും കയ്യിലുള്ള ചിട്ടകൾക്കിടയിൽ ശരിയായ സ്ഥലത്ത് തിരുകി വെക്കുന്നതുപോലെയാണ് insertion sort. ഇടതുവശത്തെ ഭാഗം എപ്പോഴും sorted ആയിരിക്കും. പുതിയ element-നേക്കാൾ വലിയവയെ ഓരോന്നായി വലത്തേക്ക് നീക്കി, ഉണ്ടാകുന്ന ഒഴിവിൽ അതിനെ വെക്കുന്നു.

```
void insertion_sort(int a[], int n)
{
    int i, j, target;
    for (i = 1; i < n; i++) {
        target = a[i];
        j = i - 1;
        while (j >= 0 && a[j] > target) { /* shift bigger items one place right */
            a[j + 1] = a[j];
            j--;
        }
        a[j + 1] = target; /* drop target into the gap */
    }
}
```

EXAMPLE

Sort 25, 57, 48, 37, 12, 92, 86, 33 by insertion sort. The sorted part is shown before the bar |.

Pass	Target	Array after the pass	What happened
Start		25 57 48 37 12 92 86 33	
1	57	25 57 48 37 12 92 86 33	57 > 25, stays
2	48	25 48 57 37 12 92 86 33	57 shifted right
3	37	25 37 48 57 12 92 86 33	57, 48 shifted
4	12	12 25 37 48 57 92 86 33	all 4 shifted, 12 goes first
5	92	12 25 37 48 57 92 86 33	stays
6	86	12 25 37 48 57 86 92 33	92 shifted
7	33	12 25 33 37 48 57 86 92	92, 86, 57, 48, 37 shifted

$n - 1 = 7$ passes give the sorted list.

EXAMPLE

Sort 5, 2, 4, 6, 1, 3.

Pass 1: 2 5 4 6 1 3 → Pass 2: 2 4 5 6 1 3 → Pass 3: 2 4 5 6 1 3 (6 stays) → Pass 4: 1 2 4 5 6 3 → Pass 5: 1 2 3 4 5 6.

Best case (already sorted): $n - 1$ comparisons, no shifts → $O(n)$

Worst case (reverse sorted): $1 + 2 + \dots + (n - 1) = n(n - 1)/2$ comparisons → $O(n^2)$

Average case: about $n^2/4$ comparisons → $O(n^2)$

In-place, stable

Exam tip: Figure 9.1 (the unit's insertion-sort example) is missing in the text, so learn a trace like the one above. Examiners like to ask for the best case: insertion sort is $O(n)$ on an already-sorted list, while the unit's bubble sort is $O(n^2)$ even then.

Bubble sort

Bubble sort: a sorting method that repeatedly compares adjacent elements and swaps them if they are out of order, so that in each pass one element moves to its final position.

Bubble sort: അടുത്തടുത്തുള്ള elements-നെ ആവർത്തിച്ച് താരതമ്യം ചെയ്ത്, ക്രമം തെറ്റിയാൽ swap ചെയ്യുന്ന, ഓരോ pass-ലും ഒരു element അതിന്റെ അവസാന സ്ഥാനത്ത് എത്തുന്ന sorting method.

In IGNOU's version the inner loop runs from the bottom of the list ($j = n$) up to $i + 1$ and swaps $a[j]$ and $a[j - 1]$ when they are out of order. So in pass i the smallest remaining item "bubbles up" to position i , the top of the unsorted part. Many swaps can happen in one pass.

മലയാളത്തിൽ

സോഡാ കുപ്പിയിലെ കുമിളകൾ മുകളിലേക്ക് പൊങ്ങുന്നതുപോലെ, ചെറിയ elements ഓരോ pass-ലും മുകളിലേക്ക് (array-യുടെ തുടക്കത്തിലേക്ക്) കയറുന്നു. എപ്പോഴും അടുത്തടുത്ത രണ്ടെണ്ണം മാത്രമേ താരതമ്യം ചെയ്യൂ. ഒരു pass-ൽ ഒരു swap-ഉം നടന്നില്ലെങ്കിൽ list sorted ആയി എന്ന് ഉറപ്പാണ്, അവിടെ നിർത്താം.

Algorithm BUBBLE SORT (IGNOU, array a[1..n])

for i = 1 to n - 1

for j = n downto i + 1

if a[j] < a[j - 1] then swap(a[j], a[j - 1])

```
void bubble_sort(int a[], int n)
{
    int i, j, temp, swaps;
    for (i = 0; i < n - 1; i++) {
        swaps = 0;
        for (j = n - 1; j > i; j--)
            if (a[j] < a[j - 1]) {
                temp = a[j]; a[j] = a[j - 1]; a[j - 1] = temp;
                swaps++;
            }
        if (swaps == 0) break;          /* already sorted: stop early */
    }
}
```

EXAMPLE

Sort 25, 57, 48, 37, 12, 92, 86, 33 by bubble sort.

Pass 1 in detail (j from the bottom): 86/33 swap → 92/33 swap → 12/33 no → 37/12 swap → 48/12 swap → 57/12 swap → 25/12 swap. The smallest item, 12, has bubbled to the top.

Pass	Array after the pass	Fixed at the top
Start	25 57 48 37 12 92 86 33	
1	12 25 57 48 37 33 92 86	12
2	12 25 33 57 48 37 86 92	12 25
3	12 25 33 37 57 48 86 92	12 25 33
4	12 25 33 37 48 57 86 92	12 25 33 37
5	12 25 33 37 48 57 86 92	no swaps → stop

With the swap flag the program stops after pass 5. IGNOU's algorithm (no flag) would still run passes 6 and 7.

EXAMPLE

Sort 5, 2, 4, 6, 1, 3.

Pass 1: 1 5 2 4 6 3 → Pass 2: 1 2 5 3 4 6 → Pass 3: 1 2 3 5 4 6 → Pass 4: **1 2 3 4 5 6** → Pass 5: no swaps, stop.

Comparisons (IGNOU's version): $(n - 1) + (n - 2) + \dots + 2 + 1 = n(n - 1)/2 = O(n^2)$ in every case

With the "no swap → stop" flag: best case (sorted input) $n - 1$ comparisons → $O(n)$

Average and worst case: $O(n^2)$; in-place, stable

Exam tip: The unit's algorithm swaps when $a[j] \leq a[j - 1]$. That also swaps equal elements, which wastes work and makes the sort unstable, so use $<$. Its loop for $i = 1$ to n also does one useless extra pass, because $n - 1$ passes are enough. Bubble sort is slow because an item moves only one position per comparison.

Quick sort

Quick sort: a divide-and-conquer sort that partitions the list around a chosen element (the pivot) so that smaller keys are before it and larger keys after it, and then sorts the two sublists recursively.

Pivot (dividing element): the element around which the list is partitioned; after partitioning it is in its final sorted position.

Quick sort: ഒരു element-നെ (pivot) അടിസ്ഥാനമാക്കി list-നെ partition ചെയ്ത്, ചെറിയ keys അതിന് മുൻപിലും വലിയവ പിന്നിലും ആക്കി, രണ്ട് sublists-ഉം recursive ആയി sort ചെയ്യുന്ന divide-and-conquer sort.

Pivot (dividing element): list-നെ partition ചെയ്യാൻ ഉപയോഗിക്കുന്ന element; partition കഴിയുമ്പോൾ അത് അതിന്റെ അവസാന sorted സ്ഥാനത്ത് എത്തിയിരിക്കും.

Quick sort was invented by C.A.R. Hoare in 1960 and is the most widely used internal sort. Choose a pivot (here the first element, $A[m]$) and rearrange the list so that the pivot is in its proper place: everything before it is smaller and everything after it is larger. Then apply the same method to sublist 1 (before the pivot) and sublist 2 (after it). The algorithm is naturally recursive.

മലയാളത്തിൽ

ക്ലാസിൽ ഒരു കുട്ടിയെ (pivot) നടുക്ക് നിർത്തി, അവനേക്കാൾ ഉയരം കുറഞ്ഞവരെ ഇടത്തും കൂടിയവരെ വലത്തും നിർത്തുന്നു എന്ന് കരുതുക. ഇപ്പോൾ pivot കുട്ടി ശരിയായ സ്ഥാനത്താണ്. ഇനി ഇടത്തെ കൂട്ടത്തിലും വലത്തെ കൂട്ടത്തിലും ഇതേ പണി ആവർത്തിച്ചാൽ മതി. ഇതാണ് divide and conquer.

Partition (the "divide" step):

1. Take $k = A[m]$ (the first element) as the pivot.
2. Move i right from the left end until an item $\geq$ pivot is found.
3. Move j left from the right end until an item $\leq$ pivot is found.
4. If $i < j$, swap $A[i]$ and $A[j]$.
5. Repeat steps 2–4 until the pointers cross ($i \geq j$).
6. Swap the pivot $A[m]$ with $A[j]$. Now sublist 1 = $A[m..j - 1]$, pivot at j , sublist 2 = $A[j + 1..n]$.
7. Quick sort sublist 1 and sublist 2 in the same way.

Program 9.3 corrected (ANSI C, temp declared, and the i-scan kept inside the array):

```
void quicksort(int A[], int m, int n)
{
    int i, j, k, temp;
    if (m < n) {
        i = m;
        j = n + 1;
        k = A[m]; /* pivot = first element */
        do {
            do ++i; while (i < n && A[i] < k); /* i < n stops running off the end */
            do --j; while (A[j] > k);
            if (i < j) {
                temp = A[i]; A[i] = A[j]; A[j] = temp;
            }
        } while (i < j);
        temp = A[m]; A[m] = A[j]; A[j] = temp; /* pivot to its final place j */
        quicksort(A, m, j - 1);
        quicksort(A, j + 1, n);
    }
}
```

EXAMPLE (FIRST PARTITION IN DETAIL)

Partition 25, 57, 48, 37, 12, 92, 86, 33 (indices 0–7) with pivot 25.

i moves right and stops at 57 ($i = 1$). j moves left past 33, 86, 92 and stops at 12 ($j = 4$). Since $i < j$, swap: 25, 12, 48, 37, 57, 92, 86, 33.

i moves on and stops at 48 ($i = 2$). j moves left from index 4, past 37 and 48, and stops at 12 ($j = 1$). Now $i > j$; the pointers have crossed.

Swap the pivot $A[0]$ with $A[j] = A[1]$: **12, 25, 48, 37, 57, 92, 86, 33**. 25 is in its final place, with sublist 1 = [12] and sublist 2 = [48 ... 33].

EXAMPLE (ALL PARTITIONS)

Call (range)	Pivot	Array after partition	Pivot's final index
Start		25 57 48 37 12 92 86 33	
[0..7]	25	12 25 48 37 57 92 86 33	1
[2..7]	48	12 25 33 37 48 92 86 57	4
[2..3]	33	12 25 33 37 48 92 86 57	2
[5..7]	92	12 25 33 37 48 57 86 92	7
[5..6]	57	12 25 33 37 48 57 86 92	5

Ranges with 0 or 1 element ([0..0], [3..3], [6..6] ...) return at once. Result: 12 25 33 37 48 57 86 92.

Best case (pivot always splits the list into halves): $O(n \log_2 n)$

Average case: $O(n \log_2 n)$ comparisons (about $1.39 n \log_2 n$)

Worst case (list already sorted or reverse sorted, first element as pivot): $n(n - 1)/2$ comparisons $\rightarrow O(n^2)$

In-place (recursion stack $O(\log n)$ on average), not stable

EXAMPLE (WORST CASE)

Quick sort 10, 20, 30, 40, 50 with the first element as pivot.

Pivot 10 stays at index 0, and sublist 2 has 4 items. Pivot 20 stays at index 1, and sublist 2 has 3 items, and so on. Every partition removes only the pivot, so there are $n - 1$ levels and $(n - 1) + (n - 2) + \dots + 1$ comparisons, which is $O(n^2)$.

Two improvements suggested by the unit: (1) switch to a simple sort such as insertion sort when a sublist becomes small; (2) use a better dividing element, for example the median of the first, middle and last items, which avoids the sorted-input worst case. Quick sort can also be written without recursion by using an explicit stack.

Exam tip: Program 9.3 in the unit uses old K&R-style parameters, never declares temp, and writes `if m < n` without brackets. More seriously, its loop `do ++i; while (A[i] < k);` runs past the end of the array when the pivot is the largest item (for example 9, 5, 7). Running it under a memory checker reports an out-of-bounds read. The textbook original assumes a sentinel $A[n + 1] = +\infty$, and the fix above adds the bound $i < n$ instead. The unit mentions only the average $O(n \log n)$. Always also state the worst case, $O(n^2)$.

2-way merge sort

Merging: combining two sorted lists into a single sorted list.

2-way merge sort: a divide-and-conquer sort that treats n items as n sorted lists of size 1 and repeatedly merges pairs of sorted lists until a single sorted list remains.

Merging: sorted ആയ രണ്ട് lists-നെ ചേർത്ത് ഒരു sorted list ഉണ്ടാക്കൽ.

2-way merge sort: n items-നെ size 1 ഉള്ള n sorted lists ആയി കണ്ട്, രണ്ടെണ്ണം വീതം ആവർത്തിച്ച് merge ചെയ്ത്, ഒരു sorted list കിട്ടുന്നതുവരെ തുടരുന്ന divide-and-conquer sort.

A list of one item is already sorted. So we start with n lists of size 1, merge them in pairs to get $n/2$ sorted lists of size 2, merge those to get lists of size 4, and so on until one list is left. Each pass doubles the list size, so there are $\lceil \log_2 n \rceil$ passes, and each pass touches all n items. Merging needs a second array of the same size. Merge sort is the best method for sorting linked lists, and it is also called concatenate sort.

മലയാളത്തിൽ

രണ്ട് ക്യൂ ഉണ്ടെന്ന് കരുതുക, രണ്ടും ഉയരം അനുസരിച്ച് നിരന്നിരിക്കുന്നു. രണ്ടിന്റെയും മുന്നിലുള്ളവരിൽ ഉയരം കുറഞ്ഞയാളെ പുതിയ ക്യൂവിലേക്ക് വിട്ടാൽ, പുതിയ ക്യൂവും sorted ആകും. ഇതാണ് merge. ഒരു ഓരോ elements-ൽ നിന്ന് തുടങ്ങി, 2, 4, 8 എന്നിങ്ങനെ വലിപ്പം ഇരട്ടിപ്പിച്ച് merge ചെയ്യുന്നതാണ് 2-way merge sort.

Program for 2-way merge sort (Check Your Progress 3, Q3). It prints the array after every pass.

```

#include <stdio.h>

void show(const char *label, int a[], int n)
{
    printf("%-10s", label);
    for (int i = 0; i < n; i++) printf(" %3d", a[i]);
    printf("\n");
}

/* merge sorted a[lo..mid] and a[mid+1..hi] into b[lo..hi] */
void merge(int a[], int b[], int lo, int mid, int hi)
{
    int i = lo, j = mid + 1, k = lo;
    while (i <= mid && j <= hi)
        b[k++] = (a[i] <= a[j]) ? a[i++] : a[j++];
    while (i <= mid) b[k++] = a[i++];
    while (j <= hi) b[k++] = a[j++];
}

/* start with n lists of size 1; merge pairs, doubling the size each pass */
void merge_sort(int a[], int n)
{
    int b[100], size, lo, i, pass = 0;
    char lab[24];
    for (size = 1; size < n; size *= 2) {
        for (lo = 0; lo < n; lo += 2 * size) {
            int mid = lo + size - 1, hi = lo + 2 * size - 1;
            if (mid >= n - 1) { /* lone last list: copy as is */
                for (i = lo; i < n; i++) b[i] = a[i];
            } else {
                if (hi > n - 1) hi = n - 1;
                merge(a, b, lo, mid, hi);
            }
        }
        for (i = 0; i < n; i++) a[i] = b[i];
        sprintf(lab, "Pass %d:", ++pass);
        show(lab, a, n);
    }
}

int main(void)
{
    int a[100], n, i;
    printf("How many numbers (1-100)? ");
    if (scanf("%d", &n) != 1 || n < 1 || n > 100) return 1;
    printf("Enter %d numbers: ", n);
    for (i = 0; i < n; i++) scanf("%d", &a[i]);
    printf("\n");
    show("Start:", a, n);
    merge_sort(a, n);
    show("Sorted:", a, n);
}

```

```

    return 0;
}

```

How many numbers (1-100)? 8

Enter 8 numbers: 25 57 48 37 12 92 86 33

```

Start:      25  57  48  37  12  92  86  33
Pass 1:     25  57  37  48  12  92  33  86
Pass 2:     25  37  48  57  12  33  86  92
Pass 3:     12  25  33  37  48  57  86  92
Sorted:     12  25  33  37  48  57  86  92

```

EXAMPLE

Show the passes of 2-way merge sort on 25, 57, 48, 37, 12, 92, 86, 33.

Pass	List size	Sorted lists after the pass
Start	1	[25] [57] [48] [37] [12] [92] [86] [33]
1	2	[25 57] [37 48] [12 92] [33 86]
2	4	[25 37 48 57] [12 33 86 92]
3	8	[12 25 33 37 48 57 86 92]

Merging [25 37 48 57] with [12 33 86 92] in pass 3: $12 < 25 \rightarrow 12$; $25 < 33 \rightarrow 25$; $33 < 37 \rightarrow 33$; $37 \rightarrow 37$; $48 \rightarrow 48$; $57 \rightarrow 57$; the first list is now empty, so copy 86, 92. That is 6 comparisons for 8 items.

EXAMPLE (ODD N)

Sort 38, 27, 43, 3, 9, 82, 10 ($n = 7$).

Pass 1: [27 38] [3 43] [9 82] [10] (10 has no partner and is copied as is).

Pass 2: [3 27 38 43] [9 10 82].

Pass 3: [3 9 10 27 38 43 82]. $\lceil \log_2 7 \rceil = 3$ passes.

Passes = $\lceil \log_2 n \rceil$; each pass does $O(n)$ work

Best = average = worst case: $O(n \log_2 n)$

Extra space: a second array of n elements (space for $2n$ in total); stable

Exam tip: The unit repeats the same two paragraphs twice, and Figure 9.2 is missing, so draw your own pass table as above. Remember the one real drawback of merge sort: it needs $O(n)$ extra memory, while heap sort is also $O(n \log n)$ but in-place.

Heap: definition, array form and construction

Heap condition: the key of each node is greater than or equal to the keys of its children, so the root has the largest key.

Heap: a complete binary tree in which every node satisfies the heap condition, stored as an array.

Heap condition: ഓരോ node-ന്റെയും key അതിന്റെ children-ന്റെ keys-നേക്കാൾ വലുതോ തുല്യമോ ആയിരിക്കണം; അതിനാൽ root-ൽ ഏറ്റവും വലിയ key വരും.

Heap: എല്ലാ node-ഉം heap condition പാലിക്കുന്ന, array ആയി സൂക്ഷിക്കുന്ന complete binary tree.

Number the nodes of a complete binary tree level by level, left to right, starting from 1 at the root. Store the node numbered j in array position j . No pointers are needed, because the family links follow from the positions.

Node at position j : left child at $2j$, right child at $2j + 1$, parent at $\lfloor j/2 \rfloor$ (1-based numbering)

(For a 0-based C array: children $2j + 1$ and $2j + 2$, parent $\lfloor (j - 1)/2 \rfloor$)

മലയാളത്തിൽ

ഒരു ഓഫീസിലെ hierarchy ഓർക്കുക: ഓരോ boss-ഉം തന്റെ കീഴിലുള്ളവരെക്കാൾ "വലിയ" ആൾ. അങ്ങനെ ഏറ്റവും മുകളിൽ (root) ഏറ്റവും വലിയ ആൾ. ഇതാണ് heap condition. Tree-യെ array-യിൽ ഇടാൻ nodes-ന് 1, 2, 3... എന്ന് നമ്പർ ഇട്ടാൽ മതി; j -ാം node-ന്റെ മക്കൾ $2j$ -ലും $2j+1$ -ലും, parent $j/2$ -ലും.

EXAMPLE

In a heap array, a node is at position 5. Where are its parent and children?

Parent = $\lfloor 5/2 \rfloor = 2$. Children = $2 \times 5 = 10$ and $2 \times 5 + 1 = 11$. (This is the unit's example: node M at position 5 has parent R at 2 and children E and I at 10 and 11. The figure itself is missing from the text.)

Operations on a heap work in two steps: (1) insert, delete or replace the node; (2) since this may break the heap condition, walk the tree and repair it. For **insertion**, the new item goes into the next free position (the last position). While it is larger than its parent, it swaps with the parent (it "moves up"). For **deleting the root**, the last item is moved to the root and then swapped with its larger child while it is smaller than that child (it "moves down").

EXAMPLE (INSERTION)

Insert 70 into the heap [81, 64, 36, 16, 49, 25, 2, 3, 9, 4] (positions 1–10).

70 goes to position 11. Its parent is $\lfloor 11/2 \rfloor = 5$, holding 49. $70 > 49$, so they swap and 70 is now at 5.

Its parent is 2, holding 64. $70 > 64$, so they swap and 70 is now at 2.

Its parent is 1, holding 81. $70 < 81$, so stop.

New heap: [81, 70, 36, 16, 64, 25, 2, 3, 9, 4, 49].

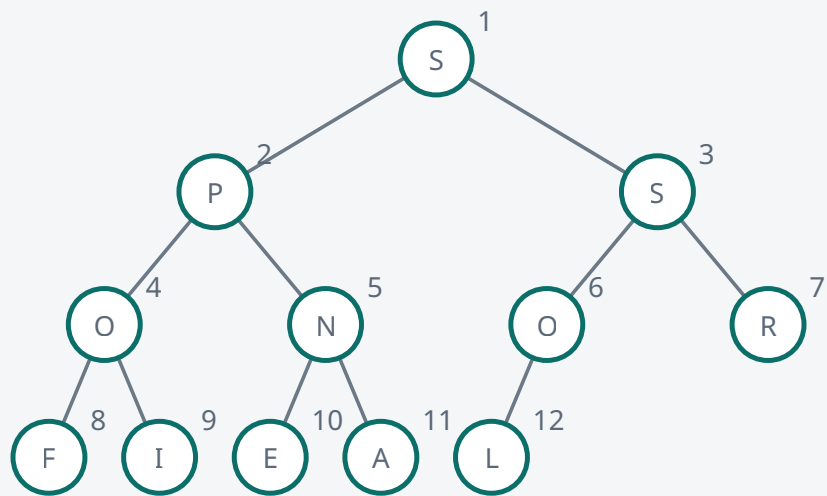
Two ways to build a heap:

- **Top-down construction:** insert the items one by one into an initially empty heap, restoring the heap condition (moving the new item up) after each insertion. This takes $O(n \log n)$ time.
- **Bottom-up construction:** put all items into the tree in the order given, then, starting from the rightmost non-leaf node (position $\lfloor n/2 \rfloor$) and moving back to the root, move each node down until the heap condition holds. This takes $O(n)$ time.

EXAMPLE (TOP-DOWN: PROFESSIONAL)

Build a heap by inserting the letters P R O F E S S I O N A L one at a time (alphabetical order: later letter = larger key).

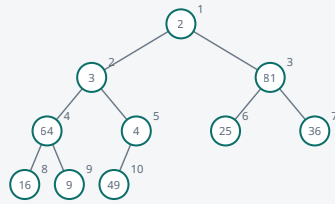
Insert	Placed at	Moves up?	Heap array (positions 1...)
P	1	root	P
R	2	$R > P \rightarrow$ to 1	R P
O	3	$O < R$, stays	R P O
F	4	$F < P$, stays	R P O F
E	5	$E < P$, stays	R P O F E
S	6	$S > O \rightarrow 3$; $S > R \rightarrow 1$	S P R F E O
S	7	$S > R \rightarrow 3$; $S = S$, stop	S P S F E O R
I	8	$I > F \rightarrow 4$; $I < P$, stop	S P S I E O R F
O	9	$O > I \rightarrow 4$; $O < P$, stop	S P S O E O R F I
N	10	$N > E \rightarrow 5$; $N < P$, stop	S P S O N O R F I E
A	11	$A < N$, stays	S P S O N O R F I E A
L	12	$L < O$, stays	S P S O N O R F I E A L



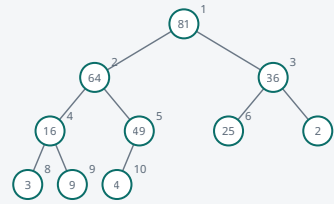
Final top-down heap for PROFESSIONAL (small numbers are array positions).

EXAMPLE (BOTTOM-UP: IGNOU'S FILE)

Build a heap from 2, 3, 81, 64, 4, 25, 36, 16, 9, 49 (n = 10).



File as a binary tree (Figure 9.7).



After bottom-up construction (Figure 9.8).

Non-leaf positions are $\lfloor 10/2 \rfloor = 5$ down to 1.

Position	Node	Action	Array afterwards
Start			2 3 81 64 4 25 36 16 9 49
5	4	child 49 larger → swap	2 3 81 64 49 25 36 16 9 4
4	64	children 16, 9 smaller → ok	no change
3	81	children 25, 36 smaller → ok	no change
2	3	larger child 64 → swap; then larger child 16 → swap	2 64 81 16 49 25 36 3 9 4
1	2	larger child 81 → swap; then larger child 36 → swap	81 64 36 16 49 25 2 3 9 4

Heap: 81, 64, 36, 16, 49, 25, 2, 3, 9, 4, which matches the unit's Figure 9.8.

Exam tip: A heap must be a complete binary tree. Always fill levels left to right, and never leave a gap. Only a parent-child order is required: siblings (like 64 and 36) can be in any order. The unit's steps for inserting R are numbered "1, 2, 3, 4, 5, 4" and refer to missing figures, so practise with a full example like the one above.

Heap sort

Heap sort: a sorting method that first builds a heap of the n items and then repeatedly removes the root (the largest key), placing it at the end of the array and restoring the heap on the remaining items.

Heap sort: ആദ്യം n items കൊണ്ട് ഒരു heap ഉണ്ടാക്കി, പിന്നീട് root (ഏറ്റവും വലിയ key) ആവർത്തിച്ച് എടുത്ത് array-യുടെ അവസാനത്ത് വെച്ച്, ബാക്കിയുള്ളവയിൽ heap വീണ്ടും ശരിയാക്കുന്ന sorting method.

The root of a heap is always the largest item. Swap it with the last item of the heap: the largest is now in its final place at the end. The heap is one smaller, and the new root may break the heap condition, so move it down. Repeat until the heap has one item. The items come out in the order largest, next largest, ..., and the array ends up in ascending order.

മലയാളത്തിൽ

ഒരു മത്സരത്തിൽ ഓരോ റൗണ്ടിലും ഏറ്റവും മികച്ചയാളെ (root) പുറത്തെടുത്ത് winners list-ൽ ചേർക്കുന്നു; ബാക്കിയുള്ളവരിൽ നിന്ന് അടുത്ത മികച്ചയാൾ മുകളിലേക്ക് വരുന്നു. ഇങ്ങനെ എല്ലാവരും പുറത്തുവരുമ്പോൾ list sorted ആകും. Heap sort-ന് extra array വേണ്ട, അതേ array-യിൽ തന്നെ (in-place) sort നടക്കുന്നു.

```
/* a[1..n] is used; a[0] is ignored */
void sift_down(int a[], int j, int n)
{
    int x = a[j], c;
    while ((c = 2 * j) <= n) {
        if (c < n && a[c + 1] > a[c]) c++;    /* larger child */
        if (x >= a[c]) break;
        a[j] = a[c];                        /* move child up */
        j = c;
    }
    a[j] = x;
}

void heap_sort(int a[], int n)
{
    int j, size, t;
    for (j = n / 2; j >= 1; j--)            /* bottom-up heap construction */
        sift_down(a, j, n);
    for (size = n; size > 1; size--) {
        t = a[1]; a[1] = a[size]; a[size] = t; /* largest to the end */
        sift_down(a, 1, size - 1);
    }
}
```

EXAMPLE (IGNOU'S FIGURE 9.9)

Sort the heap 81, 64, 36, 16, 49, 25, 2, 3, 9, 4. The heap part is before the bar |, and the sorted part after it.

Remove	Heap size	Array after restoring the heap	Output so far
	10	81 64 36 16 49 25 2 3 9 4	
81	9	64 49 36 16 4 25 2 3 9 81	81
64	8	49 16 36 9 4 25 2 3 64 81	81, 64
49	7	36 16 25 9 4 3 2 49 64 81	81, 64, 49
36	6	25 16 3 9 4 2 36 49 64 81	... 36
25	5	16 9 3 2 4 25 36 49 64 81	... 25
16	4	9 4 3 2 16 25 36 49 64 81	... 16
9	3	4 2 3 9 16 25 36 49 64 81	... 9
4	2	3 2 4 9 16 25 36 49 64 81	... 4
3	1	2 3 4 9 16 25 36 49 64 81	... 3, 2

First step in detail: swap 81 with the last item 4, giving 4 64 36 16 49 25 2 3 9 | 81. Move 4 down: its larger child is 64, so swap; at position 2 its larger child is 49, so swap; at position 5 it has no children in the heap. The heap is now 64 49 36 16 4 25 2 3 9, exactly as in the unit's figure. Sorted result: **2, 3, 4, 9, 16, 25, 36, 49, 64, 81**.

EXAMPLE

Heap sort 4, 10, 3, 5, 1.

Build the heap bottom-up: position 2 (10) is fine; position 1 (4) swaps with its larger child 10, then with 5, giving **10, 5, 3, 4, 1**.

Remove 10: 5 4 3 1 | 10. Remove 5: 4 1 3 | 5 10. Remove 4: 3 1 | 4 5 10. Remove 3: 1 | 3 4 5 10.

Result: **1, 3, 4, 5, 10**.

Build heap (bottom-up): $O(n)$; each of the $n - 1$ removals: $O(\log_2 n)$

Best = average = worst case: $O(n \log_2 n)$; in-place, not stable

All methods at a glance

Method	Best	Average	Worst	Extra space	Stable
Linear search	$O(1)$	$O(n)$	$O(n)$	$O(1)$	–
Binary search	$O(1)$	$O(\log n)$	$O(\log n)$	$O(1)$	–
Insertion sort	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	yes
Bubble sort (with flag)	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	yes
Quick sort	$O(n \log n)$	$O(n \log n)$	$O(n^2)$	$O(\log n)$ stack	no
2-way merge sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(n)$	yes
Heap sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(1)$	no

Exam tip: The unit lists the removed items as "Sorted: 81, 64, 49, ...", which is descending order. In the array, the same process leaves the items in ascending order, because each maximum goes to the end. Say which one you mean. The unit's figure captions also mention "figure 10.3" and "figure 10.8". These are typos for Figures 9.3 and 9.8.

Sorting on several keys

Sorting on several keys: ordering records on a primary key and, among records with equal primary keys, on a secondary key (and so on).

MSD sort: sorts first on the most significant key, then within each group on the next key. **LSD sort:** sorts first on the least significant key, and last on the most significant key.

Sorting on several keys: records-നെ ആദ്യം primary key അനുസരിച്ചും, primary key തുല്യമായവയെ secondary key അനുസരിച്ചും (അങ്ങനെ തുടർന്ന്) ക്രമീകരിക്കൽ.

MSD sort: ആദ്യം most significant key അനുസരിച്ച് sort ചെയ്ത്, ഓരോ group-നുള്ളിലും അടുത്ത key അനുസരിച്ച് sort ചെയ്യുന്നു. **LSD sort:** ആദ്യം least significant key അനുസരിച്ചും ഏറ്റവും അവസാനം most significant key അനുസരിച്ചും sort ചെയ്യുന്നു.

A deck of 52 cards is sorted on two keys: suit first (clubs, spades, diamonds, hearts), then face value within each suit (Ace, 2, ..., King). There are two ways to do it:

- **MSD (Most Significant Digit) sort:** sort the 52 cards into 4 piles by suit, then sort each pile by face value.
- **LSD (Least Significant Digit) sort:** sort the 52 cards into 13 piles by face value, stack the piles in order, and then sort into 4 piles by suit.

"Digit" here means a key. MSD and LSD only decide the order in which keys are used. The actual sorting on each key can be done by any of the methods in this unit, but for LSD the later passes must be **stable**, or they will destroy the order created by the earlier passes.

മലയാളത്തിൽ

Exam result list ഉണ്ടാക്കുമ്പോൾ ആദ്യം programme അനുസരിച്ചും, ഒരേ programme-ലുള്ളവരെ enrolment number അനുസരിച്ചും അടുക്കുന്നത് several keys sorting ആണ്. MSD-ൽ വലിയ key (suit) ആദ്യം, എന്നിട്ട് ഓരോ കൂട്ടത്തിലും ചെറിയ key. LSD-ൽ നേരെ തിരിച്ച്: ആദ്യം ചെറിയ key (face value), അവസാനം വലിയ key. LSD ശരിയാകണമെങ്കിൽ അവസാന pass stable sort ആയിരിക്കണം.

EXAMPLE (LSD ON TWO-DIGIT NUMBERS)

Sort 36, 91, 13, 95, 31, 16 with LSD order (units digit first, then tens digit).

Pass 1 (by units digit, stable): 91, 31, 13, 95, 36, 16.

Pass 2 (by tens digit, stable): 13, 16, 31, 36, 91, 95 → **sorted**. In pass 2, 31 stays before 36 because pass 1 had already put them in units order, and a stable sort keeps that order.

EXAMPLE (MSD)

Sort the same numbers with MSD order.

Group by tens digit: {13, 16}, {36, 31}, {91, 95}. Sort each group by the units digit: {13, 16}, {31, 36}, {91, 95}. Concatenate: **13, 16, 31, 36, 91, 95**.

Exam tip: Don't mix up the two names. **MSD** starts with the **M**ost significant key and gives piles that are then sorted separately. **LSD** starts with the least significant key and needs the whole list re-sorted (stably) on each more significant key. Radix sort is the LSD method applied to the digits of numbers.

Quick recap

- File = records; record = fields; the key is the field used to search a record.
- Linear search works on unsorted data: best 1 comparison, average $(n + 1)/2$, worst n , so $O(n)$.
- Binary search needs a sorted array: $\text{mid} = (\text{low} + \text{high})/2$, discard half each time, worst $\lfloor \log_2 n \rfloor + 1$ comparisons, so $O(\log n)$.
- For a descending array, binary search just swaps the direction tests ($\text{key} > k[\text{mid}] \rightarrow \text{go left}$).
- Applications of searching: spell checkers (sequential look-up), search engines (crawlers + keyword index), string pattern matching (tries).
- Internal sorting keeps all the data in main memory; external sorting uses disk.

- Insertion sort inserts each item into the sorted left part: $O(n)$ best, $O(n^2)$ average and worst, stable.
- Bubble sort swaps adjacent items: $n(n - 1)/2$ comparisons = $O(n^2)$; with a no-swap flag the best case is $O(n)$.
- Quick sort (Hoare, 1960) partitions around a pivot and uses divide and conquer: $O(n \log n)$ average, $O(n^2)$ worst (sorted input with first-element pivot).
- 2-way merge sort merges runs of size 1, 2, 4, ... in $\lceil \log_2 n \rceil$ passes: $O(n \log n)$ always, and needs an extra array.
- Heap = complete binary tree with parent $\geq$ children, stored as an array: children of j at $2j$ and $2j + 1$, parent at $\lfloor j/2 \rfloor$.
- Top-down heap building inserts items one by one ($O(n \log n)$); bottom-up building repairs from position $\lfloor n/2 \rfloor$ back to 1 ($O(n)$).
- Heap sort repeatedly swaps the root with the last item and moves it down: $O(n \log n)$ in every case, in-place.
- Several keys: MSD sorts on the most significant key first, LSD on the least significant first (it needs stable passes).

Check yourself

Q. 1. Linear search checks each element against the key and halts at a match. Will sorting the array before a linear search change its order of efficiency?

No. In the worst case the key may still be at the last position, or absent, so up to n comparisons are needed and the order stays $O(n)$. On a sorted array an unsuccessful search can stop early, once it passes an element larger than the key, but that only changes the constant, not the order. To gain real speed you must use binary search on the sorted array.

Q. 2. In the best case the element is found with the fewest comparisons. Where in the list is the key, and how many comparisons are made?

At the **beginning of the list** (index 0). One comparison is enough, so the best case is $O(1)$.

Q. 3. True or False: (a) the order of linear search in the worst case is $O(n/2)$; (b) linear search is more efficient than binary search; (c) for binary search, the array has to be sorted in ascending order only.

(a) **False.** The worst case is n comparisons, which is $O(n)$. Even the average $(n + 1)/2$ is $O(n)$, because constants are dropped in O -notation.

(b) **False.** Binary search is $O(\log n)$ and linear search is $O(n)$. Linear search wins only for very small or unsorted lists searched once.

(c) **False.** Binary search works on descending arrays too, if the comparisons are reversed.

Q. 4. Trace binary search for key = 60 in [5, 12, 19, 27, 33, 41, 58, 60, 72]. How many comparisons are needed?

low = 0, high = 8: mid = 4, k[4] = 33, $60 > 33 \rightarrow \text{low} = 5$.

low = 5, high = 8: mid = 6, k[6] = 58, $60 > 58 \rightarrow \text{low} = 7$.

low = 7, high = 8: mid = 7, k[7] = 60 $\rightarrow$ found at position $7 + 1 = 8$.

3 comparisons, within the worst case $\lfloor \log_2 9 \rfloor + 1 = 4$.

Q. 5. Write the binary search algorithm for an array sorted in descending order, and use it to find 30 in [90, 75, 60, 45, 30, 15].

low $\leftarrow$ 0, high $\leftarrow$ n - 1. While low $\leq$ high: mid $\leftarrow$ (low + high)/2. If key = k[mid], report position mid + 1 and stop. Else if key > k[mid], high $\leftarrow$ mid - 1 (larger keys are to the left). Else low $\leftarrow$ mid + 1. If the loop ends, report "not found".

Trace: mid = (0 + 5)/2 = 2, k[2] = 60, $30 < 60 \rightarrow \text{low} = 3$. mid = (3 + 5)/2 = 4, k[4] = 30 $\rightarrow$ found at position 5.

Q. 6. Sort 5, 2, 4, 6, 1, 3 by insertion sort and by bubble sort, showing each pass. What is the complexity of bubble sort?

Insertion: 2 5 4 6 1 3 $\rightarrow$ 2 4 5 6 1 3 $\rightarrow$ 2 4 5 6 1 3 $\rightarrow$ 1 2 4 5 6 3 $\rightarrow$ **1 2 3 4 5 6**.

Bubble (smallest bubbles to the front): 1 5 2 4 6 3 $\rightarrow$ 1 2 5 3 4 6 $\rightarrow$ 1 2 3 5 4 6 $\rightarrow$ **1 2 3 4 5 6** $\rightarrow$ no swaps in pass 5, stop.

Bubble sort makes $(n - 1) + (n - 2) + \dots + 1 = n(n - 1)/2$ comparisons, so it is $O(n^2)$, where n is the number of elements ($O(n)$ best case only with a no-swap flag).

Q. 7. Which programming technique does quick sort use? Show the first partition of 44, 33, 11, 55, 77, 90, 40, 60, 99, 22, 88, 66 with the first element as pivot.

Quick sort uses **divide and conquer** (and recursion).

Pivot 44. i stops at 55, j stops at 22: swap $\rightarrow$ 44 33 11 22 77 90 40 60 99 55 88 66.

i stops at 77, j stops at 40: swap $\rightarrow$ 44 33 11 22 40 90 77 60 99 55 88 66.

i stops at 90 (index 5), j stops at 40 (index 4): the pointers have crossed. Swap the pivot with A[4]:

40 33 11 22 44 90 77 60 99 55 88 66. 44 is in its final place, with sublist 1 = 40 33 11 22 and sublist 2 = 90 77 60 99 55 88 66.

Q. 8. Write a program in C for 2-way merge sort and show its passes on 38, 27, 43, 3, 9, 82, 10.

Use the program in the 2-way merge sort topic. `merge()` combines two sorted runs into a second array, and `merge_sort()` merges runs of size 1, 2, 4, ... until the run size reaches n .

Pass 1: 27 38 | 3 43 | 9 82 | 10. Pass 2: 3 27 38 43 | 9 10 82. Pass 3: **3 9 10 27 38 43 82**. The time is $O(n \log n)$, and the extra space is $O(n)$.

Q. 9. What is the complexity of heap sort? Build a heap (bottom-up) from 4, 10, 3, 5, 1 and sort it.

Heap sort is $O(n \log n)$ in the best, average and worst case, where n is the number of elements.

Build: position 2 (10, children 5 and 1) is fine. Position 1 (4): its larger child is 10, so swap; at position 2 its larger child is 5, so swap. Heap = 10 5 3 4 1.

Sort: 5 4 3 1 | 10 → 4 1 3 | 5 10 → 3 1 | 4 5 10 → 1 | 3 4 5 10 → **1 3 4 5 10**.

Q. 10. Explain MSD and LSD sorting using a deck of cards.

The keys are suit (most significant) and face value (least significant). **MSD**: deal the 52 cards into 4 suit piles, then sort each pile by face value, and put the piles together. **LSD**: deal the cards into 13 face-value piles, stack them in order, then (stably) deal them into 4 suit piles. Each suit pile is then already in face-value order. MSD and LSD only fix the order in which the keys are used; any sorting method can do the actual sorting on each key.

Based on the IGNOU unit texts. Check definitions against the PDFs before the exam.